Daily TOEIC

토익 스터디 캘린더 앱 — 개발 정리 및 기능별 코드

작성일: 2026 년 10 월 15 일

프로젝트 폴더: C:₩Users₩kch36₩Desktop₩DailyToeic

결과물: builds₩DailyTOEIC.apk (Android 7.0 이상)

목차

1. 개요 3
    1.1 주요 기능 한눈에 보기 3
    1.2 기술 구성 3
    1.3 폴더 구조 3
2. 개발 진행 내역 5
3. 기능별 설명과 핵심 코드 7
    3.1 앱 구조 — App · Store · Router 7
    3.2 캘린더 · 1 년 보기 · 핀치 줌 8
    3.3 날짜 바텀시트 9
    3.4 기록 화면 · 공부한 시간 화면 10
    3.5 뽀모도로 타이머 (집중 → 휴식 → 다시 집중) 10
    3.6 타임타이머 다이얼 11
    3.7 효과음 (뜨르륵 · 삐삐삐) 과 오디오 연결 12
    3.8 앱을 닫아도 울리는 알람 (안드로이드) 13
    3.9 통계 (이 주 요약 · 이 달 요약) 14
    3.10 꿀팁 배너 · 시험 카운트다운 15
    3.11 테마 · 설정 · 백업 16
    3.12 계정 로그인 · 클라우드 동기화 (환경만 구축) 16
    3.13 안드로이드 앱 (APK) 17
4. 알려진 제한과 다음 할 일 19
부록. 전체 소스 코드 20
    A.1 공통 기반 (core) 20
    A.2 홈 · 캘린더 31
    A.3 기록 · 통계 40
    A.4 타이머 · 소리 · 알람 47
    A.5 설정 · 테마 · 계정 59
    A.6 앱 시작 · 화면 · 스타일 61
    A.7 소리 데이터 77

1. 개요

Daily TOEIC 은 매일의 토익 공부를 캘린더에 기록하고, 타임타이머 스타일의 뽀모도로 타이머로 공부 시간을 재는 앱입니다. 웹 브라우저에서도 동작하고, Capacitor 로 감싼 안드로이드 앱(APK)으로도 설치할 수 있습니다.

1.1 주요 기능 한눈에 보기

- **캘린더**: 공부한 양을 색 진하기로 표시, 두 손가락으로 오므리면 1 년 학습량 보기
- **날짜 바텀시트**: 날짜를 누르면 공부 시작 / 통계 / 기록 버튼 (날짜 종류에 따라 바뀜)
- **뽀모도로 타이머**: 원판이 줄어드는 타임타이머, 집중이 끝나면 쉰 만큼 차오르는 휴식
- **효과음**: 원판을 돌릴 때 태엽 소리(뜨르륵), 시작·끝 삐삐삐
- **앱을 닫아도 알람**: 끝나는 시각에 휴대폰 시스템 알림 예약
- **기록·통계**: 날짜별 기록, 이 주/이 달 요약, 점수 추이, 파트별 학습 횟수
- **시험 D-day**: 시험일 지정, 요약 카드에 일·시·분·초 카운트다운
- **꿀팁 배너**: 앱 사용 팁과 응원 문구가 터치할 때마다 바뀜
- **테마·설정·백업**, 계정 로그인/클라우드 동기화(환경만 구축, 아직 꺼져 있음)

1.2 기술 구성

구분	내용
화면	HTML + CSS + 순수 JavaScript (프레임워크 없음)
구조	기능별 클래스 파일 25 개, window.DT 네임스페이스에 등록 (파일로 열어도 동작하도록 ES 모듈 대신 사용)
저장	localStorage (기기 안). 클라우드 동기화는 Firebase 로 준비만 되어 있음
소리	Web Audio API (태엽 소리는 원본 mp3 에서 잘라낸 WAV, 삐삐삐는 코드로 합성)
안드로이드	Capacitor 8.5 + @capacitor/local-notifications 8.3 (minSdk 24, targetSdk 36)
오프라인	서비스 워커(웹에서만). 앱 안에서는 사용하지 않음

1.3 폴더 구조

```
DailyToeic/
├ www/                      ← 앱 화면 (웹·안드로이드 공통)
│  ├ index.html, style.css, manifest.json, icon.svg, service-worker.js
│  ├ sounds/wind-up.js      ← 태엽 "뜨르륵" 소리 (base64 WAV)
```

```
│  └ js/
│     ├ app.js              ← App: 모든 객체를 만들고 연결
│     ├ core/               ← 공통 기반 (저장소, 라우터, 오디오, 계정/동기화)
│     └ features/           ← 기능별 화면·로직
├ android/                  ← Capacitor 안드로이드 프로젝트
├ builds/DailyTOEIC.apk     ← 빌드 결과
├ docs/CLOUD_SETUP.md       ← 계정 연동 켜는 방법
├ firestore.rules           ← 클라우드 보안 규칙
├ capacitor.config.json, package.json
```

2. 개발 진행 내역

요청 순서대로 무엇을 바꿨는지 정리했습니다. 각 기능의 자세한 동작과 코드는 3 장에 있습니다.

#	요청	반영 내용
1	매일 토익 기록을 남기는 스터디 캘린더	월간 캘린더(학습량 색), 날짜별 기록(시간·파트·단어·모의고사·컨디션·메모), 통계, JSON 백업, PWA
2	밝은/어두운 테마, 뽀모도로 타이머	🎨 테마 전환, 집중/휴식 타이머, 끝나면 공부 내용 기록 창
3	타임타이머처럼 색이 줄어드는 디자인	60 분 눈금 원판, 남은 시간만큼 색 원판, 드래그로 시간 설정, 색상 5 종
4	시험 D-day, 탭 버튼 제거	시험일 지정 → D-day 배너/칸 표시, 한 페이지 구성
5	한 화면 구성, 날짜 바텀시트, 기능별 객체화	홈 100dvh 한 화면, 공부 시작/통계/기록 시트, 클래스 파일 분리 + 라우터
6	공부 시작은 오늘만, 지난 날은 기록된 시간 보기	지난 날: '공부한 시간' 화면(타이머 기록만, 직접 입력 불가)
7	미래 날짜는 '아직 오픈 전'	공사 중 줄무늬 버튼 + 누르면 랜덤 농담·흔들림
8	두 손가락 줌 아웃으로 1 년 학습량	핀치 제스처(CalendarZoom) + 12 개월 히트맵(YearView), PC 는 Ctrl+휠
9	타이머 조절 시 mp3 의 뜨르륵 소리	mp3 분석 → 0.47~0.80 초 구간 추출, 1 분마다 조각 재생
10	집중/휴식 버튼 없애고 자동 전환	끝나면 색이 바뀌며 원판이 차오르는 자동 전환
11	휴식은 설정 없이 쉰 만큼 기록, 다시 집중	휴식 = 스톱워치, '다시 집중'으로 시작, 휴식 시간 기록
12~15	효과음 조정	끝: 삐삐삐 1 번 / 시작: 사인파·반음 7 개 위·3 배속, 기록 칸 터치로 편집
16	사이트에서 조절 소리 안 남	AudioHub: 첫 탭에 오디오 깨우기 (자동재생 정책 대응)
17	미래는 오픈 전만, 토요일 이 주 요약, 마지막 날 이 달 요약	날짜별 버튼 구성, 주간 요약 + 요일 그래프, 색 구분(파랑/빨강)
18	꿀팁·동기부여 배너	TipBanner: 팁 11 + 응원 13 + 내 기록 문구, 터치마다 변경
19	시험 카드: 일·시·분·초, 한 줄	시험 날 0 시 기준 실시간 카운트다운, 글자 자동 맞춤
20	계정 로그인 환경 구축	Firebase 기반 AccountService/CloudSync/AccountPanel (꺼둔 상태)

#	요청	반영 내용
21	APK 만들기	www/ 분리, Capacitor 8 안드로이드 프로젝트, 아이콘·스플래시, 빌드
22	앱 닫아도 알람, 앱에서 뜨르륵 안 남	시스템 알림 예약(정확한 알람) + WebView 오디오 연결 끊김 해결, 에뮬레이터 검증

3. 기능별 설명과 핵심 코드

각 기능의 동작 방식과 담당 파일·클래스, 그리고 가장 중요한 부분의 코드를 실제 소스에서 그대로 옮겼습니다. 파일 전체 코드는 부록에 있습니다.

3.1 앱 구조 — App · Store · Router

- **App** 이 모든 기능 객체를 만들고 서로 연결합니다. 기능끼리는 직접 부르지 않고 Store 와 Router 를 통해 이어집니다.
- **Store** 는 날짜별 기록과 설정을 localStorage 에 저장하고, 데이터가 바뀌면 onChange 로 등록한 화면들에 알립니다.
- **Router** 는 주소의 #해시로 화면을 관리합니다(#day/날짜, #timer, #stats/날짜, #record/날짜, #studylog/날짜). 휴대폰 뒤로가기 버튼이 그대로 동작합니다.

담당 파일

파일	클래스	역할
www/js/app.js	App	객체 생성·연결
www/js/core/store.js	Store	기록·설정 저장, 변경 알림
www/js/core/router.js	Router	화면 전환·뒤로가기
www/js/core/utils.js	Toast 등	날짜·시간 포맷, 공통 함수

www/js/app.js — constructor

```
constructor() {
  this.toast = new DT.Toast($("toast"));
  this.store = new DT.Store((msg) => this.toast.show(msg));
  this.router = new DT.Router();


  this.theme = new DT.ThemeManager($("btnTheme"), this.toast);
  this.settings = new DT.SettingsDialog(this.store, this.toast);


  // 계정 로그인 + 클라우드 동기화 (cloud-config.js 의 enabled 가 false 면 아무 동작 안 함)
  this.account = new DT.AccountService(DT.CLOUD_CONFIG);
  this.sync = new DT.CloudSync(this.store, this.account);
  this.accountPanel = new DT.AccountPanel(this.account, this.sync);
  this.account.init();
  this.sessionDialog = new DT.SessionDialog(this.store, this.toast);


  this.calendar = new DT.CalendarView(this.store, (k) => this.router.go("day", k));
  this.tips = new DT.TipBanner(this.store);
  this.sheet = new DT.DaySheet(this.store, {
    study: () => this.router.go("timer"),
    studyLog: (k) => this.router.go("studylog", k),
    stats: (k, scope) => this.router.go("stats", scope === "week" ? `${k}~week` : k),
    record: (k) => this.router.go("record", k),
    close: () => this.router.back(),
    toast: (msg) => this.toast.show(msg),
  });
  this.timer = new DT.TimerScreen(this.store, this.sessionDialog, this.toast, this.router);
  this.stats = new DT.StatsScreen(this.store, (k) => this.router.go("record", k));
  this.record = new DT.RecordScreen(this.store, this.sessionDialog, this.toast, this.router);
```

```
  this.studyLog = new DT.StudyLogScreen(this.store, this.sessionDialog);


  // 각 화면을 라우터에 등록 (홈 캘린더는 항상 아래에 깔려 있음)
  [this.sheet, this.timer, this.stats, this.record, this.studyLog].forEach((v) => this.router.register(v));
  // 선택된 날짜 표시는 시트/기록/통계 화면이 열려 있는 동안 유지
  this.router.register({ name: "__selection", setActive: () => {
    const { name, param } = this.router.current;
    const k = param && param.split("~")[0];
    this.calendar.setSelected(["day", "stats", "record", "studylog"].includes(name) ? k : null);
  } });


  // 화면 왼쪽 위 ‹ 버튼
  document.querySelectorAll("[data-back]").forEach((b) => b.addEventListener("click", () =>
this.router.back()));


  this.calendar.render();
  this.router.start();
  this.registerServiceWorker();
}
```

www/js/core/store.js — addSession

```
addSession(start, end) {
  const k = keyOf(new Date(start));
  const minutes = Math.max(1, Math.round((end - start) / 60000));
  const r = this.records[k] || emptyRecord();
  r.sessions = r.sessions || [];
  const s = { id: String(end), start, end, minutes, parts: [], note: "" };
  r.sessions.push(s);
  r.minutes = (r.minutes || 0) + minutes;
  this.records[k] = r;
  this.persistRecords();
  return { k, s };
}
```

www/js/core/router.js — sync

```
  window.addEventListener("popstate", () => this.sync());
}


// view: { name, setActive(active, param) }
```

3.2 캘린더 · 1 년 보기 · 핀치 줌

- 홈은 화면 높이(100dvh)에 맞춰 스크롤 없이 한 화면에 들어갑니다. 주 수(4~6 주)에 맞춰 칸 높이를 나눕니다.
- 공부 시간이 하루 목표 대비 많을수록 칸 색이 진해지고(5 단계), 목표 달성 ✓, 모의고사 점, 시험일 D-day 가 표시됩니다.
- 두 손가락을 오므리면(간격 75% 이하) 1 년 보기, 벌리면(133% 이상) 그 위치의 달로 돌아갑니다. PC 는 Ctrl+휠(트랙패드 핀치).
- 1 년 보기는 화면 크기에 맞춰 2/3/4/6 열 중 칸이 가장 크게 보이는 배치를 자동으로 고릅니다.

담당 파일

파일	클래스	역할
www/js/features/calendar.js	CalendarView	요약 카드·월간 캘린더·시험 카운트다운
www/js/features/calendar-zoom.js	CalendarZoom	핀치/Ctrl+휠 제스처
www/js/features/year-view.js	YearView	12 개월 학습량 히트맵

www/js/features/calendar-zoom.js — bindTouch

```
  this.bindTouch();
  this.bindWheel();
}


static dist(t) {
```

www/js/features/year-view.js — layout

```
window.addEventListener("resize", () => { if (!container.classList.contains("hidden")) this.layout(); });
```

3.3 날짜 바텀시트

- 날짜를 누르면 아래에서 시트가 올라옵니다. 날짜 종류에 따라 버튼이 달라집니다.
- 오늘: 📖 공부 시작(타이머) / 지난 날: ⏱ 공부한 시간(타이머 기록 보기) / 미래: 🚧 아직 오픈 전 하나만(누르면 농담) + 시험일 지정 줄
- 통계 버튼: 토요일은 '이 주 요약'(파란 그라데이션), 매달 마지막 날은 '이 달 요약'(빨간 그라데이션)

담당 파일

파일	클래스	역할
www/js/features/day-sheet.js	DaySheet	시트 표시·버튼 구성

www/js/features/day-sheet.js — renderPrimary

```
  this.renderPrimary(when, r);
  this.renderStats(k);


  // 미래 날짜는 "아직 오픈 전" 하나만. 시험일 지정은 아래 작은 줄로
  const future = when === "future";
  $("sheetStats").classList.toggle("hidden", future);
  $("sheetRecord").classList.toggle("hidden", future);
  this.el.querySelector(".sheet-actions").classList.toggle("single", future);
  $("sheetExam").classList.toggle("hidden", !future);
  $("sheetExam").textContent = this.store.isExam(k) ? "🎯 시험일 지정 해제" : "🎯 이 날을 토익 시험일로 지정";
}


// 통계 버튼: 매달 마지막 날은 "이 달 요약"(빨강), 토요일은 "이 주 요약"(파랑), 그 외는 기본
static statsScopeOf(k) {
```

www/js/features/day-sheet.js — static statsScopeOf

```
// 통계 버튼: 매달 마지막 날은 "이 달 요약"(빨강), 토요일은 "이 주 요약"(파랑), 그 외는 기본
static statsScopeOf(k) {
  const d = parseKey(k);
  const lastDay = new Date(d.getFullYear(), d.getMonth() + 1, 0).getDate();
  if (d.getDate() === lastDay) return "month";
  if (d.getDay() === 6) return "week";
  return null;
}
```

3.4 기록 화면 · 공부한 시간 화면

- 기록 화면: 시험일 지정, 공부한 파트, 단어 수, 모의고사 LC/RC, 컨디션, 메모. 학습 시간은 타이머 기록 합계로만 표시(직접 입력 불가).
- 공부한 시간 화면(지난 날짜): 그날 타이머로 기록된 시간 합계, 🫛 개수, 공부 시간대, 파트별 시간, 세션 목록.
- 세션 칸을 누르면 공부한 파트와 내용을 고칠 수 있습니다(시간은 변경 불가).

담당 파일

파일	클래스	역할
www/js/features/record.js	RecordScreen	날짜 기록 편집
www/js/features/study-log.js	StudyLogScreen	지난 날 타이머 기록 보기
www/js/features/session-dialog.js	SessionDialog	세션 공부 내용 기록·편집

www/js/features/record.js — save

```
$("recordForm").addEventListener("submit", (e) => { e.preventDefault(); this.save(); });
```

3.5 뽀모도로 타이머 (집중 → 휴식 → 다시 집중)

- 집중: 정해 둔 시간만큼 원판이 줄어듭니다. 시작할 때 '삐리릭', 끝나면 '삐삐삐' + 기록 + 공부 내용 창.
- 휴식: 시간 설정 없이 초록 원판이 쉰 만큼 차오르는 스톱워치. '다시 집중'을 누르면 휴식 시간이 직전 집중 기록에 남고, 집중 시간만큼 원판이 차오르며 바로 시작.
- 종료 시각을 저장하므로 앱을 닫았다 열어도 이어지고, 닫아둔 사이 끝났으면 열 때 기록됩니다.
- 화면(TimerScreen)과 로직(PomodoroTimer)을 분리했습니다.

담당 파일

파일	클래스	역할
www/js/features/pomodoro.js	PomodoroTimer, Alarm	타이머 상태·전환·알람
www/js/features/timer-screen.js	TimerScreen	타이머 화면·차오르는 애니메이션

www/js/features/pomodoro.js — finish

```
// 집중 시간이 다 됨: 기록 → 알람 → 휴식(스톱워치) 시작
finish(endAt) {
  clearInterval(this.tickId);
  const rec = this.store.addSession(this.state.focusStart || endAt - this.duration(), endAt);
  this.alarm.ring("집중 완료! 🫖", "휴식 중이에요. 다시 집중할 땐 '다시 집중'을 눌러 주세요.", endAt);
  this.enterBreak(endAt, rec);
  this.events.onFocusDone(rec);
}
```

www/js/features/pomodoro.js — refocus

```
// ---------- 휴식 → 다시 집중 ----------
refocus() {
  if (!this.onBreak) return;
  const restMin = Math.round(this.breakElapsed() / 60000);
  if (this.state.last) this.store.setSessionRest(this.state.last.k, this.state.last.id, restMin);
  Object.assign(this.state, { mode: "focus", running: false, remaining: this.duration(), focusStart: 0,
breakStart: 0, last: null });
  this.start();
}
```

3.6 타임타이머 다이얼

- 0 이 맨 위, 숫자는 반시계 방향(실제 타임타이머와 동일). 남은 시간만큼 부채꼴 원판을 SVG 로 그립니다.
- 멈춰 있을 때 원판을 드래그하면 손가락 각도로 분을 계산합니다. 0 근처에서 최대값↔0 으로 튀지 않게 보정합니다.
- 색상 5 종(레드·핑크·네이비·그린·틸). 휴식은 그린(집중 색이 그린이면 틸).

담당 파일

파일	클래스	역할
www/js/features/timer-dial.js	TimeTimerDial	다이얼 그리기·드래그

www/js/features/timer-dial.js — wedgePath

```
wedgePath(min) {
  const frac = Math.max(0, Math.min(1, min / this.max));
  if (frac <= 0) return "";
  if (frac >= 0.9999) return `M${CX - R_WEDGE},${CY} a${R_WEDGE},${R_WEDGE} 0 1,0 ${R_WEDGE * 2},0
a${R_WEDGE},${R_WEDGE} 0 1,0 ${-R_WEDGE * 2},0 Z`;
  const [ex, ey] = polar(frac * 360, R_WEDGE);
  // 맨 위에서 시작해 반시계 방향(sweep=0)으로 남은 시간만큼
  return `M${CX},${CY} L${CX},${CY - R_WEDGE} A${R_WEDGE},${R_WEDGE} 0 ${frac > 0.5 ? 1 : 0},0 ${ex},${ey}
Z`;
}
```

www/js/features/timer-dial.js — minutesFromPointer

```
minutesFromPointer(e) {
  const rect = this.els.svg.getBoundingClientRect();
  const dx = ((e.clientX - rect.left) / rect.width) * SIZE - CX;
  const dy = ((e.clientY - rect.top) / rect.height) * SIZE - CY;
  let deg = (Math.atan2(-dx, -dy) * 180) / Math.PI;
  if (deg < 0) deg += 360;
  const max = this.max, last = this.lastDragMin;
  let m = Math.round((deg / 360) * max);
  // 0 근처에서 최대값 ↔ 0 으로 튀지 않게
  if (last != null && last > max * 0.75 && m < max * 0.25) m = max;
  if (last != null && last < max * 0.25 && m > max * 0.75) m = 1;
  if (m === 0) m = last > max / 2 ? max : 1;
  return Math.max(1, Math.min(max, m));
}
```

3.7 효과음 (뜨르륵 · 삐삐삐) 과 오디오 연결

- 뜨르륵: 보내주신 mp3(7.8 초)를 분석해 첫 번째 태엽 소리(0.47~0.80 초, 딸깍 16 개)만 잘라 6 조각으로 나눴습니다. 1 분 조절마다 한 조각씩 이어서 재생해 빠르게 돌리면 원래 소리처럼 들립니다.
- 삐삐삐: 끝날 때는 삼각파 1046/1046/1318Hz 0.18 초 간격, 시작할 때는 사인파·반음 7 개 위·3 배속.
- AudioHub: 브라우저 자동재생 정책 때문에 첫 탭/클릭에 오디오를 깨웁니다.
- 안드로이드 앱(WebView)은 약 30 초 조용하면 스피커 연결을 끊고 다시 잇지 않는 문제가 있어(에뮬레이터에서 확인), 10 초 넘게 조용했으면 재생 직전에 연결을 새로 엽니다.

담당 파일

파일	클래스	역할
www/js/core/audio.js	AudioHub	공용 오디오·깨우기·재연결
www/js/features/wind-sound.js	WindSound	태엽 소리 조각 재생
www/js/features/sfx.js	SoundFx	삐삐삐 합성
www/sounds/wind-up.js	–	잘라낸 소리 데이터(base64 WAV)

www/js/core/audio.js — wake

```
// 소리를 내기 직전에 호출. 오래 조용했으면 연결을 다시 열고, 끝나면 resolve
wake() {
  const ctx = this.get();
  if (!ctx) return Promise.resolve();
  const idle = Date.now() - this.lastUse > IDLE_MS;
  this.lastUse = Date.now();
  if (this.waking) return this.waking;
  if (ctx.state !== "running") return ctx.resume().catch(() => {});
  if (!idle) return Promise.resolve();
  this.waking = ctx.suspend()
    .then(() => ctx.resume())
    .catch(() => {})
    .finally(() => { this.waking = null; });
```

```
  return this.waking;
}
```

www/js/features/wind-sound.js — tick

```
// 1분 조절할 때마다 호출: 다음 조각 재생
tick() {
  if (!this.ready()) return;
  const now = this.ctx.currentTime;
  if (now - this.lastTick > RESTART_AFTER) this.index = 0;
  this.lastTick = now;
  const when = Math.max(now, this.nextTime);
  if (when - now > MAX_QUEUE) return;
  const start = GRAINS[this.index];
  const end = GRAINS[this.index + 1] || CLIP_END;
  this.play(start, end, when);
  this.nextTime = when + (end - start);
  this.index = (this.index + 1) % GRAINS.length;
}
```

www/js/features/sfx.js — beep

```
// "삐삐삐": 0.18초 간격 세 음 (마지막 음이 조금 높음). semitones만큼 피치를 올리고 speed배 빠르게
// wave: "triangle"(또렷함) / "sine"(부드러움, 날카롭지 않음)
beep(semitones, speed = 1, wave = "triangle") {
  this.ctx = this.ctx || DT.audio.get();
  if (!this.ctx) return;
  // 앱 공용 오디오면 오래 조용했을 때 스피커 연결을 다시 연 뒤 재생 (테스트용 다른 오디오는 바로 재생)
  if (this.ctx === DT.audio.ctx && !this.waited) {
    DT.audio.wake().then(() => { this.waited = true; this.beep(semitones, speed, wave); this.waited =
false; });
    return;
  }
  const shift = Math.pow(2, semitones / 12);
  const k = 1 / speed; // 간격과 길이를 함께 줄임
  const t0 = this.ctx.currentTime + 0.01;
  [0, 0.18, 0.36].forEach((d, i) => {
    const at = t0 + d * k;
    const o = this.ctx.createOscillator(), g = this.ctx.createGain();
    o.type = wave;
    o.frequency.value = (i === 2 ? 1318 : 1046) * shift;
    g.gain.setValueAtTime(0.0001, at);
    // 아주 짧은 음도 '틱' 튀지 않도록 커지는 시간은 최소 6ms
    g.gain.exponentialRampToValueAtTime(0.35, at + Math.max(0.006, 0.015 * k));
    g.gain.exponentialRampToValueAtTime(0.0001, at + 0.16 * k);
    o.connect(g).connect(this.ctx.destination);
    o.start(at);
    o.stop(at + 0.18 * k);
  });
}
```

3.8 앱을 닫아도 울리는 알람 (안드로이드)

- 집중을 시작하면 끝나는 시각에 시스템 알림을 예약합니다(@capacitor/local-notifications). 일시정지·처음부터·끝내기 시 취소.
- 알림 채널 '집중 종료 알림': 앱과 같은 삐삐삐 소리(res/raw/focus_end.wav) + 진동 + 잠금 화면 표시.
- USE_EXACT_ALARM 권한으로 절전 모드에서도 정확한 시각에 울립니다(설치 시 자동 허용).

- 알림이 이미 울렸거나 나중에 앱을 연 경우 앱 안 삐삐삐는 생략해 두 번 울리지 않고, 울린 알림은 알림창에서 지웁니다.
- 검증: 에뮬레이터에서 1 분 타이머 시작 → 앱 강제 종료 → 예정 시각에 알림·소리·진동 확인(오차 0.2 초).

담당 파일

파일	클래스	역할
www/js/features/native-notifier.js	NativeNotifier	시스템 알림 예약·취소·정리
www/js/features/pomodoro.js	Alarm	알림/앱 소리 중 하나만 울리게

www/js/features/native-notifier.js — async scheduleFocusEnd

```
async scheduleFocusEnd(at) {
  if (!this.native || !(at > Date.now())) return;
  if (!(await this.ensurePermission())) return;
  try {
    await this.plugin.cancel({ notifications: [{ id: NOTIFY_ID }] });
    await this.plugin.schedule({
      notifications: [{
        id: NOTIFY_ID,
        title: "집중 완료! 🍅",
        body: "수고했어요. 앱을 열어 공부한 내용을 기록하고 휴식하세요.",
        channelId: CHANNEL_ID,
        sound: SOUND,
        smallIcon: "ic_stat_notify",
        iconColor: "#2F6FED",
        schedule: { at: new Date(at), allowWhileIdle: true }, // 절전 모드에서도 정확한 시각에
      }],
    });
    this.scheduledAt = at;
  } catch (e) {
    console.warn("[notify] 예약 실패", e);
  }
}
```

www/js/features/pomodoro.js — ring

```
ring(title, body, endAt) {
  const late = Date.now() - endAt > LATE_MS;
  const covered = this.notifier.coveredBy(endAt); // 시스템 알림이 이미 소리·진동을 냄
  this.notifier.scheduledAt = 0;
  // 이미 울린 알림 정리 (나중에 연 경우 바로, 화면을 보고 있었으면 알림 소리가 끝난 뒤)
  if (this.notifier.native) setTimeout(() => this.notifier.dismissFocusEnd(), late ? 0 : 4000);
  if (late || covered) return;
  this.fx.endBeep();
  if (navigator.vibrate) navigator.vibrate([120, 60, 120, 60, 200]);
  try {
    if (document.hidden && "Notification" in window && Notification.permission === "granted") new
Notification(title, { body, icon: "icon.svg" });
  } catch (e) {}
}
```

3.9 통계 (이 주 요약 · 이 달 요약)

- 이 달 요약: 공부한 날, 총 학습 시간, 하루 평균, 🍅 개수, 목표 달성 일수, 최고 점수.
- 이 주 요약(토요일에서 열면): 일요일~토요일 같은 항목 + 요일별 학습 시간 막대 그래프.

- 그 아래 모의고사 점수 추이(목표선), 최근 30 일 학습 시간, 파트별 학습 횟수, 전체 기록 목록.

담당 파일

파일	클래스	역할
www/js/features/stats.js	StatsScreen	통계 화면

www/js/features/stats.js — renderWeek

```
  if (this.scope === "week") this.renderWeek();
  else this.renderMonth();
  this.renderScoreChart();
  this.renderTimeChart();
  this.renderParts();
  this.renderList();
}


// 선택한 날짜가 속한 달 요약
renderMonth() {
```

3.10 꿀팁 배너 · 시험 카운트다운

- 꿀팁 배너: 💡 팁 11 개 + ✨ 한마디 13 개 + 🔥 내 기록에 맞춘 문구(연속 공부일, 오늘 목표까지 남은 시간). 터치할 때마다 무작위로 바뀌고 직전 문구는 다시 안 나옵니다.
- 시험 카드: 가장 가까운 시험일 0 시까지 일·시·분·초를 1 초마다 갱신, 카드 폭에 맞춰 글자 크기를 자동 조절해 한 줄로 표시. 당일은 '오늘 시험!'.

담당 파일

파일	클래스	역할
www/js/features/tip-banner.js	TipBanner	팁·응원 배너
www/js/features/calendar.js	CalendarView	renderCountdown / fitCountdown

www/js/features/tip-banner.js — personal

```
// 내 기록에 맞춘 한마디
personal() {
  const out = [];
  const s = this.store;
  const streak = s.streak();
  if (streak >= 2) out.push(`${streak}일 연속 공부 중! 오늘도 이어가 볼까요?`);
  const today = s.get(todayKey());
  const min = (today && today.minutes) || 0;
  const goal = s.settings.daily || 0;
  if (!min) out.push("오늘 첫 뽀모도로를 시작해 볼까요? 🍅");
  else if (goal && min >= goal) out.push(`오늘 ${fmtMin(min)} 공부! 목표 달성, 정말 잘했어요 👏`);
  else if (goal) out.push(`오늘 목표까지 ${fmtMin(goal - min)} 남았어요. 조금만 더!`);
  return out.map((text) => ({ kind: "me", text }));
}
```

www/js/features/calendar.js — renderCountdown

```
// 시험 카운트다운은 1 초마다 갱신 (화면이 보일 때만)
```

```
setInterval(() => { if (!document.hidden) this.renderCountdown(); }, 1000);
```

3.11 테마 · 설정 · 백업

- 🎨 버튼: 시스템 설정 → 밝게 → 어둡게 순환. 첫 화면 깜빡임 방지를 위해 index.html 에서 미리 적용.
- 설정: 목표 점수, 하루 목표 시간, JSON 백업 내보내기/가져오기(같은 날짜는 덮어쓰기, 시험일은 합치기).

담당 파일

파일	클래스	역할
www/js/features/theme.js	ThemeManager	테마 전환
www/js/features/settings.js	SettingsDialog	설정·백업

www/js/features/settings.js — async importBackup

```
async importBackup(e) {
  const file = e.target.files[0];
  e.target.value = "";
  if (!file) return;
  try {
    const data = JSON.parse(await file.text());
    const n = Object.keys((data && data.records) || {}).length;
    if (!confirm(`${n}개의 기록을 가져옵니다. 같은 날짜의 기존 기록은 덮어씁니다. 계속할까요?`)) return;
    this.store.importData(data);
    this.dialog.close();
    this.toast.show(`${n}개 기록을 가져왔습니다`);
  } catch (err) {
    this.toast.show("백업 파일을 읽을 수 없습니다");
  }
}
```

3.12 계정 로그인 · 클라우드 동기화 (환경만 구축)

- cloud-config.js 의 enabled 가 false 라 지금은 외부 요청이 전혀 없고, 설정 창에는 '준비 중'으로 표시됩니다.
- 켜면 Firebase 로 Google 로그인 → 계정 기록을 내려받아 합치고(같은 날짜는 이 기기 우선, 시험일은 합침) → 이후 바뀐 날짜만 올리고 지운 날짜는 삭제합니다.
- 저장 구조: users/{uid}(설정), users/{uid}/days/{날짜}(기록). 보안 규칙으로 자기 기록만 접근.
- 켜는 방법은 docs/CLOUD_SETUP.md 에 정리했습니다. Firebase 무료 요금제(Spark)로 개인 사용은 충분합니다.

담당 파일

파일	클래스	역할
www/js/core/cloud-config.js	–	켜기/끄기·설정값
www/js/core/firebase-provider.js	FirebaseProvider	Firebase 로그인·읽기/쓰기
www/js/core/account.js	AccountService	로그인 상태
www/js/core/cloud-sync.js	CloudSync	변경분 동기화
www/js/features/account-panel.js	AccountPanel	설정 창 계정 칸

www/js/core/cloud-sync.js — diff

```
// 마지막으로 올린 것과 비교해서 달라진 부분만 모으기
diff() {
  const { settings, records } = this.store.snapshot();
  const days = {};
  const upserts = {};
  Object.entries(records).forEach(([k, r]) => {
    days[k] = hash(r);
    if (this.synced.days[k] !== days[k]) upserts[k] = r;
  });
  const deletes = Object.keys(this.synced.days).filter((k) => !(k in records));
  const settingsHash = hash(settings);
  return {
    change: { settings: this.synced.settings !== settingsHash ? settings : null, upserts, deletes },
    next: { settings: settingsHash, days },
  };
}
```

3.13 안드로이드 앱 (APK)

- Capacitor 8 로 www/ 폴더를 감싼 앱입니다. 앱 ID `com.kch.dailytoeic`, 이름 'Daily TOEIC'.
- 아이콘·스플래시는 icon.svg 디자인을 Python(Pillow)으로 각 해상도에 맞춰 생성(적응형 아이콘 포함). 상단바 알림 아이콘도 별도 생성.
- 권한: INTERNET, VIBRATE, USE_EXACT_ALARM (+ 알림 플러그인의 POST_NOTIFICATIONS 등).
- 앱 안에서는 서비스 워커를 쓰지 않습니다(파일이 앱에 포함되어 있어 불필요).

다시 빌드하는 방법

```
cd C:\Users\kch36\Desktop\DailyToeic
# (이 PC 는 npm 명령이 막혀 있어 Node 로 직접 실행)
node node_modules/@capacitor/cli/bin/capacitor sync android
cd android
set JAVA_HOME=C:\Program Files\Android\Android Studio\jbr
gradlew.bat assembleDebug
# 결과: android\app\build\outputs\apk\debug\app-debug.apk → builds\DailyTOEIC.apk 로 복사
```

capacitor.config.json

```
{
  "appId": "com.kch.dailytoeic",
  "appName": "Daily TOEIC",
  "webDir": "www",
  "plugins": {
    "LocalNotifications": {
      "smallIcon": "ic_stat_notify",
```

```
      "iconColor": "#2F6FED",
      "sound": "focus_end.wav"
    }
  }
}
```

android/app/src/main/AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">


    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">


        <activity

android:configChanges="orientation|keyboardHidden|keyboard|screenSize|locale|smallestScreenSize|screenLayout|
uiMode|navigation|density"
            android:name=".MainActivity"
            android:label="@string/title_activity_main"
            android:theme="@style/AppTheme.NoActionBarLaunch"
            android:launchMode="singleTask"
            android:exported="true">


            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>


        </activity>


        <provider
            android:name="androidx.core.content.FileProvider"
            android:authorities="${applicationId}.fileprovider"
            android:exported="false"
            android:grantUriPermissions="true">
            <meta-data
                android:name="android.support.FILE_PROVIDER_PATHS"
                android:resource="@xml/file_paths"></meta-data>
        </provider>
    </application>


    <!-- Permissions -->


    <uses-permission android:name="android.permission.INTERNET" />
    <!-- 집중 시간이 끝났을 때 진동 -->
    <uses-permission android:name="android.permission.VIBRATE" />
    <!-- 앱을 닫아도 집중 종료 알림이 정확한 시각에 울리도록 (Android 13+ 타이머 앱용, 설치 시 자동 허용) -->
    <uses-permission android:name="android.permission.USE_EXACT_ALARM" />
</manifest>
```

4. 알려진 제한과 다음 할 일

항목	현재 상태	해결 방법
백업 내보내기 (앱)	앱 안에서는 파일 다운로드가 동작하지 않을 수 있음	@capacitor/filesystem + 공유 기능으로 저장
계정 연동	환경만 구축, 꺼져 있음	docs/CLOUD_SETUP.md 순서대로 Firebase 연결. 앱은 @capacitor-firebase/authentication 필요
플레이 스토어	USE_EXACT_ALARM 은 타이머/알람 앱만 허용	직접 설치(APK)는 문제없음. 스토어 배포 시 정책 확인
두 기기 동시 수정	동기화 시 같은 날짜는 이 기기 우선	기록마다 수정 시각을 넣어 최신 것 선택
실기기 확인	에뮬레이터(Android 16)에서 소리·알람 검증	실제 휴대폰에서 소리·알림 확인

부록. 전체 소스 코드

기능 묶음별로 파일 전체 코드를 실었습니다. 태엽 소리 데이터 파일(www/sounds/wind-up.js)은 약 2 만 자의 base64 데이터라 앞부분만 실었습니다.

A.1 공통 기반 (core)

www/js/core/utils.js

```
// 공통 유틸리티: 날짜/시간 포맷, DOM 헬퍼, 토스트
// 모든 파일은 window.DT 네임스페이스에 등록된다 (file:// 로 열어도 동작하도록 ES 모듈 대신 사용)
window.DT = window.DT || {};


(() => {
  "use strict";


  const WEEK = ["일", "월", "화", "수", "목", "금", "토"];
  const pad = (n) => String(n).padStart(2, "0");


  const utils = {
    PARTS: ["P1", "P2", "P3", "P4", "P5", "P6", "P7", "단어"],
    MOOD_ICON: { 1: "😭", 2: "😢", 3: "😐", 4: "😊", 5: "😄" },
    WEEK,


    $: (id) => document.getElementById(id),
    pad,


    // "2026-10-14" 형식의 날짜 키
    keyOf: (d) => `${d.getFullYear()}-${pad(d.getMonth() + 1)}-${pad(d.getDate())}`,
    parseKey: (k) => { const [y, m, d] = k.split("-").map(Number); return new Date(y, m - 1, d); },
    todayKey: () => utils.keyOf(new Date()),
    daysBetween: (fromKey, toKey) => Math.round((utils.parseKey(toKey) - utils.parseKey(fromKey)) /
86400000),


    fmtLong: (k) => { const d = utils.parseKey(k); return `${d.getFullYear()}년 ${d.getMonth() + 1}월
${d.getDate()}일 (${WEEK[d.getDay()]})`; },
    fmtShort: (k) => { const d = utils.parseKey(k); return `${d.getMonth() + 1}월 ${d.getDate()}일
(${WEEK[d.getDay()]})`; },
    fmtMin: (m) => {
      if (!m) return "0 분";
      const h = Math.floor(m / 60), r = m % 60;
      return h ? (r ? `${h}시간 ${r}분` : `${h}시간`) : `${r}분`;
    },
    fmtMinShort: (m) => (m >= 60 ? `${(m / 60).toFixed(m % 60 ? 1 : 0)}h` : `${m}m`),
    fmtClock: (ts) => { const d = new Date(ts); return `${pad(d.getHours())}:${pad(d.getMinutes())}`; },
    fmtTimer: (ms) => { const s = Math.ceil(ms / 1000); return `${pad(Math.floor(s / 60))}:${pad(s %
60)}`; },


    escapeHtml: (s) => String(s).replace(/[&<>"']/g, (c) => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", '"':
"&quot;", "'": "&#39;" }[c])),


    // 파트 선택 칩 HTML
    partChecksHtml: () => utils.PARTS.map((p) => `<label><input type="checkbox"
value="${p}"><span>${p}</span></label>`).join(""),
    checkedValues: (root) => [...root.querySelectorAll("input:checked")].map((c) => c.value),
    setChecked: (root, values) => root.querySelectorAll("input").forEach((c) => (c.checked =
values.includes(c.value))),
  };
```

```
  class Toast {
    constructor(el) {
      this.el = el;
      this.timer = null;
    }
    show(msg) {
      this.el.textContent = msg;
      this.el.classList.add("show");
      clearTimeout(this.timer);
      this.timer = setTimeout(() => this.el.classList.remove("show"), 2200);
    }
  }


  DT.utils = utils;
  DT.Toast = Toast;
})();
```

www/js/core/store.js

```
// 데이터 저장소: 날짜별 기록(records)과 설정(settings)을 localStorage 에 보관
// 데이터가 바뀌면 onChange 로 등록한 화면들에 알린다
(() => {
  "use strict";
  const { keyOf, todayKey, daysBetween, PARTS } = DT.utils;


  const RECORDS_KEY = "dailyToeic.records";
  const SETTINGS_KEY = "dailyToeic.settings";


  const DEFAULT_SETTINGS = { exams: [], goal: 800, daily: 60 };
  const DEFAULT_POMO = { focus: 25, color: "red", sound: true };
  const emptyRecord = () => ({ minutes: 0, parts: [], vocab: 0, lc: null, rc: null, mood: null, memo: "",
sessions: [] });


  class Store {
    constructor(onError) {
      this.onError = onError || (() => {});
      this.listeners = [];
      this.records = Store.read(RECORDS_KEY, {});
      this.settings = Object.assign({}, DEFAULT_SETTINGS, Store.read(SETTINGS_KEY, {}));
      this.settings.pomo = Object.assign({}, DEFAULT_POMO, this.settings.pomo);
      this.migrate();
    }


    static read(key, fallback) {
      try {
        const raw = localStorage.getItem(key);
        return raw ? JSON.parse(raw) : fallback;
      } catch (e) {
        return fallback;
      }
    }
    write(key, value) {
      try {
        localStorage.setItem(key, JSON.stringify(value));
      } catch (e) {
        this.onError("저장에 실패했습니다");
      }
    }


    // 예전 버전의 단일 시험일(examDate)을 시험 일정 목록으로 옮김
    migrate() {
      const s = this.settings;
      s.exams = s.exams || [];
      if (s.examDate) {
        if (!s.exams.includes(s.examDate)) s.exams.push(s.examDate);
        delete s.examDate;
      }
    }
```

```
    onChange(fn) { this.listeners.push(fn); }
    emit() { this.listeners.forEach((fn) => fn()); }
    persistRecords() { this.write(RECORDS_KEY, this.records); this.emit(); }
    persistSettings() { this.write(SETTINGS_KEY, this.settings); this.emit(); }


    // ---------- 기록 ----------
    get(k) { return this.records[k] || null; }


    static total(r) { return r && r.lc != null && r.rc != null ? r.lc + r.rc : null; }
    // 뽀모도로(타이머로 기록된 공부) 개수
    static tomatoes(r) { return ((r && r.sessions) || []).length; }
    static hasActivity(r) {
      return !!r && (r.minutes > 0 || (r.sessions && r.sessions.length > 0) || (r.parts && r.parts.length >
0) ||
        r.vocab > 0 || Store.total(r) != null || !!(r.memo && r.memo.trim()) || !!r.mood);
    }


    saveRecord(k, r) {
      if (Store.hasActivity(r)) this.records[k] = r;
      else delete this.records[k];
      this.persistRecords();
    }
    deleteRecord(k) {
      delete this.records[k];
      this.persistRecords();
    }


    activeKeys() { return Object.keys(this.records).filter((k) =>
Store.hasActivity(this.records[k])).sort(); }


    // 하루 목표 대비 학습량 단계 (캘린더 색 진하기)
    level(min) {
      if (!min) return 0;
      const g = this.settings.daily || 60;
      if (min < g * 0.5) return 1;
      if (min < g) return 2;
      if (min < g * 2) return 3;
      return 4;
    }


    // 연속 학습일: 오늘 기록이 없으면 어제부터 센다
    streak() {
      let n = 0;
      const d = new Date();
      if (!Store.hasActivity(this.records[keyOf(d)])) d.setDate(d.getDate() - 1);
      while (Store.hasActivity(this.records[keyOf(d)])) { n++; d.setDate(d.getDate() - 1); }
      return n;
    }
    minutesInMonth(y, m) {
      const prefix = `${y}-${String(m + 1).padStart(2, "0")}`;
      return Object.keys(this.records).filter((k) => k.startsWith(prefix)).reduce((s, k) => s +
(this.records[k].minutes || 0), 0);
    }
    bestScore() {
      const scores = Object.values(this.records).map(Store.total).filter((v) => v != null);
      return scores.length ? Math.max(...scores) : null;
    }


    // ---------- 뽀모도로 세션 ----------
    findSession(k, id) { return ((this.records[k] && this.records[k].sessions) || []).find((s) => s.id ===
id) || null; }


    addSession(start, end) {
      const k = keyOf(new Date(start));
      const minutes = Math.max(1, Math.round((end - start) / 60000));
      const r = this.records[k] || emptyRecord();
      r.sessions = r.sessions || [];
      const s = { id: String(end), start, end, minutes, parts: [], note: "" };
      r.sessions.push(s);
      r.minutes = (r.minutes || 0) + minutes;
```

```
      this.records[k] = r;
      this.persistRecords();
      return { k, s };
    }
    updateSession(k, id, parts, note) {
      const s = this.findSession(k, id);
      if (!s) return;
      s.parts = parts;
      s.note = note;
      // 세션에서 고른 파트를 그날 기록의 파트에도 반영
      const r = this.records[k];
      r.parts = [...new Set([...(r.parts || []), ...parts])].sort((a, b) => PARTS.indexOf(a) -
PARTS.indexOf(b));
      this.persistRecords();
    }
    // 집중 뒤에 쉰 시간(분)을 그 집중 기록에 남김 (공부 시간에는 포함하지 않음)
    setSessionRest(k, id, minutes) {
      const s = this.findSession(k, id);
      if (!s) return;
      s.rest = minutes;
      this.persistRecords();
    }


    deleteSession(k, id) {
      const r = this.records[k];
      const s = this.findSession(k, id);
      if (!s) return;
      r.sessions = r.sessions.filter((x) => x.id !== id);
      r.minutes = Math.max(0, (r.minutes || 0) - s.minutes);
      if (!Store.hasActivity(r)) delete this.records[k];
      this.persistRecords();
    }
    // 직전 세션에서 고른 파트 (다음 세션 기록 시 기본값으로 사용)
    lastSessionParts(k, exceptId) {
      const prev = ((this.records[k] && this.records[k].sessions) || []).filter((x) => x.id !== exceptId &&
x.parts.length).pop();
      return prev ? prev.parts : [];
    }


    // ---------- 시험 일정 ----------
    isExam(k) { return this.settings.exams.includes(k); }
    setExam(k, on) {
      const set = new Set(this.settings.exams);
      on ? set.add(k) : set.delete(k);
      this.settings.exams = [...set].sort();
      this.persistSettings();
    }
    nextExam() { return this.settings.exams.filter((k) => k >= todayKey()).sort()[0] || null; }
    static ddayText(k) {
      const n = daysBetween(todayKey(), k);
      return n > 0 ? `D-${n}` : n === 0 ? "D-DAY" : `D+${-n}`;
    }


    // ---------- 설정 ----------
    updateSettings(patch) {
      Object.assign(this.settings, patch);
      this.persistSettings();
    }
    updatePomo(patch, silent) {
      Object.assign(this.settings.pomo, patch);
      if (silent) this.write(SETTINGS_KEY, this.settings);
      else this.persistSettings();
    }


    // ---------- 클라우드 동기화 (CloudSync 가 사용) ----------
    snapshot() { return { settings: this.settings, records: this.records }; }


    // 계정에서 내려받은 기록 합치기: 같은 날짜는 이 기기 우선, 한쪽에만 있으면 가져옴. 시험일은 양쪽 합침
    mergeRemote(remote) {
      if (!remote) return;
      let changed = false;
      Object.entries(remote.records || {}).forEach(([k, r]) => {
        if (!this.records[k]) { this.records[k] = r; changed = true; }
```

```
      });
      if (remote.settings) {
        const exams = [...new Set([...this.settings.exams, ...(remote.settings.exams || [])])].sort();
        if (exams.length !== this.settings.exams.length) { this.settings.exams = exams; changed = true; }
      }
      if (!changed) return;
      this.write(RECORDS_KEY, this.records);
      this.write(SETTINGS_KEY, this.settings);
      this.emit();
    }


    // ---------- 백업 ----------
    exportJson() {
      return JSON.stringify({ app: "DailyToeic", version: 2, exportedAt: new Date().toISOString(), settings:
this.settings, records: this.records }, null, 2);
    }
    importData(data) {
      if (!data || typeof data.records !== "object") throw new Error("형식 오류");
      Object.assign(this.records, data.records);
      if (data.settings) {
        const exams = new Set([...this.settings.exams, ...(data.settings.exams || [])]);
        const pomo = Object.assign({}, this.settings.pomo, data.settings.pomo);
        Object.assign(this.settings, data.settings);
        this.settings.exams = [...exams];
        this.settings.pomo = pomo;
        this.migrate();
      }
      this.write(RECORDS_KEY, this.records);
      this.write(SETTINGS_KEY, this.settings);
      this.emit();
      return Object.keys(data.records).length;
    }
  }


  Store.emptyRecord = emptyRecord;
  DT.Store = Store;
})();
```

www/js/core/router.js

```
// 화면 전환: 주소의 #해시로 현재 화면을 관리한다
//   ""            → 캘린더(홈)
//   #day/날짜     → 홈 + 날짜 바텀시트
//   #timer        → 공부(타이머) 화면
//   #stats/날짜   → 통계 화면
//   #record/날짜  → 기록 화면
// 휴대폰 뒤로가기 버튼을 누르면 이전 화면으로 돌아간다
(() => {
  "use strict";


  class Router {
    constructor() {
      this.views = [];
      this.current = { name: "", param: null };
      window.addEventListener("popstate", () => this.sync());
    }


    // view: { name, setActive(active, param) }
    register(view) {
      this.views.push(view);
    }


    static parse(hash) {
      const [name, param] = hash.replace(/^#/, "").split("/");
      return { name: name || "", param: param || null };
    }


    go(name, param) {
```

```
      const hash = name ? `#${name}${param ? `/${param}` : ""}` : location.pathname + location.search;
      history.pushState(null, "", hash);
      this.sync();
    }


    back() {
      if (this.current.name) history.back();
    }


    // 홈으로 바로 돌아가기 (쌓인 화면 건너뜀)
    home() {
      history.replaceState(null, "", location.pathname + location.search);
      this.sync();
    }


    sync() {
      this.current = Router.parse(location.hash);
      this.views.forEach((v) => v.setActive(v.name === this.current.name, this.current.param));
    }


    // 새로고침 시 해시가 남아 있으면, 뒤로가기로 홈에 갈 수 있도록 홈을 기록에 깔아 둔다
    start() {
      const initial = location.hash;
      if (initial) {
        history.replaceState(null, "", location.pathname + location.search);
        history.pushState(null, "", initial);
      }
      this.sync();
    }
  }


  DT.Router = Router;
})();
```

www/js/core/audio.js

```
// 앱 전체가 함께 쓰는 오디오(AudioContext) 하나
// 브라우저는 사용자가 화면을 탭/클릭하기 전에는 소리를 막아 둔다(자동재생 정책).
// 특히 휴대폰은 손가락이 "닿는 순간"(pointerdown)은 인정하지 않고 "뗄 때"(탭)만 인정하므로,
// 페이지 어디서든 첫 탭·클릭·키 입력이 생기면 바로 오디오를 깨워 둔다.
(() => {
  "use strict";


  const UNLOCK_EVENTS = ["pointerup", "touchend", "click", "keydown"];
  // 안드로이드 앱(WebView)은 약 30초 동안 소리가 없으면 스피커 연결을 끊고, 그 뒤엔 소리를 내도 다시 잇지 않는다.
  // 그래서 이만큼 조용했으면 재생 직전에 오디오를 잠깐 멈췄다 다시 켜서 연결을 새로 연다.
  const IDLE_MS = 10000;


  class AudioHub {
    constructor() {
      this.ctx = null;
      this.primed = false;
      this.lastUse = 0;     // 마지막으로 소리를 낸 시각
      this.waking = null;   // 연결을 다시 여는 중이면 그 Promise
      const onGesture = () => {
        this.unlock();
        if (this.ctx && this.ctx.state === "running") {
          UNLOCK_EVENTS.forEach((ev) => document.removeEventListener(ev, onGesture, true));
        }
      };
      UNLOCK_EVENTS.forEach((ev) => document.addEventListener(ev, onGesture, true));
    }


    // 필요할 때 만든다 (지원하지 않는 환경이면 null)
    get() {
```

```
      if (!this.ctx) {
        try {
          this.ctx = new (window.AudioContext || window.webkitAudioContext)();
        } catch (e) {
          this.ctx = null;
        }
      }
      return this.ctx;
    }

    get running() { return !!this.ctx && this.ctx.state === "running"; }

    // 소리를 내기 직전에 호출. 오래 조용했으면 연결을 다시 열고, 끝나면 resolve
    wake() {
      const ctx = this.get();
      if (!ctx) return Promise.resolve();
      const idle = Date.now() - this.lastUse > IDLE_MS;
      this.lastUse = Date.now();
      if (this.waking) return this.waking;
      if (ctx.state !== "running") return ctx.resume().catch(() => {});
      if (!idle) return Promise.resolve();
      this.waking = ctx.suspend()
        .then(() => ctx.resume())
        .catch(() => {})
        .finally(() => { this.waking = null; });
      return this.waking;
    }

    unlock() {
      const ctx = this.get();
      if (!ctx) return;
      if (ctx.state !== "running") ctx.resume().catch(() => {});
      // 예전 iOS Safari는 사용자 동작 안에서 소리를 한 번 재생해야 풀린다: 무음 1샘플 재생
      if (!this.primed) {
        try {
          const src = ctx.createBufferSource();
          src.buffer = ctx.createBuffer(1, 1, 22050);
          src.connect(ctx.destination);
          src.start(0);
          this.primed = true;
        } catch (e) {}
      }
    }
  }

  DT.audio = new AudioHub();
})();
```

www/js/core/cloud-config.js

```
// 계정 로그인 / 클라우드 동기화 설정
// ※ 아직 사용하지 않음 (enabled: false). 켜는 방법은 docs/CLOUD_SETUP.md 참고
//   enabled가 false인 동안에는 외부 스크립트를 전혀 불러오지 않고, 기록은 이 기기에만 저장된다.
window.DT = window.DT || {};

DT.CLOUD_CONFIG = {
  enabled: false,
  provider: "firebase",

  // Firebase 콘솔 > 프로젝트 설정 > 내 앱(웹)에서 복사해 붙여넣기
  firebase: {
    apiKey: "",
    authDomain: "",
    projectId: "",
    appId: "",
  },
};
```

www/js/core/firebase-provider.js

```
// Firebase 연결 (로그인 + Firestore 저장). 다른 서비스로 바꾸려면 같은 메서드를 가진 클래스로 교체하면 된다
//   load()                    SDK 를 불러오고 초기화 (설정이 켜졌을 때만 호출됨)
//   onAuthChange(cb)          로그인 상태가 바뀌면 cb(user | null)
//   signIn() / signOut()      Google 로그인 / 로그아웃
//   pull(uid)                 { settings, records } 내려받기
//   push(uid, change)         { settings?, upserts: {날짜: 기록}, deletes: [날짜] } 올리기
//
// Firestore 구조 (문서 하나 1MB 제한을 피하려고 날짜별로 나눔)
//   users/{uid}                 { settings, updatedAt }
//   users/{uid}/days/{날짜}     { record, updatedAt }
(() => {
  "use strict";


  const SDK_VERSION = "10.12.2";
  const SDK_URLS = ["app", "auth", "firestore"].map(
    (name) => `https://www.gstatic.com/firebasejs/${SDK_VERSION}/firebase-${name}-compat.js`
  );
  const BATCH_LIMIT = 450; // Firestore 일괄 쓰기 한도(500)보다 조금 작게


  const loadScript = (src) => new Promise((resolve, reject) => {
    const s = document.createElement("script");
    s.src = src;
    s.async = true;
    s.onload = resolve;
    s.onerror = () => reject(new Error(`스크립트를 불러오지 못했습니다: ${src}`));
    document.head.appendChild(s);
  });


  class FirebaseProvider {
    constructor(config) {
      this.config = config;
      this.auth = null;
      this.db = null;
    }


    async load() {
      const c = this.config;
      if (!c || !c.apiKey || !c.projectId) throw new Error("Firebase 설정값(apiKey, projectId)이 비어
있습니다");
      for (const url of SDK_URLS) await loadScript(url); // 순서대로 (app → auth → firestore)
      const fb = window.firebase;
      const app = fb.apps.length ? fb.app() : fb.initializeApp(c);
      this.auth = app.auth();
      this.db = app.firestore();
      // 오프라인에서도 쓰고, 다시 연결되면 자동으로 올라가도록
      try { await this.db.enablePersistence({ synchronizeTabs: true }); } catch (e) { /* 지원 안 되는
브라우저면 무시 */ }
    }


    onAuthChange(cb) {
      this.auth.onAuthStateChanged((u) => cb(u ? { uid: u.uid, email: u.email, name: u.displayName, photo:
u.photoURL } : null));
    }


    async signIn() {
      const provider = new window.firebase.auth.GoogleAuthProvider();
      try {
        await this.auth.signInWithPopup(provider);
      } catch (e) {
        // 팝업이 막힌 환경(일부 모바일 브라우저)에서는 페이지 이동 방식으로
        if (e && (e.code === "auth/popup-blocked" || e.code === "auth/operation-not-supported-in-this-
environment")) {
          await this.auth.signInWithRedirect(provider);
        } else {
          throw e;
```

```
        }
      }
    }


    signOut() { return this.auth.signOut(); }


    userRef(uid) { return this.db.collection("users").doc(uid); }


    async pull(uid) {
      const ref = this.userRef(uid);
      const [userDoc, days] = await Promise.all([ref.get(), ref.collection("days").get()]);
      const records = {};
      days.forEach((d) => { const v = d.data(); if (v && v.record) records[d.id] = v.record; });
      return { settings: (userDoc.exists && userDoc.data().settings) || null, records };
    }


    async push(uid, change) {
      const fb = window.firebase;
      const now = fb.firestore.FieldValue.serverTimestamp();
      const ref = this.userRef(uid);
      const ops = [];
      if (change.settings) ops.push((b) => b.set(ref, { settings: change.settings, updatedAt: now }, { merge:
true }));
      Object.entries(change.upserts || {}).forEach(([k, record]) => ops.push((b) =>
b.set(ref.collection("days").doc(k), { record, updatedAt: now })));
      (change.deletes || []).forEach((k) => ops.push((b) => b.delete(ref.collection("days").doc(k))));
      for (let i = 0; i < ops.length; i += BATCH_LIMIT) {
        const batch = this.db.batch();
        ops.slice(i, i + BATCH_LIMIT).forEach((op) => op(batch));
        await batch.commit();
      }
    }
  }


  DT.FirebaseProvider = FirebaseProvider;
})();
```

www/js/core/account.js

```
// 계정 상태 관리. 실제 로그인은 provider(FirebaseProvider)가 하고, 화면은 이 객체의 상태만 본다
//   status: "disabled"(설정 꺼짐) | "loading" | "signedOut" | "signedIn" | "error"
(() => {
  "use strict";


  const PROVIDERS = { firebase: (cfg) => new DT.FirebaseProvider(cfg.firebase) };


  class AccountService {
    constructor(config) {
      this.config = config || { enabled: false };
      this.status = this.config.enabled ? "loading" : "disabled";
      this.user = null;
      this.error = null;
      this.provider = null;
      this.listeners = [];
    }


    get enabled() { return !!this.config.enabled; }
    get signedIn() { return this.status === "signedIn"; }


    onChange(fn) { this.listeners.push(fn); }
    emit() { this.listeners.forEach((fn) => fn(this)); }


    async init() {
      if (!this.enabled) return; // 꺼져 있으면 아무것도 불러오지 않음
      try {
        const make = PROVIDERS[this.config.provider];
        if (!make) throw new Error(`지원하지 않는 provider: ${this.config.provider}`);
```

```
        this.provider = make(this.config);
        await this.provider.load();
        this.provider.onAuthChange((user) => {
          this.user = user;
          this.status = user ? "signedIn" : "signedOut";
          this.emit();
        });
      } catch (e) {
        this.fail(e);
      }
    }

    fail(e) {
      this.status = "error";
      this.error = (e && e.message) || String(e);
      console.warn("[account]", e);
      this.emit();
    }

    async signIn() {
      if (!this.provider) return;
      try { await this.provider.signIn(); } catch (e) { this.fail(e); }
    }

    async signOut() {
      if (!this.provider) return;
      try { await this.provider.signOut(); } catch (e) { this.fail(e); }
    }
  }

  DT.AccountService = AccountService;
})();
```

www/js/core/cloud-sync.js

```
// 기록 클라우드 동기화: 로그인하면 계정의 기록을 내려받아 합치고, 이후 바뀐 날짜만 올린다
//   - 합치기 규칙(1차 버전): 같은 날짜가 양쪽에 있으면 이 기기 기록을 우선, 한쪽에만 있으면 가져옴
//                            시험일은 양쪽을 합침, 나머지 설정은 이 기기 우선
//   - 마지막으로 올린 내용을 기억해 두었다가(dailyToeic.sync.{uid}) 달라진 날짜만 올리고, 지운 날짜는 삭제
(() => {
  "use strict";

  const PUSH_DELAY = 2500; // 연속으로 고칠 때 모아서 한 번에 올리기
  const SYNC_KEY = (uid) => `dailyToeic.sync.${uid}`;
  const hash = (v) => JSON.stringify(v);

  class CloudSync {
    constructor(store, account) {
      this.store = store;
      this.account = account;
      this.uid = null;
      this.synced = null;   // { settings: 해시, days: { 날짜: 해시 } }
      this.timer = null;
      this.busy = false;
      this.lastSyncedAt = null;
      this.listeners = [];

      account.onChange(() => (account.signedIn ? this.start(account.user.uid) : this.stop()));
      store.onChange(() => this.schedule());
    }

    onChange(fn) { this.listeners.push(fn); }
    emit() { this.listeners.forEach((fn) => fn(this)); }

    async start(uid) {
      if (this.uid === uid) return;
      this.uid = uid;
```

```
      this.synced = DT.Store.read(SYNC_KEY(uid), { settings: null, days: {} });
      try {
        this.busy = true;
        this.emit();
        const remote = await this.account.provider.pull(uid);
        this.store.mergeRemote(remote); // 합친 결과로 화면도 갱신됨
        await this.pushNow();
      } catch (e) {
        console.warn("[sync] 처음 동기화 실패", e);
      } finally {
        this.busy = false;
        this.emit();
      }
    }

    stop() {
      clearTimeout(this.timer);
      this.uid = null;
      this.synced = null;
      this.emit();
    }

    schedule() {
      if (!this.uid) return;
      clearTimeout(this.timer);
      this.timer = setTimeout(() => this.pushNow(), PUSH_DELAY);
    }

    // 마지막으로 올린 것과 비교해서 달라진 부분만 모으기
    diff() {
      const { settings, records } = this.store.snapshot();
      const days = {};
      const upserts = {};
      Object.entries(records).forEach(([k, r]) => {
        days[k] = hash(r);
        if (this.synced.days[k] !== days[k]) upserts[k] = r;
      });
      const deletes = Object.keys(this.synced.days).filter((k) => !(k in records));
      const settingsHash = hash(settings);
      return {
        change: { settings: this.synced.settings !== settingsHash ? settings : null, upserts, deletes },
        next: { settings: settingsHash, days },
      };
    }

    async pushNow() {
      if (!this.uid) return;
      const uid = this.uid;
      const { change, next } = this.diff();
      if (!change.settings && !Object.keys(change.upserts).length && !change.deletes.length) return;
      try {
        await this.account.provider.push(uid, change);
        if (this.uid !== uid) return; // 그새 로그아웃했으면 무시
        this.synced = next;
        this.store.write(SYNC_KEY(uid), next);
        this.lastSyncedAt = Date.now();
        this.emit();
      } catch (e) {
        console.warn("[sync] 올리기 실패, 다음 변경 때 다시 시도", e);
      }
    }
  }

  DT.CloudSync = CloudSync;
})();
```

A.2 홈 · 캘린더

www/js/features/calendar.js

```
// 홈 화면: 요약 카드, 캘린더 (한 달 보기 ↔ 1 년 보기). 위쪽 배너는 TipBanner 가 담당
// 날짜를 누르면 onSelect(날짜키)를 호출한다
// 두 손가락으로 오므리면 1 년 보기, 벌리면 한 달 보기 (CalendarZoom), 1 년 보기는 YearView 가 그린다
(() => {
  "use strict";
  const { $, pad, todayKey, fmtShort, fmtMin, fmtMinShort } = DT.utils;
  const { Store, YearView, CalendarZoom } = DT;


  // 시험 카운트다운 기준 시각: 시험 날 하루가 시작되는 00 시
  const EXAM_TIME = { h: 0, m: 0 };


  class CalendarView {
    constructor(store, onSelect) {
      this.store = store;
      this.onSelect = onSelect;
      this.view = new Date();
      this.view.setDate(1);
      this.mode = "month"; // "month" | "year"
      this.selected = null;
      this.grid = $("calGrid");
      this.panel = document.querySelector(".panel.calendar");


      this.yearView = new YearView(store, $("yearView"), $("yearGrid"), (m) => this.showMonth(m));
      this.zoom = new CalendarZoom(this.panel, {
        onZoomOut: () => { if (this.mode === "month") this.showYear(); },
        onZoomIn: ({ x, y }) => {
          if (this.mode !== "year") return;
          const m = this.yearView.monthAt(x, y);
          this.showMonth(m != null ? m : this.view.getMonth());
        },
      });


      $("prevMonth").addEventListener("click", () => this.move(-1));
      $("nextMonth").addEventListener("click", () => this.move(1));
      $("todayBtn").addEventListener("click", () => this.goToday());
      $("zoomBtn").addEventListener("click", () => (this.mode === "month" ? this.showYear() :
this.showMonth(this.view.getMonth())));
      // 시험 카운트다운은 1 초마다 갱신 (화면이 보일 때만)
      setInterval(() => { if (!document.hidden) this.renderCountdown(); }, 1000);
      this.bindSwipe();
      store.onChange(() => this.render());
    }


    // 한 달 보기에선 달 이동, 1 년 보기에선 해 이동
    move(delta) {
      if (this.mode === "year") this.view.setFullYear(this.view.getFullYear() + delta);
      else this.view.setMonth(this.view.getMonth() + delta);
      this.render();
    }


    goToday() {
      this.view = new Date();
      this.view.setDate(1);
      this.render();
    }


    showYear() {
      this.mode = "year";
      this.render();
      this.animate($("yearView"), "zoom-from-big");
    }


    showMonth(m) {
      this.mode = "month";
      this.view.setMonth(m);
```

```
      this.render();
      this.animate($("calMonth"), "zoom-from-small");
    }


    animate(el, cls) {
      el.classList.remove("zoom-from-big", "zoom-from-small");
      void el.offsetWidth; // 애니메이션 다시 시작
      el.classList.add(cls);
    }


    // 좌우 스와이프로 달(1 년 보기에선 해) 이동. 두 손가락 핀치 중에는 무시
    bindSwipe() {
      let sx = null, sy = null;
      this.panel.addEventListener("touchstart", (e) => {
        if (e.touches.length === 1 && !this.zoom.pinching) { sx = e.touches[0].clientX; sy =
e.touches[0].clientY; }
        else sx = null;
      }, { passive: true });
      this.panel.addEventListener("touchend", (e) => {
        if (sx == null || this.zoom.pinching || e.touches.length) { sx = null; return; }
        const dx = e.changedTouches[0].clientX - sx, dy = e.changedTouches[0].clientY - sy;
        sx = null;
        if (Math.abs(dx) > 60 && Math.abs(dx) > Math.abs(dy)) this.move(dx < 0 ? 1 : -1);
      });
    }


    setSelected(k) {
      this.selected = k;
      this.grid.querySelectorAll(".day").forEach((b) => b.classList.toggle("selected", b.dataset.k === k));
    }


    render() {
      const year = this.mode === "year";
      $("calMonth").classList.toggle("hidden", year);
      $("yearView").classList.toggle("hidden", !year);
      $("zoomBtn").textContent = year ? "한 달" : "1 년";
      $("zoomBtn").title = year ? "한 달 보기 (두 손가락으로 벌리기)" : "1 년 보기 (두 손가락으로 오므리기)";
      this.renderSummary();
      if (year) {
        $("monthLabel").textContent = `${this.view.getFullYear()}년`;
        this.yearView.render(this.view.getFullYear());
      } else {
        this.renderGrid();
      }
    }


    // 시험까지 남은 시간: "11 일 03 시 12 분 45 초" 한 줄 (좁은 화면에선 글자를 줄여 맞춤)
    renderCountdown() {
      const el = $("statDday");
      const k = this.store.nextExam();
      if (!k) { el.textContent = "미설정"; return; }
      const [y, mo, d] = k.split("-").map(Number);
      const left = new Date(y, mo - 1, d, EXAM_TIME.h, EXAM_TIME.m) - Date.now();
      if (left <= 0) { el.textContent = "오늘 시험!"; return; }
      const s = Math.floor(left / 1000);
      const days = Math.floor(s / 86400), h = Math.floor((s % 86400) / 3600), m = Math.floor((s % 3600) /
60), sec = s % 60;
      const u = (n, unit) => `${pad(n)}<small>${unit}</small>`;
      const head = days ? `${days}<small>일</small> ` : "";
      el.innerHTML = `<span class="cd">${head}${u(h, "시")} ${u(m, "분")} ${u(sec, "초")}</span>`;
      this.fitCountdown(el);
    }


    // 카드 폭보다 길면 글자 크기를 줄여 한 줄에 맞춤
    fitCountdown(el) {
      const cd = el.firstElementChild;
      cd.style.fontSize = "";
      const avail = el.clientWidth - 2;
      // 단위 글자까지 함께 줄어들도록 em 기준. 두 번 맞춰서 오차 보정
      for (let i = 0, size = 0.86; i < 2 && avail > 0 && cd.scrollWidth > avail; i++) {
```

```
        size = Math.max(0.55, size * avail / cd.scrollWidth);
        cd.style.fontSize = `${size.toFixed(3)}rem`;
      }
    }


    renderSummary() {
      this.renderCountdown();
      $("statStreak").textContent = `${this.store.streak()}일`;
      const y = this.view.getFullYear();
      // 1년 보기에선 "이번 달" 대신 그 해 전체 학습 시간
      $("statMonthLabel").textContent = this.mode === "year" ? `${y}년` : "이번 달";
      $("statMonth").textContent = fmtMin(this.mode === "year" ? this.yearView.yearMinutes(y) :
this.store.minutesInMonth(y, this.view.getMonth()));
      const best = this.store.bestScore();
      $("statBest").textContent = best != null ? best : "-";
    }



    renderGrid() {
      const y = this.view.getFullYear(), m = this.view.getMonth();
      $("monthLabel").textContent = `${y}년 ${m + 1}월`;
      const first = new Date(y, m, 1).getDay();
      const days = new Date(y, m + 1, 0).getDate();
      const weeks = Math.ceil((first + days) / 7);
      const tk = todayKey();
      const daily = this.store.settings.daily;


      // 주 수(4~6주)에 맞춰 칸 높이를 화면에 꽉 차게 나눈다
      this.grid.style.gridTemplateRows = `repeat(${weeks}, minmax(0, 1fr))`;
      let html = "";
      for (let i = 0; i < first; i++) html += `<div class="day empty"></div>`;
      for (let d = 1; d <= days; d++) {
        const k = `${y}-${pad(m + 1)}-${pad(d)}`;
        const r = this.store.get(k);
        const dow = (first + d - 1) % 7;
        const exam = this.store.isExam(k);
        const cls = ["day", `lv${this.store.level(r && r.minutes)}`];
        if (dow === 0) cls.push("sun");
        if (dow === 6) cls.push("sat");
        if (k === tk) cls.push("today");
        if (k > tk) cls.push("future");
        if (k === this.selected) cls.push("selected");
        if (exam) cls.push("exam", k < tk ? "past" : "upcoming");
        const label = `${m + 1}월 ${d}일${exam ? `, 토익 시험 ${Store.ddayText(k)}` : ""}${r && r.minutes ? `,
${fmtMin(r.minutes)}` : ""}`;


        let inner = `<span class="num">${d}</span>`;
        if (exam) inner += `<span class="exam-tag">${k < tk ? "시험" : Store.ddayText(k)}</span>`;
        if (Store.total(r) != null) inner += `<span class="dot"></span>`;
        if (r && daily && r.minutes >= daily) inner += `<span class="goal-check">✓</span>`;
        if (r && r.minutes) inner += `<span class="min">${Store.tomatoes(r) ? "🍅" :
""}${fmtMinShort(r.minutes)}</span>`;
        html += `<button type="button" class="${cls.join(" ")}" data-k="${k}" aria-
label="${label}">${inner}</button>`;
      }
      this.grid.innerHTML = html;
      this.grid.querySelectorAll("button.day").forEach((b) => b.addEventListener("click", () =>
this.onSelect(b.dataset.k)));
    }
  }


  DT.CalendarView = CalendarView;
})();
```

www/js/features/calendar-zoom.js

```
// 캘린더 확대/축소 제스처
//  - 휴대폰: 두 손가락을 오므리면 onZoomOut(), 벌리면 onZoomIn(가운데 지점)
```

```
//  - PC: 트랙패드 핀치(Ctrl + 휠)도 같은 동작
(() => {
  "use strict";


  const OUT_RATIO = 0.75; // 손가락 간격이 처음의 75% 이하로 줄면 축소
  const IN_RATIO = 1.33;  // 133% 이상으로 벌어지면 확대


  class CalendarZoom {
    // handlers: { onZoomOut(), onZoomIn({x, y}) }
    constructor(el, handlers) {
      this.el = el;
      this.handlers = handlers;
      this.startDist = 0;
      this.fired = false;
      this.pinching = false; // 두 손가락이 닿아 있는 동안 true (스와이프와 구분용)
      this.wheelLock = 0;
      this.bindTouch();
      this.bindWheel();
    }


    static dist(t) {
      return Math.hypot(t[0].clientX - t[1].clientX, t[0].clientY - t[1].clientY);
    }
    static mid(t) {
      return { x: (t[0].clientX + t[1].clientX) / 2, y: (t[0].clientY + t[1].clientY) / 2 };
    }


    bindTouch() {
      this.el.addEventListener("touchstart", (e) => {
        if (e.touches.length === 2) {
          this.pinching = true;
          this.fired = false;
          this.startDist = CalendarZoom.dist(e.touches);
        }
      }, { passive: true });


      this.el.addEventListener("touchmove", (e) => {
        if (!this.pinching || e.touches.length !== 2 || this.fired || !this.startDist) return;
        const ratio = CalendarZoom.dist(e.touches) / this.startDist;
        if (ratio <= OUT_RATIO) { this.fired = true; this.handlers.onZoomOut(); }
        else if (ratio >= IN_RATIO) { this.fired = true;
this.handlers.onZoomIn(CalendarZoom.mid(e.touches)); }
      }, { passive: true });


      const end = (e) => { if (e.touches.length === 0) this.pinching = false; };
      this.el.addEventListener("touchend", end);
      this.el.addEventListener("touchcancel", end);
    }


    bindWheel() {
      this.el.addEventListener("wheel", (e) => {
        if (!e.ctrlKey) return; // 트랙패드 핀치는 ctrlKey 가 켜진 wheel 이벤트로 들어온다
        e.preventDefault();
        const now = Date.now();
        if (now < this.wheelLock || Math.abs(e.deltaY) < 2) return;
        this.wheelLock = now + 500;
        if (e.deltaY > 0) this.handlers.onZoomOut();
        else this.handlers.onZoomIn({ x: e.clientX, y: e.clientY });
      }, { passive: false });
    }
  }


  DT.CalendarZoom = CalendarZoom;
})();
```

www/js/features/year-view.js

```
// 1 년 학습량 보기: 12 개월을 작은 달력으로 한 화면에 보여주고, 날짜 칸을 학습량 색으로 칠한다
```

```
// 달을 누르면 onPickMonth(월)을 호출해 그 달 보기로 돌아간다
(() => {
  "use strict";
  const { pad, todayKey, fmtMin, fmtMinShort } = DT.utils;


  const TITLE_H = 18; // 작은 달력 제목 줄 높이(px)
  const GAP = 2;      // 날짜 칸 사이 간격(px)


  class YearView {
    constructor(store, container, grid, onPickMonth) {
      this.store = store;
      this.container = container;
      this.grid = grid;
      this.onPickMonth = onPickMonth;
      this.year = new Date().getFullYear();
      window.addEventListener("resize", () => { if (!container.classList.contains("hidden"))
this.layout(); });
    }


    yearMinutes(y) {
      const prefix = `${y}-`;
      return Object.keys(this.store.records).filter((k) => k.startsWith(prefix)).reduce((s, k) => s +
(this.store.records[k].minutes || 0), 0);
    }


    render(year) {
      this.year = year;
      const tk = todayKey();
      let html = "";
      for (let m = 0; m < 12; m++) {
        const first = new Date(year, m, 1).getDay();
        const days = new Date(year, m + 1, 0).getDate();
        const mins = this.store.minutesInMonth(year, m);
        let cells = "";
        for (let i = 0; i < first; i++) cells += `<i class="md empty"></i>`;
        for (let d = 1; d <= days; d++) {
          const k = `${year}-${pad(m + 1)}-${pad(d)}`;
          const r = this.store.get(k);
          const cls = ["md", `lv${this.store.level(r && r.minutes)}`];
          if (k === tk) cls.push("today");
          if (k > tk) cls.push("future");
          if (this.store.isExam(k)) cls.push("exam");
          cells += `<i class="${cls.join(" ")}" title="${m + 1}/${d}${r && r.minutes ? `
${fmtMin(r.minutes)}` : ""}"></i>`;
        }
        const isNow = year === new Date().getFullYear() && m === new Date().getMonth();
        html += `<button type="button" class="mini-month${isNow ? " now" : ""}" data-m="${m}" aria-label="${m
+ 1}월 ${fmtMin(mins)}">
          <span class="mm-title">${m + 1}월${mins ? `<small>${fmtMinShort(mins)}</small>` : ""}</span>
          <span class="mini-days">${cells}</span>
        </button>`;
      }
      this.grid.innerHTML = html;
      this.grid.querySelectorAll(".mini-month").forEach((b) => b.addEventListener("click", () =>
this.onPickMonth(Number(b.dataset.m))));
      this.layout();
    }


    // 화면 크기에 맞춰 2/3/4/6 열 중 날짜 칸이 가장 크게 보이는 배치를 고른다
    layout() {
      const W = this.container.clientWidth, H = this.container.clientHeight;
      if (!W || !H) return;
      const colGap = 10, rowGap = 8;
      let best = { cols: 3, sq: 0 };
      [2, 3, 4, 6].forEach((cols) => {
        const rows = 12 / cols;
        const monthW = (W - (cols - 1) * colGap) / cols;
        const byW = (monthW - 6 * GAP) / 7;
        const byH = ((H - (rows - 1) * rowGap) / rows - TITLE_H - 5 * GAP) / 6;
        const sq = Math.floor(Math.min(byW, byH));
        if (sq > best.sq) best = { cols, sq };
      });
      this.grid.style.setProperty("--sq", `${Math.max(4, best.sq)}px`);
      this.grid.style.setProperty("--md-gap", `${GAP}px`);
```

```
      this.grid.style.gridTemplateColumns = `repeat(${best.cols}, max-content)`;
      this.grid.style.columnGap = `${colGap}px`;
      this.grid.style.rowGap = `${rowGap}px`;
    }


    // 화면 좌표에 있는 달 (두 손가락으로 벌린 위치의 달로 들어가기)
    monthAt(x, y) {
      const el = document.elementFromPoint(x, y);
      const mm = el && el.closest(".mini-month");
      return mm ? Number(mm.dataset.m) : null;
    }
  }


  DT.YearView = YearView;
})();
```

www/js/features/tip-banner.js

```
// 홈 배너: 앱 사용 팁 + 공부 동기부여 문구 + 내 기록에 맞춘 한마디. 터치할 때마다 바뀜
// (시험 D-day 는 위쪽 요약 카드와 캘린더에서 보여주므로 여기선 표시하지 않음)
(() => {
  "use strict";
  const { $, todayKey, fmtMin } = DT.utils;


  // 앱 사용 팁
  const TIPS = [
    "미래 날짜를 누르고 '🎯 시험일로 지정'을 누르면 D-day 가 떠요",
    "캘린더를 두 손가락으로 오므리면 1 년 학습량을 한눈에 볼 수 있어요",
    "캘린더를 좌우로 밀면 다음 달, 이전 달로 넘어가요",
    "타이머 원판을 돌려서 집중 시간을 맞출 수 있어요",
    "집중이 끝나면 쉰 만큼 초록 원판이 차올라요. '다시 집중'으로 이어가요",
    "토요일엔 파란 '이 주 요약', 마지막 날엔 빨간 '이 달 요약'을 확인해 보세요",
    "뽀모도로 기록 칸을 누르면 공부한 파트와 내용을 고칠 수 있어요",
    "설정 ⚙에서 하루 목표 시간을 정하면 달성한 날에 ✓가 붙어요",
    "기록은 이 기기에만 저장돼요. 설정에서 가끔 백업해 두세요",
    "🌓 버튼으로 밝은 테마와 어두운 테마를 바꿀 수 있어요",
    "지난 날짜의 '공부한 시간'에서 그날 타이머 기록을 다시 볼 수 있어요",
  ];


  // 공부 동기부여 문구
  const QUOTES = [
    "오늘 외운 단어 하나가 시험장에서 1 점이 돼요",
    "완벽한 하루보다 꾸준한 하루가 점수를 올려요",
    "25 분만 집중해 봐요. 시작이 제일 어려워요",
    "어제의 나보다 한 문제만 더 맞히면 성공이에요",
    "틀린 문제는 다음에 맞힐 문제예요",
    "쉬고 싶은 마음, 딱 10 분만 미뤄볼까요?",
    "목표 점수는 하루하루 쌓인 공부 시간의 합이에요",
    "Part 5 는 속도, Part 7 은 체력! 꾸준함이 둘 다 키워요",
    "공부한 시간은 배신하지 않아요",
    "작은 진전도 진전이에요. 오늘도 한 칸 채워볼까요?",
    "LC 는 귀가, RC 는 눈이 기억해요. 매일 조금씩!",
    "No pain, no gain — 오늘의 노력이 내일의 점수!",
    "Slow and steady wins the race 🚗",
  ];


  class TipBanner {
    constructor(store) {
      this.store = store;
      this.el = $("examBanner");
```

```
      this.current = null; // 지금 보여주는 문구
      this.el.setAttribute("role", "button");
      this.el.setAttribute("tabindex", "0");
      this.el.setAttribute("aria-label", "팁과 응원 문구 (누르면 바뀜)");
      this.el.addEventListener("click", () => this.next());
      this.el.addEventListener("keydown", (e) => { if (e.key === "Enter" || e.key === " ")
{ e.preventDefault(); this.next(); } });
      store.onChange(() => this.render());
      this.current = this.pick();
      this.render();
    }


    // 내 기록에 맞춘 한마디
    personal() {
      const out = [];
      const s = this.store;
      const streak = s.streak();
      if (streak >= 2) out.push(`${streak}일 연속 공부 중! 오늘도 이어가 볼까요?`);
      const today = s.get(todayKey());
      const min = (today && today.minutes) || 0;
      const goal = s.settings.daily || 0;
      if (!min) out.push("오늘 첫 뽀모도로를 시작해 볼까요? 🍅");
      else if (goal && min >= goal) out.push(`오늘 ${fmtMin(min)} 공부! 목표 달성, 정말 잘했어요 👏`);
      else if (goal) out.push(`오늘 목표까지 ${fmtMin(goal - min)} 남았어요. 조금만 더!`);
      return out.map((text) => ({ kind: "me", text }));
    }


    deck() {
      return [
        ...this.personal(),
        ...TIPS.map((text) => ({ kind: "tip", text })),
        ...QUOTES.map((text) => ({ kind: "quote", text })),
      ];
    }


    // 방금 본 문구는 빼고 무작위로
    pick() {
      const items = this.deck().filter((it) => !this.current || it.text !== this.current.text);
      return items[Math.floor(Math.random() * items.length)];
    }


    next() {
      this.current = this.pick();
      this.render();
      this.el.classList.remove("tip-swap");
      void this.el.offsetWidth; // 애니메이션 다시 시작
      this.el.classList.add("tip-swap");
    }


    render() {
      this.el.classList.remove("none");
      this.el.classList.add("tip-mode");
      const badge = { tip: "💡 팁", quote: "✨ 한마디", me: "🔥 나의 기록" }[this.current.kind];
      this.el.innerHTML = `<span class="tip-text"><b class="tip-kind
${this.current.kind}">${badge}</b>${this.current.text}</span>`;
    }
  }


  DT.TipBanner = TipBanner;
})();
```

www/js/features/day-sheet.js

```
// 날짜를 누르면 아래에서 올라오는 시트: 공부 시작 / 통계 / 기록
//   미래 날짜: "아직 오픈 전" 하나만 (+ 작은 시험일 지정 줄)
//   토요일: 통계가 "이 주 요약"(파란 계열 버튼), 매달 마지막 날: "이 달 요약"(빨간 계열 버튼)
```

```
(() => {
  "use strict";
  const { $, todayKey, parseKey, fmtLong, fmtMin } = DT.utils;
  const { Store } = DT;


  class DaySheet {
    // actions: { study(), studyLog(k), stats(k, scope), record(k), close(), toast(msg) }
    constructor(store, actions) {
      this.name = "day";
      this.store = store;
      this.actions = actions;
      this.el = $("daySheet");
      this.key = null;


      $("sheetBackdrop").addEventListener("click", () => actions.close());
      $("sheetStudy").addEventListener("click", () => this.primaryAction());
      $("sheetStats").addEventListener("click", () => actions.stats(this.key, this.statsScope));
      $("sheetExam").addEventListener("click", () => this.toggleExam());
      $("sheetRecord").addEventListener("click", () => actions.record(this.key));
      this.bindSwipeDown();
      store.onChange(() => { if (this.key) this.render(); });
    }


    // 시트를 아래로 끌어내리면 닫기
    bindSwipeDown() {
      let sy = null;
      const panel = this.el.querySelector(".sheet-panel");
      panel.addEventListener("touchstart", (e) => { sy = e.touches[0].clientY; }, { passive: true });
      panel.addEventListener("touchend", (e) => {
        if (sy != null && e.changedTouches[0].clientY - sy > 60) this.actions.close();
        sy = null;
      });
    }


    // 라우터가 호출: #day/날짜 일 때 열림
    setActive(active, param) {
      if (active && param) {
        this.key = param;
        this.render();
      }
      this.el.classList.toggle("open", active);
      this.el.setAttribute("aria-hidden", String(!active));
    }


    render() {
      const k = this.key;
      const r = this.store.get(k);
      const tk = todayKey();
      $("sheetDate").textContent = fmtLong(k);


      const tags = [];
      if (k === tk) tags.push(`<span class="sheet-tag today">오늘</span>`);
      if (this.store.isExam(k)) tags.push(`<span class="sheet-tag exam">🎯 시험 ${k < tk ? "" :
Store.ddayText(k)}</span>`);
      $("sheetTags").innerHTML = tags.join("");


      const info = [];
      if (r && r.minutes) info.push(`📖 ${fmtMin(r.minutes)}`);
      if (Store.tomatoes(r)) info.push(`🍅 ${Store.tomatoes(r)}개`);
      if (Store.total(r) != null) info.push(`📝 모의고사 ${Store.total(r)}점`);
      if (r && r.vocab) info.push(`단어 ${r.vocab}개`);
      $("sheetInfo").textContent = info.length ? info.join(" · ") : k > tk ? "아직 오지 않은 날이에요" : "아직
기록이 없어요";


      const when = k === tk ? "today" : k < tk ? "past" : "future";
      this.renderPrimary(when, r);
      this.renderStats(k);
```

```
      // 미래 날짜는 "아직 오픈 전" 하나만. 시험일 지정은 아래 작은 줄로
      const future = when === "future";
      $("sheetStats").classList.toggle("hidden", future);
      $("sheetRecord").classList.toggle("hidden", future);
      this.el.querySelector(".sheet-actions").classList.toggle("single", future);
      $("sheetExam").classList.toggle("hidden", !future);
      $("sheetExam").textContent = this.store.isExam(k) ? "🎯 시험일 지정 해제" : "🎯 이 날을 토익 시험일로
지정";
    }


    // 통계 버튼: 매달 마지막 날은 "이 달 요약"(빨강), 토요일은 "이 주 요약"(파랑), 그 외는 기본
    static statsScopeOf(k) {
      const d = parseKey(k);
      const lastDay = new Date(d.getFullYear(), d.getMonth() + 1, 0).getDate();
      if (d.getDate() === lastDay) return "month";
      if (d.getDay() === 6) return "week";
      return null;
    }
    renderStats(k) {
      this.statsScope = DaySheet.statsScopeOf(k);
      const btn = $("sheetStats");
      btn.querySelector(".sub").textContent = this.statsScope === "week" ? "이 주 요약" : "이 달 요약";
      btn.classList.toggle("review-week", this.statsScope === "week");
      btn.classList.toggle("review-month", this.statsScope === "month");
    }


    toggleExam() {
      const on = !this.store.isExam(this.key);
      this.store.setExam(this.key, on);
      this.actions.toast(on ? "🎯 시험일로 지정했어요" : "시험일 지정을 해제했어요");
    }


    // 첫 번째 버튼: 오늘은 타이머로 공부 시작, 지난 날은 타이머로 기록된 공부 시간 보기, 미래는 "아직 오픈 전"
    renderPrimary(when, r) {
      const btn = $("sheetStudy");
      const n = Store.tomatoes(r);
      const look = {
        today:  { ico: "📖", lbl: "공부 시작", sub: "타이머" },
        past:   { ico: "⏱️", lbl: "공부한 시간", sub: n ? `🍅 ${n}개 기록` : "타이머 기록 없음" },
        future: { ico: "🚧", lbl: "아직 오픈 전", sub: "Coming soon…" },
      }[when];
      this.when = when;
      btn.querySelector(".ico").textContent = look.ico;
      btn.querySelector(".lbl").textContent = look.lbl;
      $("sheetStudySub").textContent = look.sub;
      btn.classList.toggle("log", when === "past");
      btn.classList.toggle("closed", when === "future");
    }


    primaryAction() {
      if (this.when === "today") this.actions.study();
      else if (this.when === "past") this.actions.studyLog(this.key);
      else this.joke();
    }


    // 미래 날짜 공부 시작을 누르면 농담 한마디 + 살짝 흔들기
    joke() {
      const lines = [
        "🚧 아직 오픈 전이에요! 오늘 공부부터 해볼까요?",
        "🔓 미래의 나는 아직 출근 전입니다",
        "⏳ 그날이 오면 문을 열어드릴게요",
        "🤡 미리 공부는 안 받아요. 오늘 꺼 먼저!",
        "🛏️ 그날의 나는 아직 자는 중이에요",
      ];
      this.actions.toast(lines[Math.floor(Math.random() * lines.length)]);
      const btn = $("sheetStudy");
      btn.classList.remove("shake");
      void btn.offsetWidth; // 애니메이션 다시 시작
      btn.classList.add("shake");
```

```
    }
  }


  DT.DaySheet = DaySheet;
})();
```

A.3 기록 · 통계

www/js/features/record.js

```
// 기록 화면: 선택한 날짜의 학습 시간, 파트, 단어, 모의고사 점수, 컨디션, 메모, 시험일 지정
(() => {
  "use strict";
  const { $, fmtLong, fmtMin, partChecksHtml, checkedValues, setChecked } = DT.utils;
  const { Store } = DT;


  const clampScore = (v) => (v === "" ? null : Math.max(5, Math.min(495, Math.round(Number(v) / 5) * 5)));


  class RecordScreen {
    constructor(store, sessionDialog, toast, router) {
      this.name = "record";
      this.store = store;
      this.sessions = sessionDialog;
      this.toast = toast;
      this.router = router;
      this.el = $("recordScreen");
      this.key = null;
      this.knownParts = null; // 화면을 연 뒤 세션 기록으로 추가된 파트를 입력칸에 반영하기 위한 기준값


      $("partChecks").innerHTML = partChecksHtml();
      $("fLC").addEventListener("input", () => this.updateTotal());
      $("fRC").addEventListener("input", () => this.updateTotal());
      $("recordForm").addEventListener("submit", (e) => { e.preventDefault(); this.save(); });
      $("btnDelete").addEventListener("click", () => this.remove());
      store.onChange(() => this.syncFromStore());
    }


    setActive(active, param) {
      if (active && param && param !== this.key) this.load(param);
      this.el.classList.toggle("open", active);
      if (!active) this.key = null;
    }


    load(k) {
      this.key = k;
      const r = this.store.get(k) || Store.emptyRecord();
      $("recordTitle").textContent = fmtLong(k);
      $("fExam").checked = this.store.isExam(k);
      this.renderMinutes(r);
      $("fVocab").value = r.vocab || "";
      $("fLC").value = r.lc != null ? r.lc : "";
      $("fRC").value = r.rc != null ? r.rc : "";
      $("fMemo").value = r.memo || "";
      setChecked($("partChecks"), r.parts || []);
      document.querySelectorAll("#moods input").forEach((c) => (c.checked = Number(c.value) === r.mood));
      $("btnDelete").classList.toggle("hidden", !this.store.get(k));
      this.knownParts = (r.parts || []).slice();
      this.renderSessions();
      this.updateTotal();
      this.el.querySelector(".screen-body").scrollTop = 0;
    }


    renderSessions() {
      const r = this.store.get(this.key);
      const list = (r && r.sessions) || [];
      $("daySessionsBox").classList.toggle("hidden", !list.length);
      this.sessions.renderList($("daySessions"), this.key, list, false);
    }
```

```
    // 학습 시간은 공부 타이머로 기록된 시간 (직접 입력 불가)
    renderMinutes(r) {
      const n = Store.tomatoes(r);
      $("fMinutes").textContent = r.minutes ? fmtMin(r.minutes) : "0 분";
      $("fMinutesSub").textContent = n ? `🍅 ${n}개` : "공부 타이머로 공부하면 자동으로 기록돼요";
    }


    // 세션 편집/삭제로 저장된 값이 바뀌면 학습 시간과 목록을 다시 그리고, 새로 추가된 파트는 체크
    syncFromStore() {
      if (!this.key || !this.knownParts) return;
      const r = this.store.get(this.key) || Store.emptyRecord();
      this.renderMinutes(r);
      const added = (r.parts || []).filter((p) => !this.knownParts.includes(p));
      if (added.length) $("partChecks").querySelectorAll("input").forEach((c) => { if
(added.includes(c.value)) c.checked = true; });
      this.knownParts = (r.parts || []).slice();
      this.renderSessions();
    }


    updateTotal() {
      const lc = $("fLC").value, rc = $("fRC").value;
      $("fTotal").textContent = lc !== "" && rc !== "" ? Number(lc) + Number(rc) : "-";
    }


    save() {
      const k = this.key;
      const lc = clampScore($("fLC").value), rc = clampScore($("fRC").value);
      const scoreOk = (lc == null) === (rc == null);
      const mood = document.querySelector("#moods input:checked");
      const prev = this.store.get(k);
      const r = {
        minutes: (prev && prev.minutes) || 0,
        parts: checkedValues($("partChecks")),
        vocab: Math.max(0, Number($("fVocab").value) || 0),
        lc: scoreOk ? lc : null,
        rc: scoreOk ? rc : null,
        mood: mood ? Number(mood.value) : null,
        memo: $("fMemo").value.trim(),
        sessions: (prev && prev.sessions) || [],
      };
      this.key = null; // 저장으로 인한 syncFromStore 무시
      if (this.store.isExam(k) !== $("fExam").checked) this.store.setExam(k, $("fExam").checked);
      this.store.saveRecord(k, r);
      this.toast.show(scoreOk ? "저장했습니다" : "LC 와 RC 를 모두 입력해야 점수가 저장됩니다");
      this.router.back();
    }


    remove() {
      const k = this.key;
      if (!confirm(`${fmtLong(k)} 기록을 삭제할까요?`)) return;
      this.key = null;
      this.store.deleteRecord(k);
      this.toast.show("삭제했습니다");
      this.router.back();
    }
  }


  DT.RecordScreen = RecordScreen;
})();
```

www/js/features/study-log.js

```
// 공부한 시간 화면 (지난 날짜): 그날 공부 타이머로 기록된 시간만 보여준다 (직접 입력 불가)
(() => {
  "use strict";
  const { $, fmtLong, fmtMin, fmtClock, PARTS } = DT.utils;
```

```
  class StudyLogScreen {
    constructor(store, sessionDialog) {
      this.name = "studylog";
      this.store = store;
      this.sessions = sessionDialog;
      this.el = $("studyLogScreen");
      this.key = null;
      store.onChange(() => { if (this.key) this.render(); });
    }

    setActive(active, param) {
      this.key = active ? param : null;
      if (active) {
        this.render();
        this.el.querySelector(".screen-body").scrollTop = 0;
      }
      this.el.classList.toggle("open", active);
    }

    render() {
      const k = this.key;
      const r = this.store.get(k);
      const list = ((r && r.sessions) || []).slice().sort((a, b) => a.start - b.start);
      const total = list.reduce((s, x) => s + x.minutes, 0);
      $("studyLogDate").textContent = fmtLong(k);

      if (!list.length) {
        $("studyLogSummary").innerHTML = "";
        $("studyLogList").innerHTML = `<div class="empty-msg">이 날은 공부 타이머로 기록된 공부가 없어요</div>`;
        return;
      }

      // 파트별 공부 시간
      const byPart = {};
      list.forEach((s) => s.parts.forEach((p) => { byPart[p] = (byPart[p] || 0) + s.minutes; }));
      const partText = PARTS.filter((p) => byPart[p]).map((p) => `<span class="badge">${p}
${fmtMin(byPart[p])}</span>`).join("");

      $("studyLogSummary").innerHTML =
        `<div class="totals">` +
        `<div>공부한 시간<b>${fmtMin(total)}</b></div>` +
        `<div>뽀모도로<b>🍅 ${list.length}개</b></div>` +
        `<div>공부 시간대<b>${fmtClock(list[0].start)}–${fmtClock(list[list.length - 1].end)}</b></div>` +
        `</div>` +
        (partText ? `<div class="log-parts">${partText}</div>` : "");
      this.sessions.renderList($("studyLogList"), k, list, false);
    }
  }

  DT.StudyLogScreen = StudyLogScreen;
})();
```

www/js/features/session-dialog.js

```
// 뽀모도로 세션 기록 창: 집중이 끝나면 "어떤 공부를 했나요?"를 묻고, 지난 세션도 편집/삭제
(() => {
  "use strict";
  const { $, fmtLong, fmtClock, escapeHtml, partChecksHtml, checkedValues, setChecked } = DT.utils;

  class SessionDialog {
    constructor(store, toast) {
      this.store = store;
      this.toast = toast;
      this.dialog = $("sessionDialog");
      this.parts = $("sessionParts");
      this.ref = null;
```

```
      this.parts.innerHTML = partChecksHtml();
      $("sessionForm").addEventListener("submit", () => this.save());
      $("sessionLater").addEventListener("click", () => this.dialog.close());
      $("sessionDelete").addEventListener("click", () => this.remove());
    }


    // justFinished: 방금 끝난 세션이면 축하 문구 + 직전 세션 파트를 기본 선택
    open(k, id, justFinished) {
      const s = this.store.findSession(k, id);
      if (!s) return;
      this.ref = { k, id };
      $("sDoneEmoji").classList.toggle("hidden", !justFinished);
      $("sessionTitle").textContent = justFinished ? "집중 완료!" : "뽀모도로 기록";
      $("sessionSub").textContent = `${fmtLong(k)} ${SessionDialog.timeText(s).replace(/<[^>]+>/g, "")}`;
      const parts = justFinished && !s.parts.length ? this.store.lastSessionParts(k, id) : s.parts;
      setChecked(this.parts, parts);
      $("sessionNote").value = s.note || "";
      $("sessionDelete").classList.toggle("hidden", justFinished);
      $("sessionLater").textContent = justFinished ? "나중에" : "취소";
      if (!this.dialog.open) this.dialog.showModal();
    }


    save() {
      if (!this.ref) return;
      this.store.updateSession(this.ref.k, this.ref.id, checkedValues(this.parts),
$("sessionNote").value.trim());
      this.toast.show("공부 내용을 기록했습니다");
    }


    remove() {
      if (!this.ref || !confirm("이 공부 기록을 삭제할까요? 학습 시간에서도 빠집니다.")) return;
      this.store.deleteSession(this.ref.k, this.ref.id);
      this.dialog.close();
      this.toast.show("삭제했습니다");
    }


    static timeText(s) {
      return `${fmtClock(s.start)}–${fmtClock(s.end)} · ${s.minutes}분${s.rest != null ? ` <span
class="rest">☕ 휴식 ${s.rest}분</span>` : ""}`;
    }


    // 세션 목록 HTML (기록 화면, 타이머 화면에서 공용)
    static rowHtml(k, s) {
      const tags = s.parts.map((p) => `<span class="badge">${p}</span>`).join("");
      const note = s.note ? escapeHtml(s.note) : `<span class="empty-note">공부 내용을 기록해 보세요</span>`;
      return `<div class="session" role="button" tabindex="0" data-edit="${k}|${s.id}" aria-label="기록 편집">
        <span class="tomato">🍅</span>
        <div><div class="s-time">${SessionDialog.timeText(s)} ${tags}</div><div class="s-
note">${note}</div></div>
      </div>`;
    }
    renderList(container, k, sessions, newestFirst) {
      const list = newestFirst ? sessions.slice().reverse() : sessions;
      container.innerHTML = list.map((s) => SessionDialog.rowHtml(k, s)).join("");
      // 기록 칸 전체를 누르면 편집 (키보드는 Enter/Space)
      container.querySelectorAll("[data-edit]").forEach((row) => {
        const open = () => { const [key, id] = row.dataset.edit.split("|"); this.open(key, id, false); };
        row.addEventListener("click", open);
        row.addEventListener("keydown", (e) => { if (e.key === "Enter" || e.key === " ")
{ e.preventDefault(); open(); } });
      });
    }
  }


  DT.SessionDialog = SessionDialog;
})();
```

www/js/features/stats.js

```
// 통계 화면: 선택한 날짜가 속한 달(또는 주)의 요약, 점수 추이, 최근 30 일 학습 시간, 파트별 횟수, 전체 기록 목록
//   param 이 "날짜~week" 이면 그 주(일~토) 요약 + 요일별 그래프
(() => {
  "use strict";
  const { $, pad, keyOf, parseKey, todayKey, fmtLong, fmtMin, fmtMinShort, escapeHtml, PARTS, MOOD_ICON,
WEEK } = DT.utils;
  const { Store } = DT;


  const svg = (w, h, inner) => `<svg viewBox="0 0 ${w} ${h}" role="img">${inner}</svg>`;


  class StatsScreen {
    // onOpenRecord(k): 목록에서 기록을 누르면 호출
    constructor(store, onOpenRecord) {
      this.name = "stats";
      this.store = store;
      this.onOpenRecord = onOpenRecord;
      this.el = $("statsScreen");
      this.key = todayKey();
      this.active = false;
      store.onChange(() => { if (this.active) this.render(); });
    }


    setActive(active, param) {
      this.active = active;
      if (active) {
        const [k, scope] = (param || todayKey()).split("~");
        this.key = k;
        this.scope = scope === "week" ? "week" : "month";
        this.render();
        this.el.querySelector(".screen-body").scrollTop = 0;
      }
      this.el.classList.toggle("open", active);
    }


    get records() { return this.store.records; }


    render() {
      $("weekChart").classList.toggle("hidden", this.scope !== "week");
      if (this.scope === "week") this.renderWeek();
      else this.renderMonth();
      this.renderScoreChart();
      this.renderTimeChart();
      this.renderParts();
      this.renderList();
    }


    // 선택한 날짜가 속한 달 요약
    renderMonth() {
      const d = parseKey(this.key);
      const y = d.getFullYear(), m = d.getMonth();
      const prefix = `${y}-${pad(m + 1)}`;
      const keys = this.store.activeKeys().filter((k) => k.startsWith(prefix));
      const minutes = keys.reduce((s, k) => s + (this.records[k].minutes || 0), 0);
      const tomatoes = keys.reduce((s, k) => s + Store.tomatoes(this.records[k]), 0);
      const goalDays = keys.filter((k) => this.store.settings.daily && this.records[k].minutes >=
this.store.settings.daily).length;
      $("statsMonthTitle").textContent = `${y}년 ${m + 1}월 요약`;
      $("monthTotals").innerHTML =
        `<div>공부한 날<b>${keys.length}일</b></div>` +
        `<div>총 학습 시간<b>${fmtMin(minutes)}</b></div>` +
        `<div>하루 평균<b>${keys.length ? fmtMin(Math.round(minutes / keys.length)) : "-"}</b></div>` +
        `<div>뽀모도로<b>🍅 ${tomatoes}개</b></div>` +
        `<div>목표 달성<b>${goalDays}일</b></div>` +
        `<div>최고 점수<b>${this.monthBest(prefix)}</b></div>`;
    }
```

```
    // 선택한 날짜가 속한 주(일요일~토요일) 요약
    weekKeys() {
      const d = parseKey(this.key);
      d.setDate(d.getDate() - d.getDay());
      const keys = [];
      for (let i = 0; i < 7; i++) { keys.push(keyOf(d)); d.setDate(d.getDate() + 1); }
      return keys;
    }
    renderWeek() {
      const days = this.weekKeys();
      const a = parseKey(days[0]), b = parseKey(days[6]);
      const keys = days.filter((k) => this.store.get(k) && Store.hasActivity(this.store.get(k)));
      const minutes = keys.reduce((s, k) => s + (this.records[k].minutes || 0), 0);
      const tomatoes = keys.reduce((s, k) => s + Store.tomatoes(this.records[k]), 0);
      const daily = this.store.settings.daily;
      const goalDays = keys.filter((k) => daily && this.records[k].minutes >= daily).length;
      const scores = keys.map((k) => Store.total(this.records[k])).filter((v) => v != null);
      $("statsMonthTitle").textContent = `이 주 요약 · ${a.getMonth() + 1}월 ${a.getDate()}일 ~ ${b.getMonth()
+ 1}월 ${b.getDate()}일`;
      $("monthTotals").innerHTML =
        `<div>공부한 날<b>${keys.length}일 / 7 일</b></div>` +
        `<div>총 학습 시간<b>${fmtMin(minutes)}</b></div>` +
        `<div>하루 평균<b>${keys.length ? fmtMin(Math.round(minutes / keys.length)) : "-"}</b></div>` +
        `<div>뽀모도로<b>🍅 ${tomatoes}개</b></div>` +
        `<div>목표 달성<b>${goalDays}일</b></div>` +
        `<div>최고 점수<b>${scores.length ? `${Math.max(...scores)}점` : "-"}</b></div>`;


      // 요일별 학습 시간 막대
      const W = 640, H = 170, L = 10, R = 10, T = 22, B = 26;
      const mins = days.map((k) => (this.records[k] && this.records[k].minutes) || 0);
      const max = Math.max(daily || 0, ...mins, 30);
      const bw = (W - L - R) / 7;
      const tk = todayKey();
      let g = "";
      if (daily) {
        const yy = T + (H - T - B) * (1 - daily / max);
        g += `<line x1="${L}" x2="${W - R}" y1="${yy}" y2="${yy}" stroke="var(--danger)" stroke-dasharray="5
4"/>`;
      }
      days.forEach((k, i) => {
        const h = ((H - T - B) * mins[i]) / max;
        const x = L + i * bw + bw * 0.18, w = bw * 0.64;
        g += `<rect x="${x}" y="${H - B - h}" width="${w}" height="${h}" rx="5" fill="var(--
lv${this.store.level(mins[i])})"><title>${k}  ${fmtMin(mins[i])}</title></rect>`;
        if (mins[i]) g += `<text x="${x + w / 2}" y="${H - B - h - 6}" font-size="12" text-anchor="middle"
fill="var(--text)">${fmtMinShort(mins[i])}</text>`;
        const color = i === 0 ? "var(--sun)" : i === 6 ? "var(--sat)" : "var(--muted)";
        g += `<text x="${x + w / 2}" y="${H - 8}" font-size="13" font-weight="${k === tk ? 800 : 500}" text-
anchor="middle" fill="${color}">${WEEK[i]}</text>`;
      });
      $("weekChart").innerHTML = svg(W, H, g);
    }


    monthBest(prefix) {
      const s = Object.keys(this.records).filter((k) => k.startsWith(prefix)).map((k) =>
Store.total(this.records[k])).filter((v) => v != null);
      return s.length ? `${Math.max(...s)}점` : "-";
    }


    renderScoreChart() {
      const el = $("scoreChart");
      const pts = this.store.activeKeys().filter((k) => Store.total(this.records[k]) != null)
        .map((k) => ({ k, v: Store.total(this.records[k]), lc: this.records[k].lc, rc:
this.records[k].rc }));
      if (!pts.length) { el.innerHTML = `<div class="empty-msg">기록에서 모의고사 점수를 입력하면 추이가
표시됩니다</div>`; return; }


      const W = 640, H = 220, L = 52, R = 20, T = 14, B = 28;
      const goal = this.store.settings.goal || 0;
      const vals = pts.map((p) => p.v).concat(goal ? [goal] : []);
      const lo = Math.max(0, Math.floor((Math.min(...vals) - 50) / 50) * 50);
```

```
      let hi = Math.min(990, Math.ceil((Math.max(...vals) + 50) / 50) * 50);
      if (hi - lo < 100) hi = Math.min(990, lo + 100);
      const x = (i) => (pts.length === 1 ? (L + W - R) / 2 : L + (i * (W - L - R)) / (pts.length - 1));
      const yv = (v) => T + ((hi - v) * (H - T - B)) / (hi - lo);


      let g = "";
      const step = hi - lo <= 300 ? 50 : 100;
      for (let v = Math.ceil(lo / step) * step; v <= hi; v += step) {
        g += `<line x1="${L}" x2="${W - R}" y1="${yv(v)}" y2="${yv(v)}" stroke="var(--line)"/>`;
        g += `<text x="${L - 6}" y="${yv(v) + 4}" font-size="11" text-anchor="end" fill="var(--
muted)">${v}</text>`;
      }
      if (goal && goal >= lo && goal <= hi) {
        g += `<line x1="${L}" x2="${W - R}" y1="${yv(goal)}" y2="${yv(goal)}" stroke="var(--danger)" stroke-
dasharray="5 4"/>`;
        g += `<text x="${W - R}" y="${yv(goal) - 5}" font-size="11" text-anchor="end" fill="var(--
danger)">목표 ${goal}</text>`;
      }
      g += `<path d="${pts.map((p, i) => `${i ? "L" : "M"}${x(i)},${yv(p.v)}`).join(" ")}" fill="none"
stroke="var(--primary)" stroke-width="2.5" stroke-linejoin="round"/>`;
      const labelEvery = Math.ceil(pts.length / 8);
      pts.forEach((p, i) => {
        g += `<circle cx="${x(i)}" cy="${yv(p.v)}" r="4" fill="var(--card)" stroke="var(--primary)" stroke-
width="2"><title>${p.k}  ${p.v}점 (LC ${p.lc} / RC ${p.rc})</title></circle>`;
        g += `<text x="${x(i)}" y="${yv(p.v) - 9}" font-size="11" text-anchor="middle" fill="var(--
text)">${p.v}</text>`;
        if (i % labelEvery === 0 || i === pts.length - 1) {
          const d = parseKey(p.k);
          g += `<text x="${x(i)}" y="${H - 8}" font-size="11" text-anchor="middle" fill="var(--
muted)">${d.getMonth() + 1}/${d.getDate()}</text>`;
        }
      });
      el.innerHTML = svg(W, H, g);
    }


    // 선택한 날짜까지 30 일
    renderTimeChart() {
      const W = 640, H = 150, L = 8, R = 8, T = 10, B = 22;
      const daily = this.store.settings.daily;
      const days = [];
      const d = parseKey(this.key);
      d.setDate(d.getDate() - 29);
      for (let i = 0; i < 30; i++) { days.push(keyOf(d)); d.setDate(d.getDate() + 1); }
      const mins = days.map((k) => (this.records[k] && this.records[k].minutes) || 0);
      const max = Math.max(daily || 0, ...mins, 30);
      const bw = (W - L - R) / 30;
      let g = "";
      if (daily) {
        const yy = T + (H - T - B) * (1 - daily / max);
        g += `<line x1="${L}" x2="${W - R}" y1="${yy}" y2="${yy}" stroke="var(--danger)" stroke-dasharray="5
4"/>`;
      }
      days.forEach((k, i) => {
        const h = ((H - T - B) * mins[i]) / max;
        g += `<rect x="${L + i * bw + 2}" y="${H - B - h}" width="${bw - 4}" height="${h}" rx="3" fill="var(-
-lv${this.store.level(mins[i])})"><title>${k}  ${fmtMin(mins[i])}</title></rect>`;
        if (i % 5 === 0 || i === 29) {
          const dd = parseKey(k);
          g += `<text x="${L + i * bw + bw / 2}" y="${H - 6}" font-size="10" text-anchor="middle" fill="var(-
-muted)">${dd.getMonth() + 1}/${dd.getDate()}</text>`;
        }
      });
      $("timeChart").innerHTML = svg(W, H, g);
    }


    renderParts() {
      const counts = Object.fromEntries(PARTS.map((p) => [p, 0]));
      Object.values(this.records).forEach((r) => (r.parts || []).forEach((p) => { if (p in counts)
counts[p]++; }));
      const max = Math.max(1, ...Object.values(counts));
      $("partChart").innerHTML = PARTS.map((p) =>
        `<div class="part-bar"><span>${p}</span><div class="track"><div class="fill"
style="width:${(counts[p] / max) * 100}%"></div></div><span class="cnt">${counts[p]}</span></div>`
      ).join("");
    }
```

```
    renderList() {
      const el = $("recordList");
      const keys = this.store.activeKeys().reverse();
      if (!keys.length) { el.innerHTML = `<div class="empty-msg">아직 기록이 없습니다</div>`; return; }
      el.innerHTML = keys.map((k) => {
        const r = this.records[k];
        const badges = (r.parts || []).map((p) => `<span class="badge">${p}</span>`).join("") +
          (Store.total(r) != null ? `<span class="badge score">${Store.total(r)}점</span>` : "") +
          (Store.tomatoes(r) ? `<span class="badge">🍅 ${Store.tomatoes(r)}</span>` : "");
        const meta = [r.minutes ? fmtMin(r.minutes) : "", r.vocab ? `단어 ${r.vocab}개` : "", r.mood ?
MOOD_ICON[r.mood] : ""].filter(Boolean).join(" · ");
        return `<div class="rec" data-k="${k}">
          <div class="rec-top"><span class="rec-date">${fmtLong(k)}</span><span class="rec-
meta">${meta}</span></div>
          <div>${badges}</div>
          ${r.memo ? `<div class="rec-memo">${escapeHtml(r.memo)}</div>` : ""}
        </div>`;
      }).join("");
      el.querySelectorAll(".rec").forEach((n) => n.addEventListener("click", () =>
this.onOpenRecord(n.dataset.k)));
    }
  }


  DT.StatsScreen = StatsScreen;
})();
```

A.4 타이머 · 소리 · 알람

www/js/features/pomodoro.js

```
// 뽀모도로 타이머 로직 (화면과 분리)
//   집중: 정해 둔 시간에서 줄어드는 타이머. 시작하면 높은 "삐삐삐", 끝나면 "삐삐삐" + 공부 기록
//   휴식: 시간 설정 없이 쉰 만큼 올라가는 스톱워치. "다시 집중"을 누르면 휴식 시간을 기록하고 집중 시작
// 실행 중에는 종료/시작 시각을 저장하므로 앱을 닫았다 열어도 이어진다
(() => {
  "use strict";


  const TIMER_KEY = "dailyToeic.timer";
  const MODE_NAME = { focus: "집중", break: "휴식" };


  const LATE_MS = 3000; // 끝난 지 이만큼 지나서야 처리되면(앱을 나중에 연 경우) 소리 없이 기록만


  // 알람: 시작/끝 "삐삐삐"(SoundFx) + 끝날 때 진동
  //   안드로이드 앱에서는 NativeNotifier 가 끝나는 시각에 시스템 알림을 예약 → 앱을 닫아도 울림
  class Alarm {
    constructor() {
      this.fx = new DT.SoundFx();
      this.notifier = new DT.NativeNotifier();
    }


    // 브라우저 정책상 사용자가 버튼을 누를 때 오디오를 깨워 둬야 나중에 소리가 난다
    unlock() {
      this.fx.unlock();
      if (!this.notifier.native && "Notification" in window && Notification.permission === "default")
Notification.requestPermission().catch(() => {});
    }


    startSound() { this.fx.startBeep(); }


    scheduleEnd(at) { this.notifier.scheduleFocusEnd(at); }
    cancelEnd() { this.notifier.cancelFocusEnd(); }
```

```
    ring(title, body, endAt) {
      const late = Date.now() - endAt > LATE_MS;
      const covered = this.notifier.coveredBy(endAt); // 시스템 알림이 이미 소리·진동을 냄
      this.notifier.scheduledAt = 0;
      // 이미 울린 알림 정리 (나중에 연 경우 바로, 화면을 보고 있었으면 알림 소리가 끝난 뒤)
      if (this.notifier.native) setTimeout(() => this.notifier.dismissFocusEnd(), late ? 0 : 4000);
      if (late || covered) return;
      this.fx.endBeep();
      if (navigator.vibrate) navigator.vibrate([120, 60, 120, 60, 200]);
      try {
        if (document.hidden && "Notification" in window && Notification.permission === "granted") new
Notification(title, { body, icon: "icon.svg" });
      } catch (e) {}
    }
  }


  class PomodoroTimer {
    // events: { onUpdate(), onFocusDone({k, s}) }
    constructor(store, events) {
      this.store = store;
      this.events = events;
      this.alarm = new Alarm();
      this.tickId = null;
      const saved = DT.Store.read(TIMER_KEY, {});
      // 예전 버전의 짧은/긴 휴식 상태는 새 집중으로 초기화
      if (saved.mode && saved.mode !== "focus" && saved.mode !== "break") Object.assign(saved, { mode:
"focus", running: false, remaining: null });
      this.state = Object.assign(
        { mode: "focus", running: false, endAt: 0, remaining: null, focusStart: 0, breakStart: 0, last:
null },
        saved
      );
      if (this.state.remaining == null) this.state.remaining = this.duration();
      document.addEventListener("visibilitychange", () => { if (!document.hidden) this.tick(); });
    }


    // 저장된 상태로 재개 (앱을 닫아둔 사이에 집중이 끝났으면 바로 완료 처리)
    resume() {
      if (this.state.mode === "break") this.startTicking();
      else if (this.state.running) {
        if (Date.now() >= this.state.endAt) this.finish(this.state.endAt);
        else this.startTicking();
      }
    }


    get pomo() { return this.store.settings.pomo; }
    get mode() { return this.state.mode; }
    get running() { return this.state.running; }
    get onBreak() { return this.state.mode === "break"; }
    duration() { return this.pomo.focus * 60000; }
    remaining() { return this.state.running ? Math.max(0, this.state.endAt - Date.now()) :
this.state.remaining; }
    breakElapsed() { return this.onBreak ? Math.max(0, Date.now() - this.state.breakStart) : 0; }
    isFresh() { return !this.onBreak && !this.state.running && this.state.remaining === this.duration(); }
    static modeName(mode) { return MODE_NAME[mode]; }


    persist() { this.store.write(TIMER_KEY, this.state); }
    update() { this.persist(); this.events.onUpdate(); }


    startTicking() {
      clearInterval(this.tickId);
      this.tickId = setInterval(() => this.tick(), 250);
    }


    tick() {
      if (!this.onBreak && this.state.running && Date.now() >= this.state.endAt)
this.finish(this.state.endAt);
      else this.events.onUpdate();
    }


    // ---------- 집중 ----------
    start() {
```

```
    if (this.onBreak) return;
    this.alarm.unlock();
    this.alarm.startSound();
    const st = this.state;
    st.running = true;
    st.endAt = Date.now() + st.remaining;
    if (!st.focusStart) st.focusStart = Date.now();
    this.alarm.scheduleEnd(st.endAt); // 앱을 닫아도 끝나는 시각에 알림
    this.startTicking();
    this.update();
  }

  pause() {
    if (this.onBreak) return;
    this.state.remaining = this.remaining();
    this.state.running = false;
    clearInterval(this.tickId);
    this.alarm.cancelEnd();
    this.update();
  }

  toggle() { this.state.running ? this.pause() : this.start(); }

  // 집중을 처음 상태(설정 시간 가득)로
  reset() {
    clearInterval(this.tickId);
    this.alarm.cancelEnd();
    Object.assign(this.state, { mode: "focus", running: false, remaining: this.duration(), focusStart:
0 });
    this.update();
  }

  // 다이얼 드래그로 집중 시간 변경 (멈춰 있을 때만)
  setMinutes(min, persistSettings) {
    if (this.state.running || this.onBreak) return;
    this.store.updatePomo({ focus: min }, !persistSettings);
    this.state.remaining = min * 60000;
    this.state.focusStart = 0;
    if (persistSettings) this.persist();
    this.events.onUpdate();
  }

  // 입력칸으로 집중 시간 변경: 아직 시작 전이면 바로 반영
  setDuration(min) {
    const fresh = this.isFresh();
    this.store.updatePomo({ focus: min });
    if (fresh) this.reset();
    else this.events.onUpdate();
  }

  // 집중 시간이 다 됨: 기록 → 알람 → 휴식(스톱워치) 시작
  finish(endAt) {
    clearInterval(this.tickId);
    const rec = this.store.addSession(this.state.focusStart || endAt - this.duration(), endAt);
    this.alarm.ring("집중 완료! 🍅", "휴식 중이에요. 다시 집중할 땐 '다시 집중'을 눌러 주세요.", endAt);
    this.enterBreak(endAt, rec);
    this.events.onFocusDone(rec);
  }

  enterBreak(at, rec) {
    Object.assign(this.state, {
      mode: "break", running: false, breakStart: at, focusStart: 0,
      last: rec ? { k: rec.k, id: rec.s.id } : null,
    });
    this.startTicking();
    this.update();
  }

  // 집중 중 그만두기: 1분 이상 했으면 그만큼 기록할지 묻고 휴식으로
  skip(confirmFn) {
    if (this.onBreak) return null;
```

```
      const doneMs = this.duration() - this.remaining();
      if (doneMs >= 60000) {
        if (!confirmFn(`지금까지 집중한 ${Math.floor(doneMs / 60000)}분을 기록하고 휴식할까요?`)) return null;
        clearInterval(this.tickId);
        this.alarm.cancelEnd();
        const end = Date.now();
        const rec = this.store.addSession(end - doneMs, end);
        this.enterBreak(end, rec);
        return rec;
      }
      if (confirmFn("1 분이 안 돼서 기록되지 않아요. 집중을 처음부터 다시 할까요?")) this.reset();
      return null;
    }


    // ---------- 휴식 → 다시 집중 ----------
    refocus() {
      if (!this.onBreak) return;
      const restMin = Math.round(this.breakElapsed() / 60000);
      if (this.state.last) this.store.setSessionRest(this.state.last.k, this.state.last.id, restMin);
      Object.assign(this.state, { mode: "focus", running: false, remaining: this.duration(), focusStart: 0,
breakStart: 0, last: null });
      this.start();
    }
  }


  DT.PomodoroTimer = PomodoroTimer;
})();
```

www/js/features/timer-screen.js

```
// 공부(타이머) 화면: 다이얼, 시작/일시정지, 색상, 시간 설정, 오늘의 뽀모도로 목록
//   집중: 원판이 줄어듦 → 끝나면 알람
//   휴식: 색이 바뀌고 쉰 만큼 원판이 차오름 → "다시 집중"을 누르면 집중 시간만큼 다시 차오르며 시작
// 화면을 닫아도 타이머는 계속 돌고, 홈 상단에 남은(휴식은 지난) 시간이 표시된다
(() => {
  "use strict";
  const { $, todayKey, fmtMin, fmtTimer, fmtShort } = DT.utils;
  const { PomodoroTimer, TimeTimerDial, WindSound } = DT;


  const FILL_MS = 1100;         // 다시 집중할 때 집중 시간만큼 원판이 차오르는 시간
  const BREAK_DIAL = 60 * 60000; // 휴식 원판은 60 분 눈금 기준으로 차오름


  class TimerScreen {
    constructor(store, sessionDialog, toast, router) {
      this.name = "timer";
      this.store = store;
      this.sessions = sessionDialog;
      this.toast = toast;
      this.router = router;
      this.el = $("timerScreen");
      this.wrap = this.el.querySelector(".timer-wrap");
      this.active = false;
      this.lastMode = null; // 집중/휴식이 바뀌었는지 알기 위한 이전 값
      this.fill = null;     // 차오르는 애니메이션 상태


      this.timer = new PomodoroTimer(store, {
        onUpdate: () => this.render(),
        onFocusDone: ({ k, s }) => this.sessions.open(k, s.id, true),
      });


      // 시간을 조절할 때 나는 "뜨르륵" 태엽 소리
      this.sound = new WindSound(DT.SOUNDS.windUp, () => this.store.settings.pomo.sound);


      this.dial = new TimeTimerDial(
        { box: $("ttBox"), svg: $("ttDial"), ticks: $("ttTicks"), wedge: $("ttWedge"), hand: $("ttHand") },
        {
```

```
          onDragStart: () => this.sound.prepare(),
          onDrag: (min) => {
            // 1분이 바뀔 때마다 소리 한 조각
            if (min !== this.store.settings.pomo.focus) this.sound.tick();
            this.timer.setMinutes(min, false);
            $("oFocus").value = min;
          },
          onDragEnd: () => this.timer.setMinutes(this.store.settings.pomo.focus, true),
        }
      );


      this.bindControls();
      this.bindOptions();
      this.bindColors();
      $("timerChip").addEventListener("click", () => this.router.go("timer"));
      store.onChange(() => this.renderSessions());


      this.timer.resume();
      this.render();
      this.renderSessions();
    }


    setActive(active) {
      this.active = active;
      this.el.classList.toggle("open", active);
      if (active) this.render();
    }


    bindControls() {
      // 버튼을 누를 때 소리도 깨워 둬야, 나중에 원판이 차오를 때 "뜨르륵"이 난다
      $("tStart").addEventListener("click", () => {
        this.sound.unlock();
        if (this.timer.onBreak) this.timer.refocus(); // 다시 집중: 집중 시간만큼 차오르며 시작
        else this.timer.toggle();
      });
      $("tReset").addEventListener("click", () => {
        this.sound.unlock();
        this.timer.reset();
        this.startFill();
      });
      $("tSkip").addEventListener("click", () => {
        const rec = this.timer.skip((msg) => confirm(msg));
        if (rec) this.sessions.open(rec.k, rec.s.id, true);
      });
    }


    bindOptions() {
      const pomo = this.store.settings.pomo;
      const input = $("oFocus");
      input.value = pomo.focus;
      input.addEventListener("change", () => {
        const v = Math.max(1, Math.min(120, Math.round(Number(input.value)) || pomo.focus));
        input.value = v;
        if (v !== pomo.focus) { this.sound.unlock(); this.sound.burst(); }
        this.timer.setDuration(v);
      });
      $("oSound").checked = pomo.sound;
      $("oSound").addEventListener("change", () => {
        this.store.updatePomo({ sound: $("oSound").checked });
        if ($("oSound").checked) { this.sound.unlock(); this.sound.burst(); } // 켜면 미리 들려주기
      });
    }


    bindColors() {
      const box = $("ttColors");
      box.innerHTML = Object.entries(TimeTimerDial.COLORS).map(([k, c]) =>
        `<button type="button" role="radio" data-color="${k}" title="${c.name}" aria-label="${c.name}"
style="background:${c.main}"></button>`).join("");
      box.querySelectorAll("button").forEach((b) =>
        b.addEventListener("click", () => { this.store.updatePomo({ color: b.dataset.color });
this.render(); })
      );
    }
```

```
    // 원판이 0에서 집중 시간까지 차오르는 애니메이션 (색은 CSS 전환으로 함께 바뀜)
    startFill() {
      if (!this.active) return;
      this.fill = { start: performance.now(), to: this.timer.duration(), lastMin: 0 };
      const step = () => {
        if (!this.fill) return;
        this.render();
        if (this.fill) requestAnimationFrame(step);
      };
      requestAnimationFrame(step);
    }


    // 애니메이션 중이면 원판에 보여줄 시간(ms), 아니면 null
    fillValue() {
      if (!this.fill) return null;
      const p = Math.min(1, (performance.now() - this.fill.start) / FILL_MS);
      const v = this.fill.to * (1 - Math.pow(1 - p, 3));
      // 차오르는 동안 1분 지날 때마다 태엽 소리
      const min = Math.floor(v / 60000);
      if (min > this.fill.lastMin) { this.sound.tick(); this.fill.lastMin = min; }
      if (p >= 1) { this.fill = null; return null; }
      return v;
    }


    // 휴식은 구분되도록 그린 (집중 색이 그린이면 틸)
    colorKey() {
      const focus = this.store.settings.pomo.color in TimeTimerDial.COLORS ? this.store.settings.pomo.color :
"red";
      if (!this.timer.onBreak) return focus;
      return focus === "green" ? "teal" : "green";
    }


    render() {
      const t = this.timer;
      const onBreak = t.onBreak;
      const rem = t.remaining();
      const elapsed = t.breakElapsed();
      const txt = onBreak ? fmtTimer(elapsed) : fmtTimer(rem);


      // 홈 상단 칩 (타이머 진행 중이거나 휴식 중일 때)
      const chip = $("timerChip");
      chip.classList.toggle("hidden", !onBreak && !t.running && t.isFresh());
      chip.textContent = onBreak ? `☕ +${txt}` : `🍅 ${txt}${t.running ? "" : " ⏸"}`;
      TimeTimerDial.applyColor(chip, this.colorKey());
      document.title = onBreak ? `☕ 휴식 ${txt} · Daily TOEIC` : t.running ? `${txt} 🍅 Daily TOEIC` :
"Daily TOEIC";


      // 휴식 → 집중으로 바뀌면 집중 시간만큼 원판이 차오른다
      const toFocus = this.lastMode === "break" && t.mode === "focus";
      this.lastMode = t.mode;
      if (!this.active) return;
      if (toFocus) this.startFill();


      TimeTimerDial.applyColor(this.wrap, this.colorKey());
      this.wrap.classList.toggle("on-break", onBreak);
      const shown = this.fillValue();
      if (onBreak) this.dial.render(Math.min(elapsed, BREAK_DIAL), BREAK_DIAL); // 쉰 만큼 차오름
      else this.dial.render(shown != null ? shown : rem, t.duration());
      this.dial.setDraggable(!onBreak && !t.running && !this.fill);


      $("ringLabel").textContent = PomodoroTimer.modeName(t.mode);
      $("ringTime").textContent = onBreak ? `+${txt}` : txt;
      $("ttHint").textContent = onBreak ? "쉰 만큼 원판이 채워져요" : "원판을 드래그해서 집중 시간을 맞출 수
있어요";
      $("ttHint").style.visibility = onBreak || !t.running ? "visible" : "hidden";
      $("tStart").textContent = onBreak ? "다시 집중" : t.running ? "일시정지" : rem < t.duration() ? "계속" :
"시작";
```

```
      $("tReset").classList.toggle("hidden", onBreak);
      $("tSkip").classList.toggle("hidden", onBreak);
      const focusColor = this.store.settings.pomo.color;
      $("ttColors").querySelectorAll("button").forEach((b) => b.setAttribute("aria-checked",
String(b.dataset.color === focusColor)));
    }


    renderSessions() {
      const k = todayKey();
      const r = this.store.get(k);
      const list = (r && r.sessions) || [];
      const sum = list.reduce((a, s) => a + s.minutes, 0);
      $("todaySessionTitle").textContent = `오늘의 뽀모도로 · ${fmtShort(k)}`;
      $("todaySessionSum").textContent = list.length ? `${list.length}개 · ${fmtMin(sum)}` : "";
      $("ringCycle").textContent = `오늘 🍅 ${DT.Store.tomatoes(r)}개`;
      if (list.length) this.sessions.renderList($("todaySessions"), k, list, true);
      else $("todaySessions").innerHTML = `<div class="empty-msg">집중 세션을 마치면 여기에 기록됩니다</div>`;
    }
  }


  DT.TimerScreen = TimerScreen;
})();
```

www/js/features/timer-dial.js

```
// 타임타이머 스타일 다이얼(SVG): 남은 시간만큼 색 원판이 채워지고 점점 줄어든다
// 멈춰 있을 때 원판을 드래그해서 시간을 맞출 수 있다
(() => {
  "use strict";


  const SIZE = 240, CX = 120, CY = 120, R_WEDGE = 80;


  // 타임타이머 제품 색상 참고: main=원판, frame=테두리, knob=가운데 손잡이
  const COLORS = {
    red:   { name: "레드",   main: "#e2432a", frame: "#f8b49c", knob: "#f9c3b1" },
    pink:  { name: "핑크",   main: "#e06aa3", frame: "#f5d0d4", knob: "#f6d9dc" },
    navy:  { name: "네이비", main: "#283152", frame: "#b3bed1", knob: "#c8d0dd" },
    green: { name: "그린",   main: "#2e6a48", frame: "#80c08e", knob: "#a8d8b1" },
    teal:  { name: "틸",     main: "#0e5a6c", frame: "#a5d9e5", knob: "#bfe4ec" },
  };


  // 0이 맨 위, 숫자는 반시계 방향으로 증가 (실제 타임타이머와 동일)
  const polar = (deg, r) => [CX - r * Math.sin((deg * Math.PI) / 180), CY - r * Math.cos((deg * Math.PI) /
180)];


  class TimeTimerDial {
    // els: { box, svg, ticks, wedge, hand }, handlers: { onDragStart(), onDrag(min), onDragEnd() }
    constructor(els, handlers) {
      this.els = els;
      this.handlers = handlers;
      this.max = 0;
      this.draggable = true;
      this.dragging = false;
      this.lastDragMin = null;
      this.bindDrag();
    }


    // 60분 또는 120분 눈금
    setMax(max) {
      if (this.max === max) return;
      this.max = max;
      const degPerMin = 360 / max;
      const labelStep = max === 60 ? 5 : 10;
      let g = "";
      for (let m = 0; m < max; m++) {
```

```
        const major = m % labelStep === 0;
        const [x1, y1] = polar(m * degPerMin, major ? 82 : 85);
        const [x2, y2] = polar(m * degPerMin, 90);
        if (max === 60 || m % 2 === 0) g += `<line class="tt-tick" x1="${x1}" y1="${y1}" x2="${x2}"
y2="${y2}" stroke-width="${major ? 2.6 : 1.3}"/>`;
        if (major) {
          const [tx, ty] = polar(m * degPerMin, 103);
          g += `<text class="tt-num" x="${tx}" y="${ty}" text-anchor="middle" dominant-
baseline="central">${m}</text>`;
        }
      }
      this.els.ticks.innerHTML = g;
    }


    wedgePath(min) {
      const frac = Math.max(0, Math.min(1, min / this.max));
      if (frac <= 0) return "";
      if (frac >= 0.9999) return `M${CX - R_WEDGE},${CY} a${R_WEDGE},${R_WEDGE} 0 1,0 ${R_WEDGE * 2},0
a${R_WEDGE},${R_WEDGE} 0 1,0 ${-R_WEDGE * 2},0 Z`;
      const [ex, ey] = polar(frac * 360, R_WEDGE);
      // 맨 위에서 시작해 반시계 방향(sweep=0)으로 남은 시간만큼
      return `M${CX},${CY} L${CX},${CY - R_WEDGE} A${R_WEDGE},${R_WEDGE} 0 ${frac > 0.5 ? 1 : 0},0
${ex},${ey} Z`;
    }


    render(remainingMs, durationMs) {
      this.setMax(Math.max(durationMs, remainingMs) > 60 * 60000 ? 120 : 60);
      const remMin = remainingMs / 60000;
      this.els.wedge.setAttribute("d", this.wedgePath(remMin));
      const [hx, hy] = polar((Math.min(remMin, this.max) / this.max) * 360, R_WEDGE);
      this.els.hand.setAttribute("x2", hx);
      this.els.hand.setAttribute("y2", hy);
    }


    setDraggable(on) {
      this.draggable = on;
      this.els.box.classList.toggle("draggable", on);
    }


    // CSS 변수로 색 지정 (target 요소 아래 전체에 적용)
    static applyColor(target, key) {
      const c = COLORS[key] || COLORS.red;
      target.style.setProperty("--tt-main", c.main);
      target.style.setProperty("--tt-frame", c.frame);
      target.style.setProperty("--tt-knob", c.knob);
    }


    minutesFromPointer(e) {
      const rect = this.els.svg.getBoundingClientRect();
      const dx = ((e.clientX - rect.left) / rect.width) * SIZE - CX;
      const dy = ((e.clientY - rect.top) / rect.height) * SIZE - CY;
      let deg = (Math.atan2(-dx, -dy) * 180) / Math.PI;
      if (deg < 0) deg += 360;
      const max = this.max, last = this.lastDragMin;
      let m = Math.round((deg / 360) * max);
      // 0 근처에서 최대값 ↔ 0 으로 튀지 않게
      if (last != null && last > max * 0.75 && m < max * 0.25) m = max;
      if (last != null && last < max * 0.25 && m > max * 0.75) m = 1;
      if (m === 0) m = last > max / 2 ? max : 1;
      return Math.max(1, Math.min(max, m));
    }


    bindDrag() {
      const svg = this.els.svg;
      const move = (e) => {
        const m = this.minutesFromPointer(e);
        this.lastDragMin = m;
        this.handlers.onDrag(m);
      };
      svg.addEventListener("pointerdown", (e) => {
        if (!this.draggable) return;
        this.dragging = true;
        this.lastDragMin = null;
        this.els.box.classList.add("dragging");
```

```
        svg.setPointerCapture(e.pointerId);
        if (this.handlers.onDragStart) this.handlers.onDragStart();
        move(e);
      });
      svg.addEventListener("pointermove", (e) => { if (this.dragging) move(e); });
      const end = () => {
        if (!this.dragging) return;
        this.dragging = false;
        this.els.box.classList.remove("dragging");
        this.handlers.onDragEnd();
      };
      svg.addEventListener("pointerup", end);
      svg.addEventListener("pointercancel", end);
    }
  }


  TimeTimerDial.COLORS = COLORS;
  DT.TimeTimerDial = TimeTimerDial;
})();
```

www/js/features/wind-sound.js

```
// 타이머 조절 효과음: 태엽 감는 "뜨르륵" 소리를 잘게 나눠, 1 분 조절할 때마다 한 조각씩 이어서 재생
// 빠르게 돌리면 원래 소리처럼 "뜨르르륵", 천천히 돌리면 칸마다 "뜨륵"
(() => {
  "use strict";


  // sounds/wind-up.js 클립(0.33 초) 안의 조각 시작 위치(초). 딸깍 소리 3 개씩 묶음
  const GRAINS = [0.0095, 0.0705, 0.119, 0.165, 0.218, 0.2615];
  const CLIP_END = 0.33;
  const RESTART_AFTER = 0.45; // 이만큼 쉬었다가 다시 돌리면 "뜨"(처음)부터
  const MAX_QUEUE = 0.06;     // 너무 빨리 돌리면 밀린 조각은 건너뜀(초)


  class WindSound {
    // isEnabled(): 설정에서 소리를 켰는지
    constructor(base64, isEnabled) {
      this.isEnabled = isEnabled;
      this.ctx = null;
      this.buffer = null;
      this.index = 0;
      this.nextTime = 0;
      this.lastTick = 0;
      this.load(base64);
    }


    load(base64) {
      try {
        this.ctx = DT.audio.get(); // 앱 공용 오디오 (첫 탭/클릭 때 자동으로 깨워짐)
        if (!this.ctx) return;
        const bin = atob(base64);
        const bytes = new Uint8Array(bin.length);
        for (let i = 0; i < bin.length; i++) bytes[i] = bin.charCodeAt(i);
        this.ctx.decodeAudioData(bytes.buffer, (buf) => { this.buffer = buf; }, () => {});
      } catch (e) {
        this.ctx = null; // 오디오를 지원하지 않는 환경이면 조용히 무시
      }
    }


    // 브라우저 정책상 사용자가 화면을 탭할 때 오디오를 깨워야 한다
    unlock() { DT.audio.unlock(); }


    // 아직 깨워지지 않았거나 스피커 연결을 다시 여는 중이면 이번 소리는 건너뜀 (밀린 소리가 한꺼번에 나지 않도록)
    ready() {
      if (!this.ctx || !this.buffer || !this.isEnabled()) return false;
      DT.audio.wake();
      if (DT.audio.waking) return false;
      if (this.ctx.state !== "running") { DT.audio.unlock(); return false; }
      return true;
```

```
    }


    // 원판을 잡는 순간 미리 연결을 열어 두면, 돌리기 시작할 때 바로 소리가 난다
    prepare() {
      DT.audio.unlock();
      DT.audio.wake();
    }


    play(start, end, when) {
      const src = this.ctx.createBufferSource();
      const gain = this.ctx.createGain();
      const len = end - start;
      // 잘린 부분이 '틱' 튀지 않도록 아주 짧게 페이드
      gain.gain.setValueAtTime(0, when);
      gain.gain.linearRampToValueAtTime(0.9, when + 0.003);
      gain.gain.setValueAtTime(0.9, when + len - 0.006);
      gain.gain.linearRampToValueAtTime(0, when + len);
      src.buffer = this.buffer;
      src.connect(gain).connect(this.ctx.destination);
      src.start(when, start, len);
    }


    // 1 분 조절할 때마다 호출: 다음 조각 재생
    tick() {
      if (!this.ready()) return;
      const now = this.ctx.currentTime;
      if (now - this.lastTick > RESTART_AFTER) this.index = 0;
      this.lastTick = now;
      const when = Math.max(now, this.nextTime);
      if (when - now > MAX_QUEUE) return;
      const start = GRAINS[this.index];
      const end = GRAINS[this.index + 1] || CLIP_END;
      this.play(start, end, when);
      this.nextTime = when + (end - start);
      this.index = (this.index + 1) % GRAINS.length;
    }


    // 숫자로 시간을 바꿨을 때: "뜨르륵" 한 번 전체
    burst() {
      if (!this.ctx || !this.buffer || !this.isEnabled()) return;
      if (DT.audio.waking || Date.now() - DT.audio.lastUse > 10000) { DT.audio.wake().then(() =>
this.burst()); return; }
      if (!this.ready()) return;
      const when = Math.max(this.ctx.currentTime, this.nextTime);
      this.play(GRAINS[0], CLIP_END, when);
      this.nextTime = when + (CLIP_END - GRAINS[0]);
      this.index = 0;
    }
  }


  DT.WindSound = WindSound;
})();
```

www/js/features/sfx.js

```
// 타이머 효과음 (파일 없이 Web Audio 로 합성)
//   endBeep():   집중이 끝날 때 "삐삐삐" 한 번
//   startBeep(): 시작할 때 "삐삐삐"를 부드러운 사인파로, START_PITCH 반음 올리고 START_SPEED 배 빠르게 한 번
(() => {
  "use strict";


  const START_PITCH = 7;  // 시작음은 끝음보다 반음 7 개(5 도) 높게 — 너무 높으면 날카로움
  const START_SPEED = 3;  // 시작음은 끝음보다 3 배 빠르게


  class SoundFx {
    constructor() { this.ctx = null; }
```

```
    // 앱 공용 오디오를 사용 (첫 탭/클릭 때 자동으로 깨워짐)
    unlock() {
      DT.audio.unlock();
      this.ctx = DT.audio.get();
    }


    // "삐삐삐": 0.18초 간격 세 음 (마지막 음이 조금 높음). semitones 만큼 피치를 올리고 speed 배 빠르게
    // wave: "triangle"(또렷함) / "sine"(부드러움, 날카롭지 않음)
    beep(semitones, speed = 1, wave = "triangle") {
      this.ctx = this.ctx || DT.audio.get();
      if (!this.ctx) return;
      // 앱 공용 오디오면 오래 조용했을 때 스피커 연결을 다시 연 뒤 재생 (테스트용 다른 오디오는 바로 재생)
      if (this.ctx === DT.audio.ctx && !this.waited) {
        DT.audio.wake().then(() => { this.waited = true; this.beep(semitones, speed, wave); this.waited =
false; });
        return;
      }
      const shift = Math.pow(2, semitones / 12);
      const k = 1 / speed; // 간격과 길이를 함께 줄임
      const t0 = this.ctx.currentTime + 0.01;
      [0, 0.18, 0.36].forEach((d, i) => {
        const at = t0 + d * k;
        const o = this.ctx.createOscillator(), g = this.ctx.createGain();
        o.type = wave;
        o.frequency.value = (i === 2 ? 1318 : 1046) * shift;
        g.gain.setValueAtTime(0.0001, at);
        // 아주 짧은 음도 '틱' 튀지 않도록 커지는 시간은 최소 6ms
        g.gain.exponentialRampToValueAtTime(0.35, at + Math.max(0.006, 0.015 * k));
        g.gain.exponentialRampToValueAtTime(0.0001, at + 0.16 * k);
        o.connect(g).connect(this.ctx.destination);
        o.start(at);
        o.stop(at + 0.18 * k);
      });
    }


    endBeep() { this.beep(0); }
    startBeep() { this.beep(START_PITCH, START_SPEED, "sine"); }
  }


  DT.SoundFx = SoundFx;
})();
```

www/js/features/native-notifier.js

```
// 앱을 닫아도 집중 종료 알람이 울리도록 휴대폰 시스템 알림을 예약 (안드로이드 앱에서만 동작)
//   집중 시작 → 끝나는 시각에 알림 예약 / 일시정지·처음부터·끝내기 → 예약 취소
//   웹(브라우저)에서는 아무 일도 하지 않음 (active === false)
(() => {
  "use strict";


  const NOTIFY_ID = 7001;            // 집중 종료 알림 (하나만 사용)
  const CHANNEL_ID = "focus-end";    // 알림 채널: 소리/진동 설정 묶음
  const SOUND = "focus_end.wav";     // android/app/src/main/res/raw/focus_end.wav ("삐삐삐")


  class NativeNotifier {
    constructor() {
      const cap = window.Capacitor;
      this.native = !!(cap && cap.isNativePlatform && cap.isNativePlatform());
      this.plugin = null;
      this.granted = false;
      this.scheduledAt = 0; // 지금 예약된 알림 시각 (없으면 0)
      if (!this.native) return;
      this.plugin = (cap.Plugins && cap.Plugins.LocalNotifications) ||
cap.registerPlugin("LocalNotifications");
      this.ready = this.setup();
```

```
}


// 알림이 실제로 울릴 수 있는 상태인지 (앱 + 알림 허용)
get active() { return this.native && this.granted; }


async setup() {
  try {
    const st = await this.plugin.checkPermissions();
    this.granted = st.display === "granted";
    await this.plugin.createChannel({
      id: CHANNEL_ID,
      name: "집중 종료 알림",
      description: "집중 시간이 끝나면 알려줘요",
      importance: 5,        // 소리 + 화면 위 팝업
      visibility: 1,        // 잠금 화면에도 표시
      sound: SOUND,
      vibration: true,
    });
  } catch (e) {
    console.warn("[notify] 준비 실패", e);
  }
}


// 처음 집중을 시작할 때 알림 허용을 물어봄 (안드로이드 13+)
async ensurePermission() {
  if (!this.native) return false;
  await this.ready;
  if (this.granted) return true;
  try {
    const st = await this.plugin.requestPermissions();
    this.granted = st.display === "granted";
  } catch (e) {
    this.granted = false;
  }
  return this.granted;
}


async scheduleFocusEnd(at) {
  if (!this.native || !(at > Date.now())) return;
  if (!(await this.ensurePermission())) return;
  try {
    await this.plugin.cancel({ notifications: [{ id: NOTIFY_ID }] });
    await this.plugin.schedule({
      notifications: [{
        id: NOTIFY_ID,
        title: "집중 완료! 🫖",
        body: "수고했어요. 앱을 열어 공부한 내용을 기록하고 휴식하세요.",
        channelId: CHANNEL_ID,
        sound: SOUND,
        smallIcon: "ic_stat_notify",
        iconColor: "#2F6FED",
        schedule: { at: new Date(at), allowWhileIdle: true }, // 절전 모드에서도 정확한 시각에
      }],
    });
    this.scheduledAt = at;
  } catch (e) {
    console.warn("[notify] 예약 실패", e);
  }
}


async cancelFocusEnd() {
  if (!this.native) return;
  this.scheduledAt = 0;
  try { await this.plugin.cancel({ notifications: [{ id: NOTIFY_ID }] }); } catch (e) {}
}


// 이미 울린 알림을 알림창에서 지움 (앱을 열어 확인했으므로). 예약 취소(cancel)로는 울린 알림이 안 지워짐
async dismissFocusEnd() {
  if (!this.native) return;
  this.scheduledAt = 0;
```

```
      try {
        await this.plugin.cancel({ notifications: [{ id: NOTIFY_ID }] });
        await this.plugin.removeDeliveredNotifications({ notifications: [{ id: NOTIFY_ID }] });
      } catch (e) {}
    }


    // 끝나는 시각에 알림이 예약돼 있었는지 (그렇다면 앱 안 소리는 생략해 두 번 울리지 않게)
    coveredBy(endAt) {
      return this.active && this.scheduledAt > 0 && Math.abs(this.scheduledAt - endAt) < 2000;
    }
  }


  DT.NativeNotifier = NativeNotifier;
})();
```

A.5 설정 · 테마 · 계정

www/js/features/settings.js

```
// 설정 창: 목표 점수, 하루 목표 시간, 백업 내보내기/가져오기
(() => {
  "use strict";
  const { $, todayKey } = DT.utils;


  class SettingsDialog {
    constructor(store, toast) {
      this.store = store;
      this.toast = toast;
      this.dialog = $("settingsDialog");


      $("btnSettings").addEventListener("click", () => this.open());
      $("btnSettingsCancel").addEventListener("click", () => this.dialog.close());
      $("settingsForm").addEventListener("submit", () => this.save());
      $("btnExport").addEventListener("click", () => this.exportBackup());
      $("btnImport").addEventListener("click", () => $("importFile").click());
      $("importFile").addEventListener("change", (e) => this.importBackup(e));
    }


    open() {
      $("sGoal").value = this.store.settings.goal || "";
      $("sDaily").value = this.store.settings.daily || "";
      this.dialog.showModal();
    }


    save() {
      this.store.updateSettings({ goal: Number($("sGoal").value) || 0, daily: Number($("sDaily").value) ||
0 });
      this.toast.show("설정을 저장했습니다");
    }


    exportBackup() {
      const blob = new Blob([this.store.exportJson()], { type: "application/json" });
      const a = document.createElement("a");
      a.href = URL.createObjectURL(blob);
      a.download = `daily-toeic-backup-${todayKey()}.json`;
      a.click();
      setTimeout(() => URL.revokeObjectURL(a.href), 1000);
    }


    async importBackup(e) {
      const file = e.target.files[0];
      e.target.value = "";
      if (!file) return;
      try {
        const data = JSON.parse(await file.text());
        const n = Object.keys((data && data.records) || {}).length;
```

```
        if (!confirm(`${n}개의 기록을 가져옵니다. 같은 날짜의 기존 기록은 덮어씁니다. 계속할까요?`)) return;
        this.store.importData(data);
        this.dialog.close();
        this.toast.show(`${n}개 기록을 가져왔습니다`);
      } catch (err) {
        this.toast.show("백업 파일을 읽을 수 없습니다");
      }
    }
  }


  DT.SettingsDialog = SettingsDialog;
})();
```

www/js/features/theme.js

```
// 밝은/어두운 테마 전환 (시스템 설정 → 밝게 → 어둡게 순환)
(() => {
  "use strict";


  const THEME_KEY = "dailyToeic.theme";
  const THEMES = [
    { id: "system", icon: "◐", name: "시스템 설정" },
    { id: "light", icon: "☼", name: "밝은 테마" },
    { id: "dark", icon: "☾", name: "어두운 테마" },
  ];


  class ThemeManager {
    constructor(button, toast) {
      this.button = button;
      this.toast = toast;
      this.theme = "system";
      try { this.theme = localStorage.getItem(THEME_KEY) || "system"; } catch (e) {}
      this.button.addEventListener("click", () => this.next());
      this.apply();
    }


    get current() { return THEMES.find((t) => t.id === this.theme) || THEMES[0]; }


    apply() {
      const t = this.current;
      if (t.id === "system") document.documentElement.removeAttribute("data-theme");
      else document.documentElement.setAttribute("data-theme", t.id);
      this.button.textContent = t.icon;
      this.button.title = `테마: ${t.name}`;
    }


    next() {
      const i = THEMES.findIndex((t) => t.id === this.theme);
      this.theme = THEMES[(i + 1) % THEMES.length].id;
      try { localStorage.setItem(THEME_KEY, this.theme); } catch (e) {}
      this.apply();
      this.toast.show(`${this.current.name}로 변경했습니다`);
    }
  }


  DT.ThemeManager = ThemeManager;
})();
```

www/js/features/account-panel.js

```
// 설정 창의 "계정" 칸: 로그인 상태, 로그인/로그아웃 버튼, 마지막 동기화 시각
// 지금은 cloud-config.js 의 enabled 가 false 라서 "준비 중"으로만 보인다
(() => {
```

```
  "use strict";
  const { $, escapeHtml, fmtClock } = DT.utils;


  class AccountPanel {
    constructor(account, sync) {
      this.account = account;
      this.sync = sync;
      this.el = $("accountBox");
      account.onChange(() => this.render());
      sync.onChange(() => this.render());
      this.el.addEventListener("click", (e) => {
        const act = e.target.closest("[data-account]");
        if (!act) return;
        if (act.dataset.account === "signin") this.account.signIn();
        if (act.dataset.account === "signout" && confirm("로그아웃할까요? 이 기기의 기록은 그대로 남아요."))
this.account.signOut();
      });
      this.render();
    }


    render() {
      const a = this.account;
      let html;
      switch (a.status) {
        case "disabled":
          html = `<p class="account-status">🔒 계정 연동은 준비 중이에요</p>
            <button type="button" class="account-btn" disabled>Google 계정으로 로그인</button>
            <p class="hint">로그인하면 기록이 계정에 저장되어 휴대폰을 바꿔도 이어서 볼 수 있게 될
예정이에요.</p>`;
          break;
        case "loading":
          html = `<p class="account-status">계정 정보를 불러오는 중…</p>`;
          break;
        case "signedOut":
          html = `<p class="account-status">로그인하면 기록이 계정에 저장돼요</p>
            <button type="button" class="account-btn" data-account="signin">Google 계정으로 로그인</button>`;
          break;
        case "signedIn": {
          const u = a.user;
          const when = this.sync.busy ? "동기화 중…" : this.sync.lastSyncedAt ? `마지막 동기화
${fmtClock(this.sync.lastSyncedAt)}` : "동기화 대기 중";
          html = `<p class="account-status">☑ ${escapeHtml(u.name || u.email ||
"로그인됨")}<br><small>${escapeHtml(u.email || "")}</small></p>
            <p class="hint">${when}</p>
            <button type="button" class="account-btn ghost" data-account="signout">로그아웃</button>`;
          break;
        }
        default:
          html = `<p class="account-status">⚠ 계정 연결에 문제가 있어요</p><p
class="hint">${escapeHtml(a.error || "")}</p>`;
      }
      this.el.innerHTML = html;
    }
  }


  DT.AccountPanel = AccountPanel;
})();
```

A.6 앱 시작 · 화면 · 스타일

www/js/app.js

```
// 앱 시작: 각 기능 객체를 만들고 서로 연결한다
(() => {
  "use strict";
  const { $ } = DT.utils;
```

```
  class App {
    constructor() {
      this.toast = new DT.Toast($("toast"));
      this.store = new DT.Store((msg) => this.toast.show(msg));
      this.router = new DT.Router();


      this.theme = new DT.ThemeManager($("btnTheme"), this.toast);
      this.settings = new DT.SettingsDialog(this.store, this.toast);


      // 계정 로그인 + 클라우드 동기화 (cloud-config.js 의 enabled 가 false 면 아무 동작 안 함)
      this.account = new DT.AccountService(DT.CLOUD_CONFIG);
      this.sync = new DT.CloudSync(this.store, this.account);
      this.accountPanel = new DT.AccountPanel(this.account, this.sync);
      this.account.init();
      this.sessionDialog = new DT.SessionDialog(this.store, this.toast);


      this.calendar = new DT.CalendarView(this.store, (k) => this.router.go("day", k));
      this.tips = new DT.TipBanner(this.store);
      this.sheet = new DT.DaySheet(this.store, {
        study: () => this.router.go("timer"),
        studyLog: (k) => this.router.go("studylog", k),
        stats: (k, scope) => this.router.go("stats", scope === "week" ? `${k}~week` : k),
        record: (k) => this.router.go("record", k),
        close: () => this.router.back(),
        toast: (msg) => this.toast.show(msg),
      });
      this.timer = new DT.TimerScreen(this.store, this.sessionDialog, this.toast, this.router);
      this.stats = new DT.StatsScreen(this.store, (k) => this.router.go("record", k));
      this.record = new DT.RecordScreen(this.store, this.sessionDialog, this.toast, this.router);
      this.studyLog = new DT.StudyLogScreen(this.store, this.sessionDialog);


      // 각 화면을 라우터에 등록 (홈 캘린더는 항상 아래에 깔려 있음)
      [this.sheet, this.timer, this.stats, this.record, this.studyLog].forEach((v) =>
this.router.register(v));
      // 선택된 날짜 표시는 시트/기록/통계 화면이 열려 있는 동안 유지
      this.router.register({ name: "__selection", setActive: () => {
        const { name, param } = this.router.current;
        const k = param && param.split("~")[0];
        this.calendar.setSelected(["day", "stats", "record", "studylog"].includes(name) ? k : null);
      } });


      // 화면 왼쪽 위 ‹ 버튼
      document.querySelectorAll("[data-back]").forEach((b) => b.addEventListener("click", () =>
this.router.back()));


      this.calendar.render();
      this.router.start();
      this.registerServiceWorker();
    }


    registerServiceWorker() {
      // 안드로이드 앱(Capacitor) 안에서는 파일이 앱에 들어 있으므로 오프라인 캐시가 필요 없음
      const native = window.Capacitor && window.Capacitor.isNativePlatform &&
window.Capacitor.isNativePlatform();
      if (native) return;
      if ("serviceWorker" in navigator && location.protocol.startsWith("http")) {
        navigator.serviceWorker.register("service-worker.js").catch(() => {});
      }
    }
  }


  DT.app = new App();
})();
```

www/index.html

```
<!doctype html>
<html lang="ko">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover">
  <meta name="theme-color" content="#2f6fed">
  <title>Daily TOEIC</title>
  <link rel="manifest" href="manifest.json">
  <link rel="icon" href="icon.svg" type="image/svg+xml">
  <link rel="stylesheet" href="style.css">
  <script>
    // 화면 깜빡임 방지: 저장된 테마를 렌더링 전에 적용
    try {
      var t = localStorage.getItem("dailyToeic.theme");
      if (t === "light" || t === "dark") document.documentElement.setAttribute("data-theme", t);
    } catch (e) {}
  </script>
</head>
<body>
  <!-- ===== 홈: 한 화면에 요약 + D-day + 캘린더 ===== -->
  <div class="home">
    <header class="topbar">
      <h1>Daily TOEIC</h1>
      <div class="top-actions">
        <button id="timerChip" class="timer-chip hidden" type="button" title="타이머로 이동"></button>
        <button id="btnTheme" class="icon-btn" aria-label="테마 변경" title="테마 변경">🌓</button>
        <button id="btnSettings" class="icon-btn" aria-label="설정" title="설정">⚙</button>
      </div>
    </header>


    <main class="home-main">
      <section class="summary">
        <div class="stat stat-exam"><span class="stat-label">시험</span><span class="stat-value"
id="statDday">-</span></div>
        <div class="stat"><span class="stat-label">연속 학습</span><span class="stat-value"
id="statStreak">0 일</span></div>
        <div class="stat"><span class="stat-label" id="statMonthLabel">이번 달</span><span class="stat-value"
id="statMonth">0 분</span></div>
        <div class="stat"><span class="stat-label">최고 점수</span><span class="stat-value" id="statBest">-
</span></div>
      </section>


      <section class="panel calendar">
        <div class="exam-banner" id="examBanner"></div>
        <div class="cal-head">
          <button id="prevMonth" class="icon-btn" aria-label="이전 달">‹</button>
          <h2 id="monthLabel"></h2>
          <button id="nextMonth" class="icon-btn" aria-label="다음 달">›</button>
          <button id="zoomBtn" class="small-btn" type="button">1 년</button>
          <button id="todayBtn" class="small-btn">오늘</button>
        </div>
        <!-- 한 달 보기 -->
        <div class="cal-month" id="calMonth">
          <div class="cal-grid cal-weekdays">
            <div class="sun">일</div><div>월</div><div>화</div><div>수</div><div>목</div><div>금</div><div
class="sat">토</div>
          </div>
          <div class="cal-grid cal-days" id="calGrid"></div>
        </div>
        <!-- 1 년 보기 (두 손가락으로 오므리면) -->
        <div class="year-view hidden" id="yearView">
          <div class="year-grid" id="yearGrid"></div>
        </div>
        <div class="legend">
          <span>학습량</span>
          <i class="lv lv0"></i><i class="lv lv1"></i><i class="lv lv2"></i><i class="lv lv3"></i><i
class="lv lv4"></i>
```

```
          <span class="legend-dot"><b class="dot"></b> 모의고사 <b class="exam-sq"></b> 시험일</span>
        </div>
      </section>
    </main>
  </div>


  <!-- ===== 날짜 바텀시트: 공부 시작 / 통계 / 기록 ===== -->
  <div class="sheet" id="daySheet" aria-hidden="true">
    <div class="sheet-backdrop" id="sheetBackdrop"></div>
    <div class="sheet-panel" role="dialog" aria-labelledby="sheetDate">
      <div class="sheet-handle"></div>
      <div class="sheet-head">
        <h2 id="sheetDate"></h2>
        <div id="sheetTags"></div>
      </div>
      <p class="sheet-info" id="sheetInfo"></p>
      <div class="sheet-actions">
        <button type="button" id="sheetStudy" class="sheet-btn study">
          <span class="ico">📖</span><span class="lbl">공부 시작</span><span class="sub"
id="sheetStudySub">타이머</span>
        </button>
        <button type="button" id="sheetStats" class="sheet-btn stats">
          <span class="ico">📊</span><span class="lbl">통계</span><span class="sub">이 달 요약</span>
        </button>
        <button type="button" id="sheetRecord" class="sheet-btn record">
          <span class="ico">📝</span><span class="lbl">기록</span><span class="sub">이 날 기록</span>
        </button>
      </div>
      <button type="button" id="sheetExam" class="sheet-exam hidden"></button>
    </div>
  </div>


  <!-- ===== 공부(타이머) 화면 ===== -->
  <section class="screen" id="timerScreen" aria-label="공부 타이머">
    <header class="screen-head">
      <button type="button" class="icon-btn" data-back aria-label="뒤로">‹</button>
      <h2>공부 타이머</h2>
      <span class="screen-head-spacer"></span>
    </header>
    <div class="screen-body">
      <div class="timer-wrap">


        <!-- 타임타이머 스타일: 남은 시간만큼 색 원판이 채워지고 점점 줄어듦 -->
        <div class="tt" id="ttBox">
          <div class="tt-face">
            <svg id="ttDial" viewBox="0 0 240 240" role="img" aria-label="타이머 다이얼">
              <g id="ttTicks"></g>
              <path id="ttWedge" class="tt-wedge"/>
              <line id="ttHand" class="tt-hand" x1="120" y1="120" x2="120" y2="40"/>
              <circle class="tt-knob-shadow" cx="121" cy="122" r="15"/>
              <circle class="tt-knob" cx="120" cy="120" r="14"/>
            </svg>
          </div>
        </div>
        <div class="tt-readout">
          <span class="ring-label" id="ringLabel">집중</span>
          <span class="ring-time" id="ringTime">25:00</span>
          <span class="ring-cycle" id="ringCycle">오늘 🍅 0개</span>
        </div>
        <p class="tt-hint" id="ttHint">원판을 드래그해서 집중 시간을 맞출 수 있어요</p>


        <div class="timer-controls">
          <button type="button" id="tReset" title="처음으로">↺</button>
          <button type="button" id="tStart" class="main-btn">시작</button>
          <button type="button" id="tSkip" title="지금까지 기록하고 휴식하기">⏭</button>
        </div>
        <div class="tt-colors" id="ttColors" role="radiogroup" aria-label="타이머 색상"></div>
```

```
        <details class="timer-settings">
          <summary>시간 설정</summary>
          <div class="timer-opts single">
            <label>집중 시간 (분)<input type="number" id="oFocus" min="1" max="120"
inputmode="numeric"></label>
          </div>
          <label class="timer-check"><input type="checkbox" id="oSound"> 시간 조절할 때 태엽 소리
(뜨르륵)</label>
        </details>
      </div>


      <div class="session-head">
        <h3 id="todaySessionTitle">오늘의 뽀모도로</h3>
        <span id="todaySessionSum"></span>
      </div>
      <div class="sessions" id="todaySessions"></div>
    </div>
  </section>


  <!-- ===== 통계 화면 ===== -->
  <section class="screen" id="statsScreen" aria-label="통계">
    <header class="screen-head">
      <button type="button" class="icon-btn" data-back aria-label="뒤로">‹</button>
      <h2>통계</h2>
      <span class="screen-head-spacer"></span>
    </header>
    <div class="screen-body">
      <h3 id="statsMonthTitle">이 달 요약</h3>
      <div class="totals" id="monthTotals"></div>
      <div id="weekChart" class="chart week-chart hidden"></div>
      <h3>모의고사 점수 추이</h3>
      <div id="scoreChart" class="chart"></div>
      <h3>최근 30 일 학습 시간</h3>
      <div id="timeChart" class="chart"></div>
      <h3>파트별 학습 횟수</h3>
      <div id="partChart" class="parts-bars"></div>
      <h3>전체 기록</h3>
      <div id="recordList"></div>
    </div>
  </section>


  <!-- ===== 기록 화면 ===== -->
  <section class="screen" id="recordScreen" aria-label="기록">
    <header class="screen-head">
      <button type="button" class="icon-btn" data-back aria-label="뒤로">‹</button>
      <h2 id="recordTitle">기록</h2>
      <span class="screen-head-spacer"></span>
    </header>
    <form class="screen-body record-form" id="recordForm">
      <label class="exam-check"><input type="checkbox" id="fExam"> 🎯 이 날은 토익 시험일</label>


      <div class="minutes-readonly">
        <span class="mr-label">학습 시간 <small>(공부 타이머 기록)</small></span>
        <span class="mr-value" id="fMinutes">0 분</span>
        <span class="mr-sub" id="fMinutesSub"></span>
      </div>


      <fieldset>
        <legend>공부한 파트</legend>
        <div class="part-checks" id="partChecks"></div>
      </fieldset>


      <label>외운 단어 수
        <input type="number" id="fVocab" min="0" inputmode="numeric">
      </label>
```

```
      <fieldset>
        <legend>모의고사 점수 (선택)</legend>
        <div class="score-row">
          <label>LC <input type="number" id="fLC" min="5" max="495" step="5" inputmode="numeric"
placeholder="0~495"></label>
          <label>RC <input type="number" id="fRC" min="5" max="495" step="5" inputmode="numeric"
placeholder="0~495"></label>
          <div class="score-total">합계 <b id="fTotal">-</b></div>
        </div>
      </fieldset>


      <fieldset>
        <legend>컨디션</legend>
        <div class="moods" id="moods">
          <label><input type="radio" name="mood" value="1"><span>😫</span></label>
          <label><input type="radio" name="mood" value="2"><span>😕</span></label>
          <label><input type="radio" name="mood" value="3"><span>😐</span></label>
          <label><input type="radio" name="mood" value="4"><span>🙂</span></label>
          <label><input type="radio" name="mood" value="5"><span>😄</span></label>
        </div>
      </fieldset>


      <fieldset id="daySessionsBox">
        <legend>뽀모도로 기록</legend>
        <div class="sessions" id="daySessions"></div>
      </fieldset>


      <label>메모 / 오답 노트
        <textarea id="fMemo" rows="4" placeholder="틀린 문제, 새로 알게 된 표현 등"></textarea>
      </label>


      <div class="record-actions">
        <button type="button" id="btnDelete" class="danger">삭제</button>
        <span class="spacer"></span>
        <button type="submit" class="primary">저장</button>
      </div>
    </form>
  </section>


  <!-- ===== 공부한 시간 화면 (지난 날짜: 타이머로 기록된 시간 보기) ===== -->
  <section class="screen" id="studyLogScreen" aria-label="공부한 시간">
    <header class="screen-head">
      <button type="button" class="icon-btn" data-back aria-label="뒤로">‹</button>
      <h2>공부한 시간</h2>
      <span class="screen-head-spacer"></span>
    </header>
    <div class="screen-body">
      <h3 id="studyLogDate"></h3>
      <div id="studyLogSummary"></div>
      <h3>공부 타이머 기록</h3>
      <div class="sessions" id="studyLogList"></div>
    </div>
  </section>


  <!-- ===== 뽀모도로 완료 / 세션 편집 창 ===== -->
  <dialog id="sessionDialog">
    <form method="dialog" id="sessionForm">
      <p class="done-emoji" id="sDoneEmoji">🍅</p>
      <h2 id="sessionTitle" style="text-align:center">집중 완료!</h2>
      <p class="done-sub" id="sessionSub"></p>


      <fieldset>
        <legend>어떤 공부를 했나요?</legend>
        <div class="part-checks" id="sessionParts"></div>
```

```
      </fieldset>

      <label>공부 내용
        <textarea id="sessionNote" rows="3" placeholder="예: 해커스 RC 3회 Part 5 풀이, 오답 12개
정리"></textarea>
      </label>

      <div class="dialog-actions">
        <button type="button" id="sessionDelete" class="danger hidden">삭제</button>
        <span class="spacer"></span>
        <button type="button" id="sessionLater">나중에</button>
        <button type="submit" class="primary">저장</button>
      </div>
    </form>
  </dialog>

  <!-- ===== 설정 창 ===== -->
  <dialog id="settingsDialog">
    <form method="dialog" id="settingsForm">
      <h2>설정</h2>
      <p class="hint" style="margin:0">시험 일정은 캘린더에서 미래 날짜를 누르고 '🎯 시험일로 지정'을 누르면
등록됩니다.</p>
      <label>목표 점수 <input type="number" id="sGoal" min="10" max="990" step="5"
inputmode="numeric"></label>
      <label>하루 목표 학습 시간 (분) <input type="number" id="sDaily" min="0" step="1"
inputmode="numeric"></label>

      <fieldset>
        <legend>계정</legend>
        <div id="accountBox" class="account-box"></div>
      </fieldset>

      <fieldset>
        <legend>백업</legend>
        <div class="backup-row">
          <button type="button" id="btnExport">내보내기 (JSON)</button>
          <button type="button" id="btnImport">가져오기</button>
          <input type="file" id="importFile" accept="application/json,.json" hidden>
        </div>
        <p class="hint">기록은 이 기기에 저장됩니다. 기기를 바꾸거나 데이터를 지우기 전에 내보내기 해두세요.</p>
      </fieldset>

      <div class="dialog-actions">
        <span class="spacer"></span>
        <button type="button" id="btnSettingsCancel">닫기</button>
        <button type="submit" class="primary">저장</button>
      </div>
    </form>
  </dialog>

  <div id="toast" class="toast"></div>

  <!-- 공통 → 기능별 → 앱 시작 순서로 로드 -->
  <script src="js/core/utils.js"></script>
  <script src="js/core/audio.js"></script>
  <script src="js/core/store.js"></script>
  <script src="js/core/router.js"></script>
  <script src="js/core/cloud-config.js"></script>
  <script src="js/core/firebase-provider.js"></script>
  <script src="js/core/account.js"></script>
  <script src="js/core/cloud-sync.js"></script>
  <script src="js/features/theme.js"></script>
  <script src="js/features/calendar-zoom.js"></script>
  <script src="js/features/year-view.js"></script>
```

```
  <script src="js/features/calendar.js"></script>
  <script src="js/features/tip-banner.js"></script>
  <script src="js/features/day-sheet.js"></script>
  <script src="js/features/session-dialog.js"></script>
  <script src="js/features/study-log.js"></script>
  <script src="js/features/record.js"></script>
  <script src="sounds/wind-up.js"></script>
  <script src="js/features/wind-sound.js"></script>
  <script src="js/features/timer-dial.js"></script>
  <script src="js/features/sfx.js"></script>
  <script src="js/features/native-notifier.js"></script>
  <script src="js/features/pomodoro.js"></script>
  <script src="js/features/timer-screen.js"></script>
  <script src="js/features/stats.js"></script>
  <script src="js/features/settings.js"></script>
  <script src="js/features/account-panel.js"></script>
  <script src="js/app.js"></script>
</body>
</html>
```

www/style.css

```
:root {
  --bg: #f5f7fb;
  --card: #ffffff;
  --text: #1d2330;
  --muted: #6b7385;
  --line: #e3e7ef;
  --primary: #2f6fed;
  --primary-weak: #e8efff;
  --danger: #d6455d;
  --sun: #d6455d;
  --sat: #2f6fed;
  --lv0: #eef1f6;
  --lv1: #cfe0ff;
  --lv2: #93b6ff;
  --lv3: #5288f5;
  --lv4: #2457c7;
  --dot: #ff9f1c;
  --focus: #e5533d;
  --focus-weak: #fde9e5;
  --break: #1f9d6c;
  --break-weak: #e1f5ec;
  --radius: 14px;
  color-scheme: light;
}
/* 다크 테마: 시스템 설정이 다크이고 사용자가 라이트를 고르지 않았을 때 */
@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) {
    --bg: #12151c;
    --card: #1c212b;
    --text: #e8ecf3;
    --muted: #949cad;
    --line: #2c3340;
    --primary: #5b8dff;
    --primary-weak: #23314f;
    --lv0: #252b37;
    --lv1: #23365e;
    --lv2: #2c4f96;
    --lv3: #3f6fd6;
    --lv4: #6d9bff;
    --focus: #ff7a63;
    --focus-weak: #3d2420;
    --break: #3ccf94;
    --break-weak: #1a3a2e;
    color-scheme: dark;
  }
}
/* 다크 테마: 사용자가 직접 다크를 골랐을 때 */
:root[data-theme="dark"] {
  --bg: #12151c;
  --card: #1c212b;
  --text: #e8ecf3;
  --muted: #949cad;
  --line: #2c3340;
  --primary: #5b8dff;
  --primary-weak: #23314f;
```

```
  --lv0: #252b37;
  --lv1: #23365e;
  --lv2: #2c4f96;
  --lv3: #3f6fd6;
  --lv4: #6d9bff;
  --focus: #ff7a63;
  --focus-weak: #3d2420;
  --break: #3ccf94;
  --break-weak: #1a3a2e;
  color-scheme: dark;
}


* { box-sizing: border-box; }
html, body { margin: 0; height: 100%; }
body {
  overflow: hidden;
  background: var(--bg);
  color: var(--text);
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", "Malgun Gothic", "Apple SD Gothic Neo", sans-
serif;
  -webkit-tap-highlight-color: transparent;
}
button, input, textarea { font: inherit; color: inherit; }
button { cursor: pointer; }


.topbar {
  flex: none; display: flex; align-items: center; justify-content: space-between;
  padding: 8px 12px 8px 16px; padding-top: max(8px, env(safe-area-inset-top));
  background: var(--primary); color: #fff;
}
.topbar h1 { margin: 0; font-size: 1.2rem; letter-spacing: .3px; }
.topbar .icon-btn { color: #fff; }


/* 홈: 화면 높이(100dvh)에 딱 맞추고 캘린더가 남는 공간을 채움 */
.home { height: 100vh; height: 100dvh; display: flex; flex-direction: column; }
.home-main {
  flex: 1; min-height: 0; width: 100%; max-width: 720px; margin: 0 auto;
  display: flex; flex-direction: column; gap: 8px;
  padding: 10px 12px; padding-bottom: max(10px, env(safe-area-inset-bottom));
}


.icon-btn {
  background: none; border: none; font-size: 1.5rem; line-height: 1;
  padding: 4px 10px; border-radius: 8px; color: var(--text);
}
.icon-btn:hover { background: rgba(127,127,127,.12); }
.small-btn {
  border: 1px solid var(--line); background: var(--card); border-radius: 8px;
  padding: 4px 10px; font-size: .85rem;
}


/* 요약 */
.summary {
  flex: none; display: grid; gap: 6px;
  /* 시험 카운트다운이 한 줄로 들어가도록 첫 칸을 조금 넓게 */
  grid-template-columns: minmax(0, 1.45fr) repeat(3, minmax(0, 1fr));
}
.stat {
  background: var(--card); border-radius: 12px; padding: 6px 4px;
  display: flex; flex-direction: column; align-items: center; gap: 2px;
  box-shadow: 0 1px 3px rgba(0,0,0,.05);
}
.stat-label { font-size: .68rem; color: var(--muted); white-space: nowrap; }
.stat-value { font-size: .98rem; font-weight: 700; white-space: nowrap; }


/* 탭 */
.panel { background: var(--card); border-radius: var(--radius); padding: 14px; box-shadow: 0 1px 3px
rgba(0,0,0,.05); }
.hidden { display: none !important; }


/* 캘린더 */
.panel.calendar { flex: 1; min-height: 0; display: flex; flex-direction: column; padding: 10px; }
```

```
.cal-head { flex: none; display: flex; align-items: center; gap: 6px; margin-bottom: 4px; }
.cal-head h2 { flex: 1; text-align: center; margin: 0; font-size: 1.15rem; }
.cal-grid { display: grid; grid-template-columns: repeat(7, minmax(0, 1fr)); gap: 5px; }
.cal-weekdays { flex: none; }
.cal-weekdays div { text-align: center; font-size: .75rem; color: var(--muted); padding: 2px 0 4px; }
.cal-days { flex: 1; min-height: 0; }
.cal-weekdays .sun { color: var(--sun); }
.cal-weekdays .sat { color: var(--sat); }


.day {
  position: relative; min-height: 0; overflow: hidden; border: none; border-radius: 10px;
  background: var(--lv0); padding: 4px; text-align: left;
  display: flex; flex-direction: column; justify-content: space-between;
  transition: transform .08s;
}
.day:active { transform: scale(.95); }
.day.empty { background: transparent; pointer-events: none; }
.day .num { font-size: .82rem; font-weight: 600; }
.day.sun .num { color: var(--sun); }
.day.sat .num { color: var(--sat); }
.day .min { font-size: .66rem; text-align: right; opacity: .9; }
.day.today { outline: 2px solid var(--primary); outline-offset: 1px; }
.day.selected { outline: 3px solid var(--text); outline-offset: 1px; opacity: 1; z-index: 1; }
.day.future { opacity: .55; }
.day .dot {
  position: absolute; top: 5px; right: 5px; width: 7px; height: 7px; border-radius: 50%; background: var(--
dot);
}
.day.lv1 { background: var(--lv1); }
.day.lv2 { background: var(--lv2); }
.day.lv3 { background: var(--lv3); color: #fff; }
.day.lv4 { background: var(--lv4); color: #fff; }
.day.lv3 .num, .day.lv4 .num { color: #fff !important; }
.day .goal-check { position: absolute; bottom: 3px; left: 5px; font-size: .62rem; }


.legend { flex: none; display: flex; align-items: center; gap: 4px; margin-top: 8px; font-size: .75rem;
color: var(--muted); flex-wrap: wrap; }
.legend .lv { display: inline-block; width: 12px; height: 12px; border-radius: 3px; }
.lv0 { background: var(--lv0); } .lv1 { background: var(--lv1); } .lv2 { background: var(--lv2); }
.lv3 { background: var(--lv3); } .lv4 { background: var(--lv4); }
.legend-dot { margin-left: auto; display: inline-flex; align-items: center; gap: 4px; }
.legend-dot .dot { display: inline-block; width: 8px; height: 8px; border-radius: 50%; background: var(--
dot); }


/* 통계 */
.screen-body h3 { margin: 4px 0 8px; font-size: .95rem; }
.screen-body h3:not(:first-child) { margin-top: 22px; }
.chart svg { width: 100%; height: auto; display: block; }
.chart .empty-msg, .empty-msg { color: var(--muted); font-size: .85rem; padding: 18px 0; text-align:
center; }
.parts-bars { display: flex; flex-direction: column; gap: 6px; }
.part-bar { display: grid; grid-template-columns: 52px 1fr 32px; align-items: center; gap: 8px; font-
size: .82rem; }
.part-bar .track { background: var(--lv0); border-radius: 6px; height: 14px; overflow: hidden; }
.part-bar .fill { background: var(--primary); height: 100%; border-radius: 6px; }
.part-bar .cnt { text-align: right; color: var(--muted); }
.totals { display: grid; grid-template-columns: repeat(3, minmax(0, 1fr)); gap: 8px; }
.totals div { background: var(--primary-weak); border-radius: 10px; padding: 10px; text-align: center; font-
size: .78rem; color: var(--muted); }
.totals b { display: block; font-size: 1.05rem; color: var(--text); }


/* 목록 */
.rec {
  border-bottom: 1px solid var(--line); padding: 10px 2px; cursor: pointer;
}
.rec:last-child { border-bottom: none; }
.rec-top { display: flex; justify-content: space-between; align-items: baseline; gap: 8px; }
.rec-date { font-weight: 700; }
.rec-meta { font-size: .8rem; color: var(--muted); }
.rec-memo { font-size: .85rem; margin-top: 4px; white-space: pre-wrap; color: var(--text); opacity: .85;
  display: -webkit-box; -webkit-line-clamp: 2; -webkit-box-orient: vertical; overflow: hidden; }
.badge { display: inline-block; background: var(--primary-weak); color: var(--primary); border-radius: 6px;
padding: 1px 6px; font-size: .72rem; margin-right: 3px; font-weight: 600; }
.badge.score { background: #fff1de; color: #b4640a; }
```

```
/* 다이얼로그 */
dialog {
  border: none; border-radius: 18px; padding: 0; width: min(440px, calc(100vw - 24px));
  background: var(--card); color: var(--text); max-height: calc(100vh - 24px);
}
dialog::backdrop { background: rgba(10, 14, 22, .45); }
dialog form { padding: 18px; display: flex; flex-direction: column; gap: 12px; }
dialog h2 { margin: 0 0 2px; font-size: 1.15rem; }
dialog label, .record-form label { display: flex; flex-direction: column; gap: 4px; font-size: .85rem; color:
var(--muted); }
dialog input[type=number], dialog input[type=date], dialog textarea,
.record-form input[type=number], .record-form textarea {
  border: 1px solid var(--line); border-radius: 10px; padding: 9px 10px; background: var(--bg); width: 100%;
}
dialog textarea, .record-form textarea { resize: vertical; }
fieldset { border: 1px solid var(--line); border-radius: 12px; padding: 8px 10px 10px; margin: 0; }
legend { font-size: .85rem; color: var(--muted); padding: 0 4px; }


.minute-row { display: flex; gap: 6px; }
.minute-row input { flex: 1; }
.chip { border: 1px solid var(--line); background: var(--primary-weak); color: var(--primary); border-radius:
10px; padding: 0 12px; font-weight: 600; }


.part-checks { display: grid; grid-template-columns: repeat(4, 1fr); gap: 6px; }
.part-checks label { flex-direction: row; }
.part-checks input { display: none; }
.part-checks span {
  flex: 1; text-align: center; border: 1px solid var(--line); border-radius: 9px; padding: 6px 0; font-
size: .8rem; color: var(--text);
}
.part-checks input:checked + span { background: var(--primary); border-color: var(--primary); color: #fff; }


.score-row { display: flex; gap: 8px; align-items: flex-end; }
.score-row label { flex: 1; }
.score-total { font-size: .85rem; color: var(--muted); padding-bottom: 9px; white-space: nowrap; }
.score-total b { color: var(--text); font-size: 1.05rem; }


.moods { display: flex; justify-content: space-between; }
.moods label { flex-direction: row; }
.moods input { display: none; }
.moods span { font-size: 1.6rem; filter: grayscale(1); opacity: .45; padding: 2px 6px; border-radius: 10px; }
.moods input:checked + span { filter: none; opacity: 1; background: var(--primary-weak); }


.dialog-actions { display: flex; gap: 8px; margin-top: 4px; }
.dialog-actions .spacer { flex: 1; }
.dialog-actions button, .backup-row button {
  border: 1px solid var(--line); background: var(--card); border-radius: 10px; padding: 8px 14px;
}
.primary { background: var(--primary) !important; color: #fff !important; border-color: var(--
primary) !important; }
.danger { color: var(--danger) !important; }
.backup-row { display: flex; gap: 8px; flex-wrap: wrap; }
.hint { font-size: .75rem; color: var(--muted); margin: 8px 0 0; }


.toast {
  z-index: 100; position: fixed; left: 50%; bottom: 28px; transform: translateX(-50%) translateY(20px);
  background: #1d2330; color: #fff; padding: 9px 16px; border-radius: 20px; font-size: .88rem;
  width: max-content; max-width: calc(100vw - 32px); text-align: center;
  opacity: 0; pointer-events: none; transition: all .25s;
}
.toast.show { opacity: .95; transform: translateX(-50%) translateY(0); }


@media (max-width: 420px) {
  .home-main { padding: 8px 10px; }
  .stat-value { font-size: .9rem; }
  .day .min { font-size: .58rem; }
  .cal-grid { gap: 3px; }
}


/* 상단 버튼 묶음 */
.top-actions { display: flex; gap: 2px; align-items: center; }
.topbar .icon-btn { font-size: 1.3rem; }
```

```
/* ---------- 뽀모도로 ---------- */
.timer-wrap { display: flex; flex-direction: column; align-items: center; gap: 12px; padding: 4px 0; }


/* 타임타이머: 색상은 JS 가 .timer-wrap 에 --tt-main / --tt-frame / --tt-knob 로 지정 */
.timer-wrap { --tt-main: #e2432a; --tt-frame: #f8b49c; --tt-knob: #f9c3b1; }
.tt {
  width: min(300px, 74vw, 40dvh); aspect-ratio: 1; padding: 5.5%;
  background: var(--tt-frame); border-radius: 26%;
  box-shadow: inset 0 -6px 0 rgba(0,0,0,.08), 0 10px 24px rgba(0,0,0,.14);
  transition: background .6s;
}
.tt-face {
  position: relative; width: 100%; height: 100%; background: #fbfbf8; border-radius: 22%;
  box-shadow: inset 0 0 0 3px rgba(0,0,0,.05), inset 0 3px 8px rgba(0,0,0,.08);
}
.tt-face svg { width: 100%; height: 100%; display: block; touch-action: none; }
.tt.draggable svg { cursor: grab; }
.tt.dragging svg { cursor: grabbing; }
.tt-tick { stroke: #2b2b2b; stroke-linecap: round; }
.tt-num { font: 600 15px -apple-system, "Segoe UI", sans-serif; fill: #2b2b2b; }
.tt-wedge { fill: var(--tt-main); transition: fill .6s; }
.tt-hand { stroke: #fff; stroke-width: 3; stroke-linecap: round; filter: drop-shadow(0 1px 1px
rgba(0,0,0,.35)); }
.tt-knob { fill: var(--tt-knob); stroke: rgba(0,0,0,.12); stroke-width: 1; }
.tt-knob-shadow { fill: rgba(0,0,0,.18); }
:root[data-theme="dark"] .tt-face { background: #eeede8; }
@media (prefers-color-scheme: dark) { :root:not([data-theme="light"]) .tt-face { background: #eeede8; } }


.tt-readout { display: flex; align-items: baseline; gap: 10px; }
.ring-time { font-size: 2rem; font-weight: 700; font-variant-numeric: tabular-nums; letter-spacing: 1px; }
.ring-label { font-size: .8rem; font-weight: 700; color: #fff; background: var(--tt-main); padding: 3px 10px;
border-radius: 20px; align-self: center; transition: background .5s; }
.ring-cycle { font-size: .8rem; color: var(--muted); }
.tt-hint { margin: -8px 0 0; font-size: .75rem; color: var(--muted); }


.tt-colors { display: flex; gap: 10px; }
.tt-colors button {
  width: 28px; height: 28px; border-radius: 50%; border: 3px solid var(--card);
  box-shadow: 0 0 0 1px var(--line); padding: 0;
}
.tt-colors button[aria-checked="true"] { box-shadow: 0 0 0 2px var(--text); }


.timer-controls { display: flex; gap: 10px; align-items: center; }
.timer-controls button {
  border: 1px solid var(--line); background: var(--card); border-radius: 50px; padding: 10px 18px; font-
weight: 600;
}
.timer-controls .main-btn {
  background: var(--tt-main); border-color: var(--tt-main); color: #fff; padding: 12px 34px; font-size:
1.05rem;
}


.timer-opts { display: grid; grid-template-columns: repeat(3, 1fr); gap: 8px; width: 100%; max-width:
420px; }
.timer-opts label { display: flex; flex-direction: column; gap: 3px; font-size: .75rem; color: var(--muted);
text-align: center; }
.timer-opts input {
  border: 1px solid var(--line); border-radius: 10px; padding: 7px; background: var(--bg); text-align:
center; width: 100%;
}
.timer-check { display: flex; gap: 6px; align-items: center; font-size: .8rem; color: var(--muted); }


.session-head { display: flex; justify-content: space-between; align-items: baseline; margin-top: 18px; }
.session-head h3 { margin: 0 !important; }
.session-head span { font-size: .82rem; color: var(--muted); }


.sessions { display: flex; flex-direction: column; gap: 6px; margin-top: 8px; }
.session {
  display: grid; grid-template-columns: auto 1fr; gap: 10px; align-items: center;
  cursor: pointer; transition: transform .08s, filter .15s;
  background: var(--focus-weak); border-radius: 10px; padding: 8px 10px; font-size: .85rem;
}
```

```
.session .tomato { font-size: 1.1rem; }
.session .s-time { font-weight: 600; }
.session .s-note { color: var(--text); opacity: .85; font-size: .82rem; word-break: break-all; }
.session .empty-note { color: var(--muted); font-style: italic; }


.done-emoji { font-size: 2.4rem; text-align: center; margin: 0; }
.done-sub { text-align: center; color: var(--muted); font-size: .88rem; margin: -6px 0 0; }




/* 시험 일정 / D-day */
.exam-banner {
  flex: none; display: flex; align-items: center; justify-content: space-between; gap: 10px;
  background: var(--focus-weak); border-radius: 10px; padding: 6px 12px; margin-bottom: 6px; font-
size: .85rem;
}
.exam-banner.none { background: var(--bg); color: var(--muted); justify-content: center; font-size: .82rem; }
.exam-banner .dd { font-size: 1.2rem; font-weight: 800; color: var(--focus); white-space: nowrap; }
.day.exam { box-shadow: inset 0 0 0 2px var(--focus); opacity: 1; }
.day .exam-tag {
  align-self: center; background: var(--focus); color: #fff; border-radius: 6px;
  font-size: .62rem; font-weight: 700; padding: 1px 4px; white-space: nowrap; line-height: 1.3;
}
.day.exam.past .exam-tag { background: var(--muted); }
.legend .exam-sq { display: inline-block; width: 10px; height: 10px; border-radius: 3px; box-shadow: inset 0
0 0 2px var(--focus); margin-left: 6px; }
.exam-check { flex-direction: row !important; align-items: center; gap: 8px !important; color: var(--
text) !important; font-weight: 600;
  background: var(--focus-weak); border-radius: 10px; padding: 8px 10px; }
.exam-check input { width: 18px; height: 18px; accent-color: var(--focus); }


/* ---------- 타이머 진행 중 칩 (홈 상단) ---------- */
.timer-chip {
  border: none; border-radius: 20px; padding: 4px 10px; margin-right: 4px;
  background: var(--tt-main, #e2432a); color: #fff; font-weight: 700; font-size: .85rem;
  font-variant-numeric: tabular-nums; box-shadow: 0 0 0 2px rgba(255,255,255,.6);
}


/* ---------- 날짜 바텀시트 ---------- */
.sheet { position: fixed; inset: 0; z-index: 50; pointer-events: none; }
.sheet.open { pointer-events: auto; }
.sheet-backdrop { position: absolute; inset: 0; background: rgba(10, 14, 22, .4); opacity: 0; transition:
opacity .25s; }
.sheet.open .sheet-backdrop { opacity: 1; }
.sheet-panel {
  position: absolute; left: 0; right: 0; bottom: 0; margin: 0 auto; max-width: 560px;
  background: var(--card); border-radius: 20px 20px 0 0;
  padding: 8px 16px; padding-bottom: max(18px, env(safe-area-inset-bottom));
  box-shadow: 0 -8px 30px rgba(0,0,0,.18);
  transform: translateY(105%); transition: transform .28s cubic-bezier(.2,.8,.2,1);
}
.sheet.open .sheet-panel { transform: translateY(0); }
.sheet-handle { width: 40px; height: 5px; border-radius: 3px; background: var(--line); margin: 0 auto 10px; }
.sheet-head { display: flex; align-items: center; gap: 8px; flex-wrap: wrap; }
.sheet-head h2 { margin: 0; font-size: 1.1rem; }
.sheet-tag { display: inline-block; font-size: .72rem; font-weight: 700; border-radius: 6px; padding: 2px
7px; margin-right: 4px; }
.sheet-tag.today { background: var(--primary-weak); color: var(--primary); }
.sheet-tag.exam { background: var(--focus); color: #fff; }
.sheet-info { margin: 6px 0 14px; font-size: .85rem; color: var(--muted); }
.sheet-actions { display: grid; grid-template-columns: repeat(3, minmax(0, 1fr)); gap: 10px; }
.sheet-btn {
  border: none; border-radius: 16px; padding: 14px 6px 12px;
  display: flex; flex-direction: column; align-items: center; gap: 3px;
  transition: transform .08s;
}
.sheet-btn:active { transform: scale(.96); }
.sheet-btn .ico { font-size: 1.7rem; line-height: 1.1; }
.sheet-btn .lbl { font-weight: 700; font-size: .95rem; }
.sheet-btn .sub { font-size: .7rem; opacity: .75; }
.sheet-btn.study { background: var(--focus); color: #fff; }
.sheet-btn.stats { background: var(--primary-weak); color: var(--primary); }
.sheet-btn.record { background: var(--break-weak); color: var(--break); }


/* ---------- 전체 화면 페이지 (타이머/통계/기록) ---------- */
```

```
.screen {
  position: fixed; inset: 0; z-index: 60; background: var(--bg);
  display: flex; flex-direction: column;
  transform: translateX(100%); visibility: hidden;
  transition: transform .28s cubic-bezier(.2,.8,.2,1), visibility 0s linear .28s;
}
.screen.open { transform: none; visibility: visible; transition: transform .28s cubic-bezier(.2,.8,.2,1); }
.screen-head {
  flex: none; display: flex; align-items: center; gap: 4px;
  padding: 8px 8px; padding-top: max(8px, env(safe-area-inset-top));
  background: var(--card); border-bottom: 1px solid var(--line);
}
.screen-head h2 { flex: 1; margin: 0; text-align: center; font-size: 1.05rem; }
.screen-head .icon-btn { font-size: 1.8rem; width: 44px; }
.screen-head-spacer { width: 44px; }
.screen-body {
  flex: 1; min-height: 0; overflow-y: auto; -webkit-overflow-scrolling: touch;
  width: 100%; max-width: 720px; margin: 0 auto;
  padding: 14px 16px; padding-bottom: max(24px, env(safe-area-inset-bottom));
}


/* 기록 화면 폼 */
.record-form { display: flex; flex-direction: column; gap: 12px; }
.record-actions {
  position: sticky; bottom: -24px; display: flex; gap: 8px; padding: 10px 0;
  background: linear-gradient(transparent, var(--bg) 30%);
}
.record-actions .spacer { flex: 1; }
.record-actions button { border: 1px solid var(--line); background: var(--card); border-radius: 12px;
padding: 11px 22px; font-weight: 600; }


/* 타이머 시간 설정 접기 */
.timer-settings { width: 100%; max-width: 420px; }
.timer-settings summary { text-align: center; font-size: .82rem; color: var(--muted); cursor: pointer; list-
style: none; padding: 4px; }
.timer-settings summary::-webkit-details-marker { display: none; }
.timer-settings summary::after { content: " \25BE"; }
.timer-settings[open] summary::after { content: " \25B4"; }
.timer-settings .timer-opts { margin: 6px 0 8px; }
.timer-settings .timer-check { justify-content: center; }


/* 시트 첫 버튼: 지난 날짜 = 공부한 시간 보기 */
.sheet-btn.study.log { background: #f0a02b; color: #fff; }
/* 미래 날짜: "아직 오픈 전" 공사 중 테이프 무늬 */
.sheet-btn.study.closed {
  color: #3b3200;
  background: repeating-linear-gradient(-45deg, #ffd43b 0 12px, #fff3bf 12px 24px);
}
.sheet-btn.closed .sub { font-style: italic; }
.sheet-btn.shake { animation: shake .4s; }
@keyframes shake {
  20% { transform: translateX(-6px) rotate(-2deg); }
  40% { transform: translateX(6px) rotate(2deg); }
  60% { transform: translateX(-4px); }
  80% { transform: translateX(4px); }
}


/* 기록 화면: 학습 시간 (타이머 기록, 읽기 전용) */
.minutes-readonly {
  display: grid; grid-template-columns: 1fr auto; align-items: baseline; gap: 2px 10px;
  background: var(--card); border: 1px solid var(--line); border-radius: 12px; padding: 10px 12px;
}
.minutes-readonly .mr-label { font-size: .85rem; color: var(--muted); }
.minutes-readonly .mr-label small { font-size: .72rem; }
.minutes-readonly .mr-value { font-size: 1.2rem; font-weight: 800; grid-row: span 2; align-self: center; }
.minutes-readonly .mr-sub { font-size: .75rem; color: var(--muted); }


/* 공부한 시간 화면 */
.log-parts { display: flex; flex-wrap: wrap; gap: 4px; margin-top: 10px; }
.log-parts .badge { font-size: .78rem; padding: 3px 8px; }


/* 바텀시트가 열려 있을 땐 토스트를 시트 위로 */
```

```
.sheet.open ~ .toast { bottom: 230px; }


/* ---------- 한 달 ↔ 1년 보기 ---------- */
/* 캘린더 위에서 브라우저 확대 대신 두 손가락 제스처를 쓰도록 */
.panel.calendar { touch-action: pan-x pan-y; }
.cal-month { flex: 1; min-height: 0; display: flex; flex-direction: column; }
.year-view { flex: 1; min-height: 0; display: flex; align-items: center; justify-content: center; overflow:
hidden; }
.year-grid { display: grid; justify-content: center; align-content: center; }


.mini-month {
  border: none; background: none; padding: 0; border-radius: 6px;
  display: flex; flex-direction: column; align-items: flex-start; color: var(--text);
}
.mini-month:active { opacity: .7; }
.mm-title { height: 18px; font-size: .72rem; font-weight: 700; display: flex; align-items: baseline; gap:
4px; }
.mm-title small { font-weight: 600; color: var(--muted); font-size: .62rem; }
.mini-month.now .mm-title { color: var(--primary); }
.mini-days { display: grid; grid-template-columns: repeat(7, var(--sq)); grid-auto-rows: var(--sq); gap:
var(--md-gap); }
.md { display: block; border-radius: 2px; background: var(--lv0); }
.md.empty { background: transparent; }
.md.lv1 { background: var(--lv1); } .md.lv2 { background: var(--lv2); }
.md.lv3 { background: var(--lv3); } .md.lv4 { background: var(--lv4); }
.md.future { opacity: .45; }
.md.today { outline: 1.5px solid var(--primary); outline-offset: 0; }
.md.exam { box-shadow: inset 0 0 0 1.5px var(--focus); opacity: 1; }


.zoom-from-big { animation: zoomFromBig .25s ease-out; }
.zoom-from-small { animation: zoomFromSmall .25s ease-out; }
@keyframes zoomFromBig { from { transform: scale(1.15); opacity: 0; } }
@keyframes zoomFromSmall { from { transform: scale(.85); opacity: 0; } }


/* 집중 시간 입력 하나만 */
.timer-opts.single { grid-template-columns: minmax(0, 160px); justify-content: center; }
/* 휴식 중: 시간이 올라가는 것을 강조 */
.timer-wrap.on-break .ring-time { color: var(--tt-main); }
:root[data-theme="dark"] .timer-wrap.on-break .ring-time { color: var(--tt-frame); }
@media (prefers-color-scheme: dark) { :root:not([data-theme="light"]) .timer-wrap.on-break .ring-time
{ color: var(--tt-frame); } }
.session .rest { font-weight: 500; color: var(--break); font-size: .78rem; margin-left: 2px; }


/* 기록 칸을 눌러 편집 */
.session:hover { filter: brightness(.97); }
.session:active { transform: scale(.98); }
.session:focus-visible { outline: 2px solid var(--primary); outline-offset: 2px; }


/* 미래 날짜: "아직 오픈 전" 버튼 하나만 크게 */
.sheet-actions.single { grid-template-columns: 1fr; }
.sheet-actions.single .sheet-btn { flex-direction: row; justify-content: center; gap: 10px; padding: 16px; }
.sheet-actions.single .sheet-btn .ico { font-size: 1.6rem; }
.sheet-actions.single .sheet-btn .lbl { font-size: 1.05rem; }
.sheet-exam {
  display: block; width: 100%; margin-top: 10px; padding: 10px; border-radius: 12px;
  border: 1px dashed var(--focus); background: none; color: var(--focus); font-weight: 600; font-
size: .85rem;
}


/* 토요일(이 주 요약): 파란 계열 / 매달 마지막 날(이 달 요약): 빨간 계열 통계 버튼 */
.sheet-btn.stats.review-week {
  background: linear-gradient(160deg, #74c0fc, #1c7ed6); color: #fff;
  box-shadow: 0 4px 14px rgba(28, 126, 214, .35);
}
.sheet-btn.stats.review-month {
  background: linear-gradient(160deg, #ff8787, #e03131); color: #fff;
  box-shadow: 0 4px 14px rgba(224, 49, 49, .35);
}
.sheet-btn.stats.review-week .sub, .sheet-btn.stats.review-month .sub { opacity: .95; font-weight: 700; }


.week-chart { margin-top: 12px; }
```

```
/* ---------- 팁 / 응원 문구 배너 (누를 때마다 바뀜) ---------- */
.exam-banner { cursor: pointer; user-select: none; -webkit-user-select: none; min-height: 2.9em; }
.exam-banner:active { transform: scale(.985); }
.exam-banner.tip-mode { background: var(--primary-weak); }
.exam-banner .tip-text { flex: 1; min-width: 0; font-size: .8rem; line-height: 1.35; color: var(--text); }
.tip-kind {
  display: inline-block; font-size: .66rem; font-weight: 800; border-radius: 6px; padding: 1px 6px; margin-
right: 6px;
  vertical-align: 1px; background: var(--card); color: var(--primary);
}
.tip-kind.quote { color: #e8590c; }
.tip-kind.me { color: var(--focus); }
.exam-banner.tip-swap > * { animation: tipSwap .3s ease-out; }
@keyframes tipSwap { from { opacity: 0; transform: translateY(4px); } }


/* 시험 카운트다운 카드: 일·시·분·초 한 줄 */
.stat-exam #statDday { display: block; width: 100%; text-align: center; overflow: hidden; font-variant-
numeric: tabular-nums; }
.stat-exam .cd { display: inline-block; white-space: nowrap; font-size: .86rem; }
.stat-exam .cd small { font-size: .72em; font-weight: 600; color: var(--muted); margin: 0 .08em 0 .04em; }
.summary .stat { justify-content: center; }


/* 설정: 계정 칸 */
.account-box { display: flex; flex-direction: column; gap: 6px; }
.account-status { margin: 0; font-size: .88rem; color: var(--text); }
.account-status small { color: var(--muted); }
.account-btn {
  border: 1px solid var(--line); background: var(--card); border-radius: 10px; padding: 9px 14px; font-
weight: 600;
}
.account-btn:disabled { opacity: .5; cursor: not-allowed; }
.account-btn.ghost { color: var(--muted); }
.account-box .hint { margin: 0; }
```

www/service-worker.js

```
const CACHE_NAME = "daily-toeic-v34";
const ASSETS = [
  "./", "./index.html", "./style.css", "./manifest.json", "./icon.svg",
  "./js/core/utils.js", "./js/core/audio.js", "./js/core/store.js", "./js/core/router.js", "./js/core/cloud-
config.js", "./js/core/firebase-provider.js", "./js/core/account.js", "./js/core/cloud-sync.js",
  "./js/features/theme.js", "./js/features/calendar-zoom.js", "./js/features/year-view.js",
"./js/features/calendar.js", "./js/features/tip-banner.js", "./js/features/day-sheet.js",
  "./js/features/session-dialog.js", "./js/features/study-log.js", "./js/features/record.js", "./sounds/wind-
up.js", "./js/features/wind-sound.js", "./js/features/timer-dial.js",
  "./js/features/sfx.js", "./js/features/native-notifier.js", "./js/features/pomodoro.js",
"./js/features/timer-screen.js", "./js/features/stats.js",
  "./js/features/settings.js", "./js/features/account-panel.js", "./js/app.js",
];


self.addEventListener("install", (e) => {
  e.waitUntil(caches.open(CACHE_NAME).then((c) => c.addAll(ASSETS)));
  self.skipWaiting();
});


self.addEventListener("activate", (e) => {
  e.waitUntil(
    caches.keys().then((keys) => Promise.all(keys.filter((k) => k !== CACHE_NAME).map((k) =>
caches.delete(k))))
  );
  self.clients.claim();
});


self.addEventListener("fetch", (e) => {
  if (e.request.method !== "GET") return;
  // 앱 파일만 캐시. 로그인/클라우드 같은 다른 사이트 요청은 브라우저가 직접 처리
  if (new URL(e.request.url).origin !== location.origin) return;
  // 네트워크 우선, 오프라인이면 캐시 사용
  e.respondWith(
```

```
    fetch(e.request)
      .then((res) => {
        if (res.ok && new URL(e.request.url).origin === location.origin) {
          const copy = res.clone();
          caches.open(CACHE_NAME).then((c) => c.put(e.request, copy));
        }
        return res;
      })
      .catch(() => caches.match(e.request))
  );
});
```

www/manifest.json

```
{
  "name": "Daily TOEIC 스터디 캘린더",
  "short_name": "Daily TOEIC",
  "start_url": "./index.html",
  "display": "standalone",
  "background_color": "#f5f7fb",
  "theme_color": "#2f6fed",
  "icons": [
    { "src": "icon.svg", "sizes": "any", "type": "image/svg+xml", "purpose": "any maskable" }
  ]
}
```

firestore.rules

```
rules_version = '2';


// Daily TOEIC 클라우드 기록 보안 규칙
// 로그인한 사용자는 자기 자신의 기록(users/{자기 uid} 아래)만 읽고 쓸 수 있다
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{uid} {
      allow read, write: if request.auth != null && request.auth.uid == uid;


      match /days/{day} {
        allow read, write: if request.auth != null && request.auth.uid == uid
          && day.matches('^[0-9]{4}-[0-9]{2}-[0-9]{2}$');
      }
    }


    // 그 밖의 모든 경로는 막음
    match /{document=**} {
      allow read, write: if false;
    }
  }
}
```

A.7 소리 데이터

www/sounds/wind-up.js

```
// 타이머 조절 효과음: 8mm 무비 카메라 태엽 감는 소리 중 "뜨르륵" 구간 (원본 0.47~0.80 초, 모노 22kHz WAV)
// index.html 을 파일로 열어도 재생되도록 base64 로 넣어 둠
window.DT = window.DT || {};
DT.SOUNDS = DT.SOUNDS || {};
DT.SOUNDS.windUp =
"UklGRlw7AABXQVZFZm10IBAAAAABAAEAIlYAAESsAAACABAATElTVFYCAABJTkZPSUFSVPMAAABQdWJsaWMgTGFpcnNpZGUgSW5zdXJhbmNl
IEdyb3VwLCBJbmMuIC0gTG …(생략)
```